

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT THÀNH PHỐ HỒ CHÍ MINH  
KHOA CƠ KHÍ ĐỘNG LỰC

**ĐỒ ÁN TỐT NGHIỆP**

**THIẾT KẾ HỆ THỐNG ĐA TÁC VỤ  
TRIỂN KHAI TRÊN JETSON NANO ỨNG DỤNG  
CHO XE TỰ HÀNH**

SVTH : ĐỖ TRẦN NHẬT TUỜNG  
MSSV: 18145485  
GVHD: ThS. NGUYỄN TRUNG HIẾU

Tp. Hồ Chí Minh, tháng 02 năm 2023

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT THÀNH PHỐ HỒ CHÍ MINH  
KHOA CƠ KHÍ ĐỘNG LỰC

**ĐỒ ÁN TỐT NGHIỆP**

Chuyên ngành: Công nghệ Kỹ thuật ô tô

Tên đề tài

**THIẾT KẾ HỆ THỐNG ĐA TÁC VỤ  
TRIỂN KHAI TRÊN JETSON NANO ỨNG DỤNG  
CHO XE TỰ HÀNH**

SVTH: ĐỖ TRẦN NHẬT TƯỜNG  
MSSV: 18145485  
GVHD: ThS. NGUYỄN TRUNG HIẾU

Tp. Hồ Chí Minh, tháng 02 năm 2023

TP. Hồ Chí Minh, ngày 11 tháng 10 năm 2022

## NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

Họ tên sinh viên: 1. Đỗ Trần Nhật Tường MSSV: 18145485  
(E-mail: [18145485@student.hcmute.edu.vn](mailto:18145485@student.hcmute.edu.vn)) Điện thoại: 0908632205)

Ngành: Công nghệ kỹ thuật ô tô

Khóa: K18

Lớp: 181452A

### 1. Tên đề tài

THIẾT KẾ HỆ THỐNG ĐA TÁC VỤ TRIỂN KHAI TRÊN JETSON NANO ỨNG DỤNG CHO XE TỰ HÀNH

### 2. Nhiệm vụ đề tài

- **Nhiệm vụ 1:** Khảo sát đề tài, tham khảo tài liệu, đọc và tóm tắt đưa ra hướng và lựa chọn đề tài.
- **Nhiệm vụ 2:** Viết đề cương tóm tắt các yêu cầu của đề tài, thiết kế luồng xử lý của hệ thống.
- **Nhiệm vụ 3:** Nghiên cứu model phát hiện vật thể, và dựa vào model đó thiết kế nhánh phân đoạn làn đường.
- **Nhiệm vụ 4:** Tiến hành thi công chương trình hệ thống và triển khai lên phần cứng Jetson Nano.
- **Nhiệm vụ 5:** Kiểm tra thực nghiệm hệ thống, đánh giá và hiệu chỉnh chức năng hệ thống.
- **Nhiệm vụ 6:** Viết luận văn báo cáo đề tài.

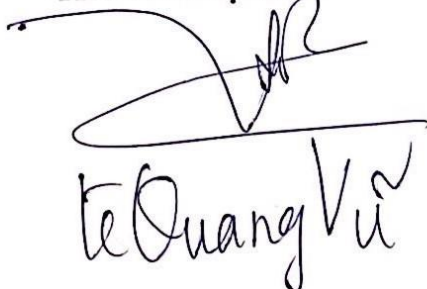
### 3. Sản phẩm của đề tài

Module AI ứng dụng cho xe tự hành

4. Ngày giao nhiệm vụ đề tài: 30/10/2022

5. Ngày hoàn thành nhiệm vụ: 13/02/2023

TRƯỞNG BỘ MÔN



Trương Bộ Môn

CÁN BỘ HƯỚNG DẪN



Nguyễn Tây Hiền

Bộ môn: Điện tử ô tô

**PHIẾU NHẬN XÉT ĐỒ ÁN TỐT NGHIỆP**  
(Dành cho giảng viên hướng dẫn)

Họ và tên sinh viên: Đỗ Trần Nhật Tường MSSV: 18145485 Hội đồng:.....

Tên đề tài: THIẾT KẾ HỆ THỐNG ĐA TÁC VỤ TRIỂN KHAI TRÊN JETSON NANO ỨNG DỤNG CHO XE TỰ HÀNH

Ngành đào tạo: Công nghệ Kỹ thuật Ô tô

Họ và tên GV hướng dẫn: ThS. Nguyễn Trung Hiếu

**Ý KIẾN NHẬN XÉT**

**1. Nhận xét về tinh thần, thái độ làm việc của sinh viên (không đánh máy)**

Sinh viên có tinh thần làm việc cầu tiến, biết lắng nghe góp ý từ GVHD. Chủ động tìm hiểu về nghiên cứu.  
Mặc dù thực hiện đề tài 1 mình nhưng đã nỗ lực có kết quả tốt.

**2. Nhận xét về kết quả thực hiện của ĐATN (không đánh máy)**

**2.1. Kết cấu, cách thức trình bày ĐATN:**

- Tốt  
- Thuộc yêu cầu của khoa.  
- Thẩm mỹ, sạch sẽ.

**2.2. Nội dung đồ án:**

(Cơ sở lý luận, tính thực tiễn và khả năng ứng dụng của đồ án, các hướng nghiên cứu có thể tiếp tục phát triển)

Đồ án có cơ sở về lý luận chặt chẽ. Các phương pháp thực hiện, thực xây dựng bài bản.  
Khả năng ứng dụng của đề án khá lớn mà phạm vi thực hiện trên sa bàn, thí nghiệm thực tế.

**2.3. Kết quả đạt được:**

Các kết quả của phương pháp được sinh viên ứng dụng.

Đào tạo của lái xe là học và đạt được những kỹ năng lái xe.

2.4. Những tồn tại (nếu có):

.....

.....

.....

.....

.....

3. Đánh giá:

| TT | Mục đánh giá                                                                                                           | Điểm tối đa | Điểm đạt được |
|----|------------------------------------------------------------------------------------------------------------------------|-------------|---------------|
| 1. | <b>Hình thức và kết cấu ĐATN</b>                                                                                       | 30          | 20            |
|    | Đúng format với đầy đủ cả hình thức và nội dung của các mục                                                            | 10          |               |
|    | Mục tiêu, nhiệm vụ, tổng quan của đề tài                                                                               | 10          |               |
|    | Tính cấp thiết của đề tài                                                                                              | 10          |               |
| 2. | <b>Nội dung ĐATN</b>                                                                                                   | 50          | 49            |
|    | Khả năng ứng dụng kiến thức toán học, khoa học và kỹ thuật, khoa học xã hội...                                         | 5           |               |
|    | Khả năng thực hiện/phân tích/tổng hợp/đánh giá                                                                         | 10          |               |
|    | Khả năng thiết kế chế tạo một hệ thống, thành phần, hoặc quy trình đáp ứng yêu cầu đưa ra với những ràng buộc thực tế. | 15          |               |
|    | Khả năng cải tiến và phát triển                                                                                        | 15          |               |
|    | Khả năng sử dụng công cụ kỹ thuật, phần mềm chuyên ngành...                                                            | 5           |               |
| 3. | <b>Đánh giá về khả năng ứng dụng của đề tài</b>                                                                        | 10          | 10            |
| 4. | <b>Sản phẩm cụ thể của ĐATN</b>                                                                                        | 10          | 10            |
|    | <b>Tổng điểm</b>                                                                                                       | 100         | 99/100        |

4. Kết luận:

- ☒ Được phép bảo vệ
- ☐ Không được phép bảo vệ

TP.HCM, ngày 12 tháng 02 năm 2023

Giảng viên hướng dẫn

((Ký, ghi rõ họ tên))

  
Nguyễn Thụy Hiền

Bộ môn: Điện tử ô tô

## PHIẾU NHẬN XÉT ĐỒ ÁN TỐT NGHIỆP

(Dành cho giảng viên phản biện)

Họ và tên sinh viên: Đỗ Trần Nhật Tường MSSV: 18145485 Hội đồng:.....

Tên đề tài: THIẾT KẾ HỆ THỐNG ĐA TÁC VỤ TRIỂN KHAI TRÊN JETSON NANO ỨNG DỤNG CHO XE TỰ HÀNH

Ngành đào tạo: Công nghệ Kỹ thuật Ô tô

Họ và tên GV phản biện: (Mã GV) .....

### Ý KIẾN NHẬN XÉT

**1. Kết cấu, cách thức trình bày ĐATN:**

Kết cấu, cách thức trình bày rõ ràng

**2. Nội dung đồ án:**

(Cơ sở lý luận, tính thực tiễn và khả năng ứng dụng của đồ án, các hướng nghiên cứu có thể tiếp tục phát triển)

Đề tài có cơ sở lý luận chặt chẽ từ cơ sở lý thuyết đến nghiên cứu thực nghiệm.

**3. Kết quả đạt được:**

Đồng ý với GVHD

**4. Những thiếu sót và tồn tại của ĐATN:**

**5. Câu hỏi:**

6. **Đánh giá:**

| TT | Mục đánh giá                                                                                                            | Điểm tối đa | Điểm đạt được |
|----|-------------------------------------------------------------------------------------------------------------------------|-------------|---------------|
| 1. | <b>Hình thức và kết cấu ĐATN</b>                                                                                        | 30          | 30            |
|    | Đúng format với đầy đủ cả hình thức và nội dung của các mục                                                             | 10          | 10            |
|    | Mục tiêu, nhiệm vụ, tổng quan của đề tài                                                                                | 10          | 10            |
|    | Tính cấp thiết của đề tài                                                                                               | 10          | 10            |
| 2. | <b>Nội dung ĐATN</b>                                                                                                    | 50          | 48            |
|    | Khả năng ứng dụng kiến thức toán học, khoa học và kỹ thuật, khoa học xã hội...                                          | 5           | 5             |
|    | Khả năng thực hiện/phân tích/tổng hợp/đánh giá                                                                          | 10          | 10            |
|    | Khả năng thiết kế, chế tạo một hệ thống, thành phần, hoặc quy trình đáp ứng yêu cầu đưa ra với những ràng buộc thực tế. | 15          | 15            |
|    | Khả năng cải tiến và phát triển                                                                                         | 15          | 15            |
|    | Khả năng sử dụng công cụ kỹ thuật, phần mềm chuyên ngành...                                                             | 5           | 5             |
| 3. | <b>Đánh giá về khả năng ứng dụng của đề tài</b>                                                                         | 10          | 10            |
| 4. | <b>Sản phẩm cụ thể của ĐATN</b>                                                                                         | 10          | 10            |
|    | <b>Tổng điểm</b>                                                                                                        | 100         | 98            |

7. **Kết luận:**

- ☒ Được phép bảo vệ  
☐ Không được phép bảo vệ

TP.HCM, ngày 23 tháng 02 năm 2023

Giảng viên phản biện

(Ký, ghi rõ họ tên)

  
 Nguyễn Thanh Tuyền

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT  
THÀNH PHỐ HỒ CHÍ MINH  
KHOA CƠ KHÍ ĐỘNG LỰC

## XÁC NHẬN HOÀN THÀNH ĐỒ ÁN

Tên đề tài: THIẾT KẾ HỆ THỐNG ĐA TÁC VỤ TRIỂN KHAI TRÊN JETSON  
NANO ỨNG DỤNG CHO XE TỰ HÀNH

Họ và tên Sinh viên: Đỗ Trần Nhật Tường

MSSV: 18145485

Ngành: Công nghệ Kỹ thuật ô tô

Sau khi tiếp thu và điều chỉnh theo góp ý của Giảng viên hướng dẫn, Giảng viên phản biện và các thành viên trong Hội đồng bảo vệ. Đồ án tốt nghiệp đã được hoàn chỉnh đúng theo yêu cầu về nội dung và hình thức.

Chủ tịch Hội đồng:

Nguyễn Văn Tuấn

[Signature]

Giảng viên hướng dẫn:

NGUYỄN TRUNG HIỆU

[Signature]

Giảng viên phản biện:

Nguyễn Thanh Tuyền

[Signature]

Tp. Hồ Chí Minh, ngày      tháng 02 năm 2023

## LỜI CẢM ƠN

Lời đầu tiên em biết rằng sự hỗ trợ của gia đình em là nguồn cảm hứng cũng như nguồn động lực tiếp thêm sức mạnh cho em để hoàn thành đồ án tốt nghiệp. Em hy vọng rằng bản thân sẽ tiếp tục giữ vững niềm tin cùng với sự hỗ trợ của gia đình trong tương lai sẽ làm gia đình em có thể tự hào về những gì em đạt được. Em cũng xin cảm ơn đến quý Thầy/Cô trường Đại học Sư Phạm Kỹ Thuật Thành Phố Hồ Chí Minh không chỉ dạy cho em nhiều kiến thức quý báu mà còn hướng dẫn cho em biết thêm nhiều điều trong quá trình học tập. Chính vì những điều đó cũng đã giúp em hoàn thành tốt đề tài của mình.

Đặc biệt, em xin bày tỏ lòng biết ơn chân thành sâu sắc đến Thầy Nguyễn Trung Hiếu – Trường Đại Học Sư Phạm Kỹ Thuật Thành phố Hồ Chí Minh đã trực tiếp hướng dẫn, giúp đỡ, tạo điều kiện cho em trong quá trình thực hiện đồ án. Nhờ những lời khuyên, góp ý và định hướng của Thầy là động lực lớn giúp em vượt qua những khó khăn và tiến bộ hơn trong nghiên cứu của mình. Kiến thức và kinh nghiệm của thầy không những giúp em hoàn thành tốt đề tài mà còn là tấm gương để em học tập và noi theo trên con đường sau này.

Ngoài ra, em cũng xin gửi lời cảm ơn sâu sắc đến Thầy Trần Vũ Hoàng – Trường Đại Học Sư Phạm Kỹ Thuật Thành phố Hồ Chí Minh đã dành thời gian và tâm huyết để hướng dẫn và hỗ trợ cho em trong quá trình làm đồ án. Nhờ có sự chỉ dẫn và giúp đỡ tận tình của Thầy, em đã hoàn thành đồ án một cách tốt nhất và học hỏi được rất nhiều kiến thức quý giá thông qua những cuộc thi mà thầy hướng dẫn. Đối với em, Thầy là một người thầy tận tâm, am hiểu sâu rộng về lĩnh vực chuyên môn, luôn sẵn sàng giải đáp mọi thắc mắc của em trong suốt quá trình làm đồ án.

Cảm ơn các bạn phòng lab đã tận tình giúp đỡ em, mặc dù mỗi nhóm một đề tài nhưng mỗi người trên lab lại có một thế mạnh riêng từ đó có thể trao đổi và bổ sung những kiến thức còn thiếu cho nhau.

Một lần nữa em xin cảm ơn quý Thầy/Cô, và chúc quý Thầy/Cô luôn vui vẻ và có sức khỏe dồi dào để dẫn dắt những thế hệ sau ngày một tốt hơn.

## TÓM TẮT

Trong những năm gần đây, với sự phát triển mạnh mẽ của trí tuệ nhân tạo, đặc biệt là học sâu, ngành công nghệ thông tin đã gặt hái được rất nhiều thành công. Có thể kể đến một ứng dụng điển hình của các mô hình học sâu là “XE TỰ HÀNH”. Tuy nhiên, với ứng dụng này lại gặp nhiều xung đột về quy tắc đạo đức AI vì thế gần đây có nhiều phương pháp nghiên cứu về xe tự hành đã được đầu tư và phát triển để các mô hình học sâu có hiệu suất và độ chính xác cao có thể kể đến như phân đoạn làn đường, phát hiện đối tượng, nhận dạng biển báo giao thông, nhận dạng bối cảnh và dự đoán điều khiển góc lái. Ngoài ra, xe tự hành còn cho biết thông tin trạng thái của xe từ CANBUS và được trang bị thêm các cảm biến có độ chính xác và thời gian đáp ứng cao như LIDAR, cảm biến Laser,... nhằm mục đích có thêm nhiều thông tin hơn để tăng mức độ tin cậy cho hệ thống. Mặc dù đạt được hiệu suất cao, nhưng xử lý từng nhiệm vụ riêng lẻ mất nhiều thời gian hơn so với việc giải quyết tất cả nhiệm vụ cùng một lúc. Trong khi đó, các ứng dụng xe tự hành đòi hỏi phải chạy được trên thời gian thực và độ chính xác cao trên một phần cứng hạn chế. Do đó, vấn đề cấp bách là thiết kế một hệ thống đảm bảo cả hai yêu cầu này trong thực tế. Vì thế trong bài báo cáo này tôi đề xuất một hệ thống đa tác vụ có thể giảm thời gian xử lý bằng cách xử lý nhiều nhiệm vụ cùng một lúc và hiệu năng cũng được cải thiện nhờ có thêm nhiều thông tin để xử lý.

Để thực hiện được đề tài, tôi chia hệ thống thành ba phần nghiên cứu chính:

- Mô hình học sâu (phân đoạn làn đường, phát hiện đối tượng, nhận dạng bối cảnh) và được đánh giá qua ba giai đoạn: mô phỏng, xe mô hình, xe Renault Twizy.
- LIDAR (phát hiện vật cản).
- CANBUS (thông tin trạng thái của xe).

Sau thời gian nghiên cứu và thực hiện, hệ thống đã hoàn thành và thực nghiệm.

# MỤC LỤC

|                                                    |          |
|----------------------------------------------------|----------|
| LỜI CẢM ƠN .....                                   | i        |
| TÓM TẮT .....                                      | ii       |
| MỤC LỤC .....                                      | iii      |
| DANH MỤC CÁC CHỮ VIẾT TẮT VÀ KÝ HIỆU .....         | ix       |
| DANH MỤC HÌNH .....                                | x        |
| DANH MỤC BẢNG.....                                 | xiv      |
| <b>Chương 1. TỔNG QUAN ĐỀ TÀI .....</b>            | <b>1</b> |
| 1.1. Lý do chọn đề tài.....                        | 1        |
| 1.2. Tình hình nghiên cứu trong và ngoài nước..... | 1        |
| 1.2.1. Tình hình nghiên cứu trong nước .....       | 1        |
| 1.2.2. Tình hình nghiên cứu ngoài nước.....        | 2        |
| 1.3. Mục tiêu đề tài .....                         | 3        |
| 1.4. Phương pháp nghiên cứu .....                  | 3        |
| 1.5. Đối tượng nghiên cứu .....                    | 4        |
| 1.6. Giới hạn đề tài.....                          | 4        |
| 1.7. Bố cục đề tài.....                            | 4        |
| <b>Chương 2. CƠ SỞ LÝ THUYẾT.....</b>              | <b>5</b> |
| 2.1. Mô hình học sâu.....                          | 5        |
| 2.1.1. Môi trường .....                            | 5        |
| 2.1.1.1. Hệ điều hành UBUNTU .....                 | 5        |
| 2.1.1.2. Ngôn ngữ PYTHON .....                     | 5        |
| 2.1.1.3. CUDA .....                                | 6        |
| 2.1.1.4. CUDNN .....                               | 6        |
| 2.1.1.5. Phần mềm mô phỏng UNITY .....             | 7        |

|                                                                 |    |
|-----------------------------------------------------------------|----|
| 2.1.2. PYTORCH.....                                             | 7  |
| 2.1.3. Máy học (Machine Learning) [4].....                      | 9  |
| 2.1.4. Học sâu (Deep Learning) [5] .....                        | 9  |
| 2.1.5. Mạng Noron tích chập (Convolutional Neural Network)..... | 9  |
| 2.1.5.1. Lớp tích chập (Convolution Layer) .....                | 10 |
| 2.1.5.2. Lớp gộp (Pooling Layer) .....                          | 12 |
| 2.1.5.3. Lớp kết nối đầy đủ ( Fully Connected Layer) .....      | 13 |
| 2.1.6. Hàm kích hoạt (Activation Function) .....                | 13 |
| 2.1.7. Các loại tích chập thay thế tích chập truyền thống.....  | 13 |
| 2.1.7.1. Pointwise Convolution [13] .....                       | 13 |
| 2.1.7.2. Grouped Convolution [13].....                          | 14 |
| 2.1.7.3. Depthwise Separable Convolution [13].....              | 15 |
| 2.1.8. Tăng cường dữ liệu (Data Augmentations) .....            | 16 |
| 2.1.9. Bài toán phát hiện vật thể (Object Detection) .....      | 17 |
| 2.1.9.1. Kiến trúc YOLOv7 [19].....                             | 18 |
| 2.1.9.2. Label Assignment .....                                 | 20 |
| 2.1.10. Mô hình phân loại hình ảnh (Image Segmentation) .....   | 20 |
| 2.1.11. Các phương pháp đánh giá .....                          | 22 |
| 2.1.11.1. IoU .....                                             | 22 |
| 2.1.11.2. Precision, Recall, AP, mAP [29] .....                 | 23 |
| 2.1.12. Phương pháp tiếp cận đa tác vụ .....                    | 23 |
| 2.2. LIDAR .....                                                | 25 |
| 2.2.1. Nguyên lý hoạt động.....                                 | 25 |
| 2.2.2. Ứng dụng trên LIDAR.....                                 | 25 |
| 2.2.2.1. SLAM .....                                             | 25 |

|                                                                       |           |
|-----------------------------------------------------------------------|-----------|
| 2.2.2.2. DATMO (Detection and tracking of moving object) .....        | 26        |
| 2.3. Hệ thống thông tin .....                                         | 26        |
| 2.3.1. Giới thiệu về mạng giao tiếp trên ô tô .....                   | 26        |
| 2.3.2. Tổng quan về mạng CAN ứng dụng chuẩn giao tiếp trên ô tô ..... | 27        |
| 2.3.2.1. Các khái niệm cơ bản về mạng CAN .....                       | 27        |
| 2.3.2.2. Cấu trúc và các thành phần trong CAN BUS .....               | 28        |
| 2.3.2.2.1. Cấu trúc mạng CAN .....                                    | 28        |
| 2.3.2.2.2. Cấu trúc Node hay Station.....                             | 29        |
| 2.3.2.2.3. Tín hiệu trên BUS CAN .....                                | 29        |
| 2.3.2.3. Cấu trúc các khung dữ liệu trong CAN [32].....               | 31        |
| 2.3.3. Các toán tử bitwise.....                                       | 31        |
| 2.3.4. Các chuẩn giao tiếp khác .....                                 | 32        |
| 2.3.4.1. Chuẩn giao tiếp UART [33].....                               | 32        |
| 2.3.4.2. Chuẩn giao tiếp SPI [34].....                                | 33        |
| 2.4. GIAO DIỆN NGƯỜI DÙNG (UI) .....                                  | 34        |
| 2.4.1. Ngôn ngữ front-end.....                                        | 34        |
| 2.4.1.1. Ngôn ngữ HTML .....                                          | 35        |
| 2.4.1.2. Ngôn ngữ CSS .....                                           | 35        |
| 2.4.1.3. Ngôn ngữ Java Script.....                                    | 36        |
| 2.4.2. API .....                                                      | 36        |
| 2.4.3. FLASK.....                                                     | 37        |
| <b>Chương 3. THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG.....</b>                   | <b>38</b> |
| 3.1. Thiết kế mô hình học sâu .....                                   | 38        |
| 3.1.1. Tập dữ liệu .....                                              | 39        |
| 3.1.2. Ứng dụng trên phần mềm mô phỏng UNITY.....                     | 39        |

|                                                 |    |
|-------------------------------------------------|----|
| 3.1.2.1. Cài đặt môi trường .....               | 40 |
| 3.1.2.1.1. Cài đặt CUDA .....                   | 40 |
| 3.1.2.1.2. Cài đặt cuDNN .....                  | 41 |
| 3.1.2.2. Cách tạo phần mềm mô phỏng Unity ..... | 42 |
| 3.1.2.2.1. Tạo và vẽ địa hình .....             | 42 |
| 3.1.2.2.2. Tạo đường đi .....                   | 43 |
| 3.1.2.2.3. Tạo vật cản và checkpoints .....     | 45 |
| 3.1.2.2.4. Kiểm tra và xây dựng map .....       | 46 |
| 3.1.2.3. Mô hình học sâu sử dụng .....          | 47 |
| 3.1.2.3.1. Lane Detection.....                  | 48 |
| 3.1.2.3.1.1. Phân tích bài toán.....            | 48 |
| 3.1.2.3.1.2. Khảo sát .....                     | 50 |
| 3.1.2.3.1.3. Vấn đề gặp phải .....              | 51 |
| 3.1.2.3.2. Object Detection.....                | 52 |
| 3.1.2.3.2.1. Phân tích bài toán.....            | 52 |
| 3.1.2.3.2.2. Khảo sát .....                     | 54 |
| 3.1.2.4. Đánh giá .....                         | 55 |
| 3.1.3. Ứng dụng trên mô hình.....               | 57 |
| 3.1.3.1. Những thách thức.....                  | 58 |
| 3.1.3.2. Thu thập thêm dữ liệu .....            | 59 |
| 3.1.3.3. Thiết kế mô hình học sâu .....         | 60 |
| 3.1.3.3.1. Mã hóa .....                         | 60 |
| 3.1.3.3.2. Giải mã (Heads).....                 | 62 |
| 3.1.3.3.3. Hàm mất mát (Loss Function).....     | 62 |
| 3.1.3.4. Đánh giá .....                         | 63 |

|                                                                           |           |
|---------------------------------------------------------------------------|-----------|
| 3.1.4. Thử nghiệm trên xe điện RENAULT TWIZY .....                        | 65        |
| 3.2. LIDAR .....                                                          | 66        |
| 3.2.1. Thông số kỹ thuật [50].....                                        | 67        |
| 3.2.2. Cấu hình thiết bị bằng phần mềm SOPAS ET.....                      | 69        |
| 3.2.3. Lấy dữ liệu từ LMS 511-PRO .....                                   | 71        |
| 3.2.3.1. Phân cứng.....                                                   | 71        |
| 3.2.3.2. Giải mã.....                                                     | 73        |
| 3.2.4. Hiển thị dữ liệu .....                                             | 74        |
| 3.3. Thu thập thông số xe.....                                            | 75        |
| 3.3.1. Môi trường nghiên cứu .....                                        | 76        |
| 3.3.2. Phương pháp thu thập dữ liệu CAN Model xe điện RENAULT TWIZY ..... | 76        |
| 3.3.2.1. Kiểm tra tín hiệu mạng CAN bằng máy Hantek 6022BE .....          | 76        |
| 3.3.2.2. Thiết kế MODULE thu thập dữ liệu trực tiếp. ....                 | 77        |
| 3.3.3. Giải mã các ID gói dữ liệu.....                                    | 79        |
| 3.3.4. Giao tiếp Arduino và Jetson Nano bằng Python [55] .....            | 82        |
| 3.4. Giao diện người dùng .....                                           | 83        |
| 3.4.1. Cấu trúc.....                                                      | 84        |
| 3.4.2. Chức năng .....                                                    | 85        |
| 3.4.2.1. Bật/Tắt webcam của server.....                                   | 85        |
| 3.4.2.2. Bật/Tắt webcam của client.....                                   | 86        |
| 3.4.2.3. Chọn/Tắt file video đã chọn.....                                 | 87        |
| 3.4.2.4. Dự đoán kết quả trên thời gian thực .....                        | 87        |
| 3.4.2.5. Hiển thị biểu đồ Lidar và Speed theo thời gian thực .....        | 89        |
| 3.4.2.6. Cập nhật thông tin của xe theo thời gian thực .....              | 89        |
| <b>Chương 4. KẾT QUẢ THỰC NGHIỆM .....</b>                                | <b>91</b> |

|                                                    |            |
|----------------------------------------------------|------------|
| 4.1. Môi trường đánh giá .....                     | 91         |
| 4.2. Kết quả mô hình học sâu.....                  | 93         |
| 4.3. Kết quả LIDAR.....                            | 97         |
| 4.4. Kết quả thu thập thông số xe .....            | 98         |
| 4.5. Tổng kết toàn hệ thống .....                  | 99         |
| <b>Chương 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....</b> | <b>102</b> |
| 5.1. Kết quả.....                                  | 102        |
| 5.2. Hướng phát triển .....                        | 102        |
| <b>TÀI LIỆU THAM KHẢO .....</b>                    | <b>103</b> |
| <b>PHỤ LỤC.....</b>                                | <b>108</b> |

## **DANH MỤC CÁC CHỮ VIẾT TẮT VÀ KÝ HIỆU**

CNN: Convolution Nerural Network

CPU: Central Processing Unit

CUDA: Compute Unified Device Architecture

FPS: Frames per second

GPUs: Graphics Processing Unit

MCU: Micro Controller Unit

IoU: Intersection over Union

AP: Average Precision

mAP: Mean Average Precision

ROI: Region Of Interest

R-CNN: Region - Convolutional Neutral Network

YOLO: You only look Once

SSD: Single Shot Detetor

LiDAR: Light Detection and Ranging

USB: Universal Serial Bus

CAN: Universal Serial Bus

RS-232: Recommended Standard 232

UART: Universal Asynchronous Receiver / Transmitter

SPI: Serial Perialpheral Bus

MISO: Master Input Slave Output

MOSI: Master Output Slave Input

SCLK: Serial Clock

SS / CS: Slave Select / Chip Select

## DANH MỤC HÌNH

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| Hình 2.1. Workflow của Pytorch [3] .....                                   | 8  |
| Hình 2.2. Kiến trúc cơ bản của một Convolution Neural Network [7] .....    | 10 |
| Hình 2.3. Minh họa Convolution Layer [8] .....                             | 11 |
| Hình 2.4. Padding [9] .....                                                | 11 |
| Hình 2.5. Stride [10] .....                                                | 12 |
| Hình 2.6. Max pooling và Average Pooling [11] .....                        | 12 |
| Hình 2.7. Minh họa về Fully Connected Layer [12] .....                     | 13 |
| Hình 2.8. Pointwise convolution [13] .....                                 | 14 |
| Hình 2.9. Minh họa Grouped Convolution [13] .....                          | 15 |
| Hình 2.10. Minh họa Channel Shuffle [13] .....                             | 15 |
| Hình 2.11. Minh họa cho Depthwise Separable Convolution [13] .....         | 16 |
| Hình 2.12. Ví dụ về tăng cường dữ liệu .....                               | 17 |
| Hình 2.13. So sánh YOLOv7 với những phiên bản YOLO khác .....              | 19 |
| Hình 2.14. Backbone .....                                                  | 19 |
| Hình 2.15. Phân đoạn ngữ nghĩa và phân đoạn cá thể [26] .....              | 21 |
| Hình 2.16. Phát hiện khối Unet [26] .....                                  | 21 |
| Hình 2.17. Xe tự hành [26] .....                                           | 22 |
| Hình 2.18. Kiến trúc Mask R-CNN [31] .....                                 | 24 |
| Hình 2.19. Nguyên lý tính toán khoảng cách của thiết bị [29] .....         | 25 |
| Hình 2.20. Cấu trúc BUS và Node của mạng CAN .....                         | 28 |
| Hình 2.21. Dây cáp BUS CAN [8] .....                                       | 29 |
| Hình 2.22. Mức điện áp trên đường CAN_H và CAN_L khi truyền dữ liệu .....  | 30 |
| Hình 2.23. Xác định mức điện áp logic qua sự sai lệch CAN_L và CAN_H ..... | 31 |
| Hình 2.24. Sơ đồ giao tiếp Uart .....                                      | 33 |
| Hình 2.25. Các chân giao tiếp SPI của Arduino Uno .....                    | 34 |
| Hình 3.1. Pipeline hệ thống đa tác vụ xe tự hành .....                     | 38 |
| Hình 3.2. Pipeline bài toán xe tự hành .....                               | 38 |
| Hình 3.3. Bộ dữ liệu Carla [36] .....                                      | 39 |
| Hình 3.4. Tham số phù hợp với GeForce RTX 3090 .....                       | 40 |
| Hình 3.5. Xác định phiên bản CUDA .....                                    | 40 |

|                                                                                         |    |
|-----------------------------------------------------------------------------------------|----|
| Hình 3.6. Các câu lệnh cần thiết để tải CUDA.....                                       | 41 |
| Hình 3.7. Chọn phiên bản của cuDNN .....                                                | 41 |
| Hình 3.8. Một số package để tạo map mô phỏng .....                                      | 42 |
| Hình 3.9. Các công cụ vẽ địa hình.....                                                  | 43 |
| Hình 3.10. Các công cụ chỉnh sửa làn đường .....                                        | 43 |
| Hình 3.11. Thêm Lane mới.....                                                           | 44 |
| Hình 3.12. Cập nhật địa hình .....                                                      | 44 |
| Hình 3.13. Điều chỉnh biên báo .....                                                    | 45 |
| Hình 3.14. Map đường đi.....                                                            | 45 |
| Hình 3.15. Tạo vật cản và Checkpoints .....                                             | 46 |
| Hình 3.16. Nền tảng xây dựng map .....                                                  | 47 |
| Hình 3.17. Phương pháp giải quyết bài toán xe tự hành trên phần mềm mô phỏng .....      | 47 |
| Hình 3.18. Phương pháp xử lý ảnh truyền thống.....                                      | 49 |
| Hình 3.19. Các bước huấn luyện một mô hình học sâu .....                                | 50 |
| Hình 3.20. So sánh kết quả giữa Bisenet [31] và Unet [27] .....                         | 51 |
| Hình 3.21. Sử dụng mô hình Unet cho bài toán phân đoạn làn đường.....                   | 51 |
| Hình 3.22. Bắt được những đoạn đường rất xa gây nhiều cho việc điều khiển.....          | 51 |
| Hình 3.23. Các trường hợp đối tượng nhỏ.....                                            | 53 |
| Hình 3.24. Trực quan hoá dữ liệu .....                                                  | 53 |
| Hình 3.25. Kết quả Yolov7 .....                                                         | 55 |
| Hình 3.26. Kết quả mô phỏng xe tự hành .....                                            | 56 |
| Hình 3.27. Xe mô hình.....                                                              | 57 |
| Hình 3.28. Sa bàn xe tự hành.....                                                       | 57 |
| Hình 3.29. Vấn đề trong việc triển khai các mô hình học sâu lên các máy tính nhúng .... | 59 |
| Hình 3.30. Bộ dữ liệu huấn luyện mô hình.....                                           | 59 |
| Hình 3.31. Kiến trúc của mạng .....                                                     | 60 |
| Hình 3.32. Cấu trúc BiFPN [45] .....                                                    | 61 |
| Hình 3.33. Kết quả ứng dụng trên xe mô hình .....                                       | 65 |
| Hình 3.34. Model xe điện thử nghiệm Renault Twizy .....                                 | 65 |
| Hình 3.35. Bổ sung hai tác vụ LIDAR và CANBUS cho hệ thống.....                         | 66 |
| Hình 3.36. Sơ đồ giải thuật LiDAR .....                                                 | 67 |

|                                                                                       |    |
|---------------------------------------------------------------------------------------|----|
| Hình 3.37. Thông số kỹ thuật LMS511-PRO .....                                         | 68 |
| Hình 3.38. Sơ đồ dây nguồn LMS511 .....                                               | 68 |
| Hình 3.39. Giao thức giao tiếp của LMS511 .....                                       | 69 |
| Hình 3.40. Thiết lập cấu hình hoạt động LMS 511 .....                                 | 70 |
| Hình 3.41. Sơ đồ dây giao tiếp dữ liệu LMS511 .....                                   | 72 |
| Hình 3.42. Mạch giao tiếp LMS 511-PRO .....                                           | 72 |
| Hình 3.43. Điện tín yêu cầu gửi dữ liệu về máy tính.....                              | 73 |
| Hình 3.44. Mẫu dữ liệu phản hồi từ thiết bị.....                                      | 74 |
| Hình 3.45. Tam giác vuông .....                                                       | 75 |
| Hình 3.46. Kết quả thiết bị LMS 511-PRO .....                                         | 75 |
| Hình 3.47. Sơ đồ tổng quan về phương pháp thu thập dữ liệu .....                      | 76 |
| Hình 3.48. Các chân trên cổng OBD .....                                               | 76 |
| Hình 3.49. Kết quả xung đo được.....                                                  | 77 |
| Hình 3.50. CAN bus signal levels.....                                                 | 77 |
| Hình 3.51. Module MCP 2515.....                                                       | 78 |
| Hình 3.52. Sơ đồ kết nối CAN BUS SHIELD.....                                          | 78 |
| Hình 3.53. Dữ liệu từ CAN BUS .....                                                   | 79 |
| Hình 3.54. Giao diện người dùng .....                                                 | 84 |
| Hình 3.55. Các chức năng của giao diện người dùng.....                                | 85 |
| Hình 3.56. Sơ đồ xử lý bật webcam server.....                                         | 85 |
| Hình 3.57. Sơ đồ xử lý tắt webcam server.....                                         | 86 |
| Hình 3.58. Sơ đồ xử lý bật webcam client.....                                         | 86 |
| Hình 3.59. Sơ đồ xử lý tắt webcam client.....                                         | 87 |
| Hình 3.60. Sơ đồ xử lý mở file .....                                                  | 87 |
| Hình 3.61. Sơ đồ xử lý tắt file.....                                                  | 87 |
| Hình 3.62. Sơ đồ xử lý khi người dùng gửi yêu cầu dự đoán bằng mô hình học sâu .....  | 88 |
| Hình 3.63. Sơ đồ xử lý khi người dùng gửi yêu cầu tắt dự đoán bằng mô hình học sâu .. | 89 |
| Hình 3.64. Sơ đồ xử lý hiển thị biểu đồ LiDAR.....                                    | 89 |
| Hình 3.65. Sơ đồ xử lý hiển thị biểu đồ SPEED.....                                    | 89 |
| Hình 3.66. Sơ đồ xử lý cập nhật thông tin của xe.....                                 | 90 |
| Hình 4.1. Sơ đồ mạch điện hệ thống.....                                               | 92 |

|                                                                                   |     |
|-----------------------------------------------------------------------------------|-----|
| Hình 4.2. Sơ đồ phần cứng hệ thống.....                                           | 92  |
| Hình 4.3. Kết quả của mô hình học đa tác vụ.....                                  | 96  |
| Hình 4.4. Kết quả của mô hình ở điều kiện xấu .....                               | 97  |
| Hình 4.5. Kết quả nhiệm vụ phát hiện vật cản.....                                 | 98  |
| Hình 4.6. Gói dữ liệu ID 0x19F và ID 0x196 xử lý, mã hóa và xuất ra Monitor ..... | 99  |
| Hình 4.7. Chi phí tính toán của hệ thống .....                                    | 100 |
| Hình 4.8. Hệ thống đa tác vụ ứng dụng cho xe tự hành .....                        | 100 |
| Hình 4.9. Phần cứng hệ thống đa tác vụ ứng dụng cho xe tự hành .....              | 101 |

## DANH MỤC BẢNG

|                                                                               |    |
|-------------------------------------------------------------------------------|----|
| Bảng 2.1. Phân loại đánh giá kết quả dự đoán của mô hình.....                 | 23 |
| Bảng 2.2. Bảng tốc độ truyền bit .....                                        | 28 |
| Bảng 2.3. Bảng chân trị cho AND .....                                         | 32 |
| Bảng 2.4. Bảng chân trị của OR .....                                          | 32 |
| Bảng 3.1. So sánh các mạng hiện đại về nhiệm vụ phân đoạn. ....               | 50 |
| Bảng 3.2. So sánh các phiên bản YOLO về phát hiện và nhận diện vật thể. ....  | 54 |
| Bảng 3.3. Kết quả của các cấu hình khác nhau cho tất cả các tác vụ. ....      | 63 |
| Bảng 3.4. Bảng tốc độ truyền của chuẩn giao tiếp RS-232 [52] .....            | 71 |
| Bảng 3.5. Bảng giá trị ID 0x155 .....                                         | 79 |
| Bảng 3.6. Bảng giá trị 0x196 .....                                            | 80 |
| Bảng 3.7. Bảng giá trị ID 0x19F .....                                         | 80 |
| Bảng 3.8. Bảng giá trị ID 0x59B .....                                         | 80 |
| Bảng 3.9. Bảng giá trị ID 0x424 .....                                         | 81 |
| Bảng 3.10. Bảng giá trị ID 0x425 .....                                        | 81 |
| Bảng 3.11. Bảng giá trị ID 0x556 .....                                        | 82 |
| Bảng 3.12. Bảng giá trị ID 0x599 .....                                        | 82 |
| Bảng 3.13. Bảng giá trị ID 0x554 .....                                        | 82 |
| Bảng 3.14. Bảng giá trị ID 0x557 .....                                        | 82 |
| Bảng 3.15. Bảng giá trị ID 0x55E .....                                        | 82 |
| Bảng 3.16. Bảng giá trị ID 0x55F .....                                        | 82 |
| Bảng 4.1. Thống kê phần cứng và phần mềm của các tác vụ .....                 | 91 |
| Bảng 4.2. So sánh giữa việc học đa tác vụ và học từng tác vụ .....            | 93 |
| Bảng 4.3. So sánh các mô hình học đa tác vụ triển khai trên Jetson Nano ..... | 94 |
| Bảng 4.4. Chu trình gửi lên các gói ID dữ liệu cho mạng CAN.....              | 99 |

## CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

### 1.1. Lý do chọn đề tài

Với tình hình nghiên cứu hiện tại có thể thấy các phương pháp hiện nay đang xử lý độc lập các nhiệm vụ và chưa đủ thông tin để hệ thống xe tự hành đưa ra quyết định điều này gây nên rất nhiều nhược điểm. Thứ nhất là việc phân cứng bị hạn chế trên các ứng dụng xe tự hành nên tốc độ xử lý của hệ thống sẽ rất chậm, dẫn đến tốc độ của xe sẽ bị chậm, hoặc hệ thống không thể đáp ứng kịp thời cho người lái dẫn đến không có sự an toàn. Thứ hai là các thông tin của các nhiệm vụ này là có liên quan với nhau tuy nhiên nếu xử lý độc lập thì các thông tin này sẽ không có sự liên kết với nhau gây phí phạm tài nguyên. Thứ ba là với sự phát triển không ngừng của hệ thống hạ tầng ngày nay thì với việc chỉ áp dụng các mô hình học sâu là chưa đủ do các mô hình này khá phụ thuộc vào dữ liệu học được.

Nhận thấy được những nhược điểm trên do đó tôi quyết định thực hiện đề tài: **“THIẾT KẾ HỆ THỐNG ĐA TÁC VỤ TRIỂN KHAI TRÊN JETSON NANO ỨNG DỤNG CHO XE TỰ HÀNH”** với ý tưởng ngoài việc áp dụng các mô hình học sâu như “con mắt”, tôi còn cung cấp thêm cho hệ thống cảm biến vật cản (LIDAR) và thông tin trạng thái của xe (CANBUS) như là các “giác quan” nhằm giúp cho hệ thống có nhiều thông tin hơn để đưa ra quyết định cũng như tăng độ tin cậy cho hệ thống.

### 1.2. Tình hình nghiên cứu trong và ngoài nước

#### 1.2.1. Tình hình nghiên cứu trong nước

Ở Việt Nam, công nghệ xe tự hành cũng đã có những phát triển tích cực với sự dẫn đầu của những công ty dẫn đầu như Tập đoàn Phenikaa và mới đây là VinFast. Không chỉ vậy, các nhà nghiên cứu ở trong nước cũng đang rất tích cực tham gia vào phát triển công nghệ này. Bằng chứng là hàng năm không ít các cuộc thi nhằm kích thích tư duy sáng tạo dành cho học sinh, sinh viên được tổ chức. Đây cũng chính là môi trường để công nghệ xe tự hành được phổ biến rộng rãi, đồng thời cũng là nơi để những giải pháp mới được ứng dụng và phát triển, từ đó công nghệ xe tự hành ngày càng hoàn thiện và phát triển hơn. Nhờ vậy, có rất nhiều phương pháp đã được đưa ra nhằm giải quyết các vấn đề về nhận diện làn đường và biển báo cũng như nhiều thử thách khác trong bài toán

này. Để hệ thống có thể giải quyết được hầu hết các thử thách, thông thường các nhà nghiên cứu sẽ kết hợp nhiều mô hình lại với nhau, mỗi mô hình sẽ giải quyết một thử thách hoặc nhiều hơn. Đối với các hệ thống được kết hợp từ nhiều mô hình nhỏ như vậy, nhiều yếu tố cần phải được quan tâm đặc biệt là tốc độ và độ chính xác của hệ thống. Tuy nhiên, tồn đọng lớn nhất của mô hình có dạng trên là độ trễ tăng kéo theo độ chính xác giảm xuống.

### **1.2.2. Tình hình nghiên cứu ngoài nước**

Trong lĩnh vực xe tự hành để có một hệ thống ổn định và hiệu quả thì phải giải quyết được các thách thức bao gồm: bóng các đối tượng xung quanh chiếu xuống làn đường gây nhiễu, các khúc cua và những vật cản trên đường. Thông thường, các phương pháp truyền thống giải quyết các vấn đề này bằng các thông tin của pixel trong ảnh như độ lớn gradient của pixel. Ví dụ, trong một bài báo của Yu-Kuen Tsai và các cộng sự [1] đã đề xuất phương pháp sử dụng toán tử Sobel để thu được độ lớn gradient cho mỗi pixel và loại bỏ cạnh có độ lớn gradient thấp bằng cách áp dụng ngưỡng, từ đó trích xuất được các cạnh và chia khung đầu vào thành nhiều vùng để sử dụng biến đổi Hough trong việc tính các đặc trưng của vạch kẻ đường, cuối cùng là sử dụng bộ điều khiển Fuzzy để lái. Tuy kết quả thu được là khá tốt khi áp dụng cho các xe golf tự hành trong các khuôn viên trường hay sân vận động, nhưng để áp dụng phương pháp trích xuất thủ công này cho các phương tiện tham gia giao thông trên các đường lộ nơi có nhiều sự biến đổi, và ảnh hưởng bởi nhiễu rất nhiều thì hiệu suất có thể bị ảnh hưởng đáng kể. Do đó, để cải thiện tính linh hoạt và tính khái quát của thông tin hay các đặc trưng, thì các nhà nghiên cứu hiện nay thường dựa vào các phương pháp học sâu có giám sát để trích xuất các đặc trưng cũng như phân loại đối tượng. Dựa vào đó, Vijay Badrinarayanan và các cộng sự [2] đã thiết kế ra mạng SegNet sử dụng kiến trúc mã hóa - giải mã để phân đoạn hình ảnh, phân loại từng pixel để xác định các vật thể có trong ảnh, phương pháp này được áp dụng cho việc phát hiện làn đường di chuyển, ngoài ra còn được dùng để hiện nhiều đối tượng khác. Mặc dù, phương pháp phân đoạn hình ảnh này mang lại kết quả rất tốt, độ chính xác cao, nhưng lại mang lại vấn đề về tốc độ xử lý khi thực thi trên các phần cứng nhất, điều đó dẫn đến vấn đề về xử lý thời gian thực cho mô hình này. Một vấn đề nữa phát sinh khi ứng dụng phương pháp này cho xe tự hành di chuyển ngoài thực tế đó chính là nhận diện biển báo, vì phương pháp này cho ta biết được vị trí cũng như phân

biệt được các vật thể khác nhau nhưng không thể cho ta biết biển báo thể hiện điều gì, chính vì vậy ta cần phải thêm vào hệ thống một mạng nhận diện biển báo tốt, nhưng điều này cũng đồng nghĩa với việc hệ thống lại càng phức tạp hơn.

### **1.3. Mục tiêu đề tài**

- Thiết kế mô hình học đa tác vụ cho bài toán xe tự hành. Bằng cách xử lý đồng thời các tác vụ phân đoạn làn đường, nhận diện biển báo, vật cản, và nhận dạng bối cảnh, phương pháp này có thể cải thiện tương đối về độ trễ so với các phương pháp trước, cũng như tiết kiệm tài nguyên hệ thống. Và được đánh giá qua ba giai đoạn: mô phỏng, xe mô hình, Renault Twizy.

- Xây dựng hệ thống phát hiện vật cản. Sử dụng LIDAR LMS511-PRO thông qua chuẩn giao tiếp RS-232 với Jetson Nano.

- Thiết kế hệ thống thông tin (CANBUS). Sử dụng Arduino và module MCP-2515 đọc và giải mã data

- Thiết kế giao diện thao tác với người dùng. Bằng cách sử dụng Flask để tạo API, lập trình HTML và CSS để tạo giao diện tương tác với người dùng gồm các tính năng (Mở Camera, Dừng Camera, Prediction)

- Hệ thống có thể chạy thời gian thực

### **1.4. Phương pháp nghiên cứu**

- Tìm hiểu và tạo map mô phỏng trên phần mềm Unity.

- Tìm hiểu và tổng hợp những nghiên cứu về các mạng cho bài toán nhận diện và phân đoạn ứng dụng cho xe tự hành.

- Tìm hiểu và tổng hợp những nghiên cứu về bài toán đa tác vụ.

- Nghiên cứu và thiết kế mô hình đa tác vụ ứng dụng cho xe tự hành.

- Viết chương trình cho hệ thống và huấn luyện mạng.

- Phân tích và đánh giá kết quả thu được thông qua ba giai đoạn: mô phỏng, xe mô hình, xe Renault Twizy.

- Đọc Datasheet LIDAR LMS511-PRO để thu thập và giải mã dữ liệu.

- Sử dụng các nguồn tài liệu trên Internet và các giáo trình để tìm hiểu về mạng CAN.
- Áp dụng kiến thức đã học từ môn thực tập Điện II để giải mã dữ liệu mạng CAN.
- Thiết kế giao diện thao tác với người dùng bằng cách tạo API để nhiều người dùng có thể truy cập hệ thống.

### **1.5. Đối tượng nghiên cứu**

- Ngôn ngữ Python, C/C++, HTML & CSS, framework Pytorch.
- Ứng dụng mô phỏng Unity.
- Mạng phân đoạn, phát hiện và nhận diện vật thể, bối cảnh.
- Mạng học đa tác vụ.
- Thiết bị LIDAR LMS511-PRO.
- Đọc và giải mã mạng CAN trên xe điện Renault Twizy.
- Thiết kế giao diện người dùng
- Nghiên cứu xử lý phân luồng cho các tác vụ.

### **1.6. Giới hạn đề tài**

Vì đề tài tập trung phát triển giải thuật, thu thập thông tin, tốc độ xử lý nên chưa ứng dụng các giải thuật điều khiển và các đánh giá dựa trên việc tự chạy thử nghiệm trên các môi trường khác nhau (Unity, sân xe mô hình, Trường Đại học Sư phạm Kỹ thuật TPHCM).

### **1.7. Bố cục đề tài**

Chương 1: TỔNG QUAN ĐỀ TÀI

Chương 2: CƠ SỞ LÝ THUYẾT

Chương 3: THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG

Chương 4: KẾT QUẢ THỰC NGHIỆM

Chương 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

## CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

### 2.1. Mô hình học sâu

Phần này sẽ giới thiệu về môi trường làm việc với các mô hình AI; Pytorch, đây là một framework được sử dụng khá phổ biến trong việc nghiên cứu các mô hình học sâu; các khái niệm về máy học, học sâu; giới thiệu về mạng tích chập; một số loại tích chập có thể thay thế tích chập truyền thống để làm nhẹ mô hình tăng tốc độ; giới thiệu về cách tăng cường dữ liệu khi huấn luyện; bài toán phát hiện và nhận diện vật thể ở đây chúng tôi sử dụng mạng YOLOv7 cho bài toán này do đó sẽ giới thiệu về mạng YOLOv7 và ý tưởng mà tác giả thiết kế mạng này; giới thiệu về bài toán phân đoạn làn đường, nhận dạng bối cảnh và cuối cùng là hướng tiếp cận của phương pháp học đa tác vụ.

#### 2.1.1. Môi trường

##### 2.1.1.1. Hệ điều hành UBUNTU

Ubuntu là hệ điều hành máy tính dựa trên Debian GNU/Linux, một bản phân phối Linux thông dụng. Tên của nó bắt nguồn từ "ubuntu" trong tiếng Zulu, có nghĩa là "tình người". Triết lý của ubuntu: "Tôi được là chính mình nhờ có những người xung quanh" - một khía cạnh tích cực của cộng đồng. Mục đích của Ubuntu là cung cấp một hệ điều hành ổn định, cập nhật cho người dùng thường, tập trung vào sự tiện dụng và dễ dàng cài đặt. Ubuntu được đánh xếp hạng là bản phân phối Linux thông dụng nhất cho máy tính để bàn, chiếm khoảng 30% số bản Linux được cài đặt trên máy tính để bàn năm 2007.

##### 2.1.1.2. Ngôn ngữ PYTHON

Python là một ngôn ngữ lập trình bậc cao cho các mục đích lập trình đa năng, do Guido van Rossum tạo ra và lần đầu ra mắt vào năm 1991. Python được thiết kế với ưu điểm mạnh là dễ đọc, dễ học và dễ nhớ. Python là ngôn ngữ có hình thức rất sáng sủa, cấu trúc rõ ràng, thuận tiện cho người mới học lập trình. Cấu trúc của Python còn cho phép người sử dụng viết mã lệnh với số lần gõ phím tối thiểu. Với cú pháp cực kì đơn giản và thanh lịch, Python là lựa chọn hoàn hảo cho những ai lần đầu tiên học lập trình. Tuy nhiên, đây cũng là ngôn ngữ nổi tiếng về sự chặt chẽ,

nhanh, mạnh, và có mặt ở mọi hệ điều hành. Được sử dụng trong lập trình từ những game đơn giản nhất, cho đến những thuật toán tìm kiếm phức tạp nhất.

Code của Python dễ đọc, ngắn gọn (ngắn hơn code Java 3-5 lần, và C++ 5-10 lần). Python là ngôn ngữ hướng đối tượng được ứng dụng rất đa dạng. Vì vậy, những hiểu biết về cấu trúc và kiến trúc Python sẽ là nền tảng tham chiếu rất tốt khi bạn muốn chuyển sang học các ngôn ngữ lập trình khác. Cùng với Ruby, Python là ngôn ngữ lập trình giúp developer nhận mức lương cao thứ nhì (khoảng \$107,000/năm) tại Mỹ. Thiết thực trong thiết kế web/ứng dụng web: Django, web framework được viết bởi Python sẽ khiến lập trình web trở nên đơn giản hơn, giúp code của bạn nhanh và ổn định hơn nhiều, đồng thời tiết kiệm hàng tấn thời gian lập trình.

#### **2.1.1.3. CUDA**

CUDA là từ viết tắt của Compute Unified Device Architecture - Kiến trúc hợp nhất tính toán của các thiết bị điện tử được phát triển độc quyền bởi hãng công nghệ NVIDIA. Còn về CUDA core hay nhân CUDA thì chúng ta có thể hiểu đây là một nhân xử lý trong GPU của card đồ họa - đơn vị chịu trách nhiệm tính toán các thông tin, dữ liệu đồ họa cần được kết xuất. Nhân CUDA tích hợp trong GPU của card đồ họa rời, càng nhiều nhân CUDA thì khả năng tính toán đồng thời nhiều thông tin càng nhanh và chính xác.

CUDA Core đóng vai trò quan trọng trong quá trình tính toán, xử lý thông tin nhận được của GPU. Chúng là một trong những thành phần quyết định đến chất lượng hình ảnh hiển thị khi người dùng chơi game; file đồ họa được render sau khi thiết kế có độ tỉ mỉ, chính xác ra sao; hay thậm chí là thời gian nghiên cứu một chủ đề khoa học nhanh hay chậm. Nhân CUDA vừa qua đã và đang đóng góp rất nhiều trong công cuộc nghiên cứu Vacxin chống virus Corona.

#### **2.1.1.4. CUDNN**

cuDNN – NVIDIA CUDA Deep Neural Network: Là một thư viện nền tảng cho các deep neural network được tăng tốc bởi GPU. cuDNN cung cấp các thiết lập

được tinh chỉnh cho các thủ tục được chuẩn hóa như forward and backward convolution, pooling, normalization và các lớp kích hoạt.

#### **2.1.1.5. Phần mềm mô phỏng UNITY**

Phần mềm mô phỏng Unity bao gồm các kiến thức về cách sử dụng các công nghệ mô phỏng để tạo ra các hình ảnh và trải nghiệm trực quan cho người dùng. Mô phỏng trong Unity bao gồm các khái niệm như mô phỏng động lực, mô phỏng vật lý, mô phỏng ánh sáng và mô phỏng hệ thống tăng tốc.

Cụ thể mô phỏng động lực giúp tạo ra các hiệu ứng động như chuyển động, lật, xoay và va đập cho các đối tượng trong trò chơi hoặc ứng dụng của. Mô phỏng vật lý trong Unity giúp tạo ra các hiệu ứng vật lý như trọng lực, va chạm, giảm sức chảy và các hiệu ứng hấp dẫn khác. Mô phỏng ánh sáng trong Unity giúp tạo ra các hiệu ứng ánh sáng như sáng, bóng và các hiệu ứng chiếu sáng một cách chính xác và trực quan.

Unity có thể sử dụng trong các ứng dụng của AI bằng cách kết hợp các thư viện AI và các thuật toán với nền tảng của Unity. Một số cách sử dụng Unity trong AI bao gồm:

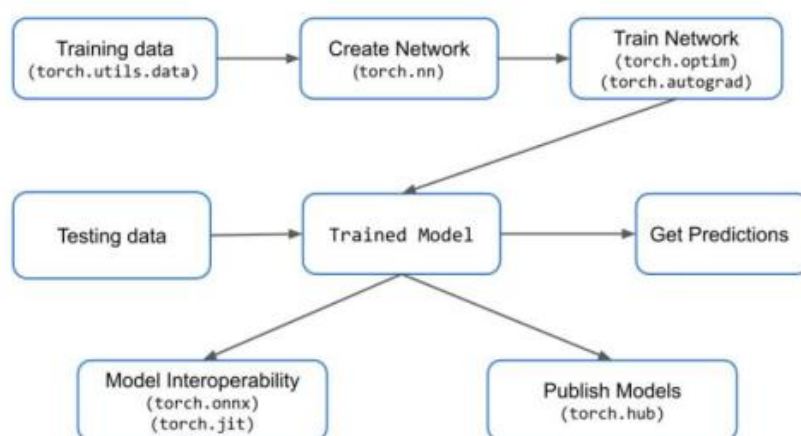
1. Tạo mô hình trạng thái cho các agent trong một môi trường game.
2. Sử dụng Unity để tạo môi trường học cho các mạng neural hoặc các thuật toán khác.
3. Sử dụng Unity để tạo môi trường thực tế ảo để đào tạo các mạng neural hoặc các thuật toán khác.
4. Tích hợp các thuật toán AI vào trò chơi để tạo ra các đối tượng hoạt động tự nhiên hơn hoặc các trận chiến AI độc đáo.
5. Sử dụng Unity để tạo ra các ứng dụng VR/AR với tích hợp AI.

Tổng quan, Unity là một công cụ mạnh mẽ cho việc tạo ra các ứng dụng AI với tính năng đồ họa 3D và tạo môi trường học thực tế ảo.

#### **2.1.2. PYTORCH**

Pytorch là framework được phát triển bởi Facebook. Đây là một ông lớn về công nghệ đầu tư rất nhiều nguồn lực cho việc phát triển Trí tuệ nhân tạo. Pytorch được phát triển

với giấy phép mã nguồn mở do đó nó tạo được cho mình một cộng đồng rất lớn. Một cộng đồng lớn đồng nghĩa với nhiều tài nguyên để học và các vấn đề của bạn có thể đã có ai đó giải quyết và chia sẻ với cộng đồng. Pytorch cùng với Tensorflow và Keras là một trong những framework phổ biến được sử dụng trong các bài toán về Deep Learning hiện nay. Đặc biệt, trong các lĩnh vực nghiên cứu, hầu như các tác giả đều sử dụng Pytorch để triển khai bài toán của mình. Pytorch cho thấy lợi thế của nó trong lĩnh vực nghiên cứu bởi việc rất dễ dàng để bạn debug và visuallize, ngoài ra nó theo cơ chế Dynamic Graphs cho phép giảm thời gian huấn luyện mô hình. Pytorch sử dụng kiểu dữ liệu chủ yếu là Tensor. Tensor là một mảng n-chiều với các phần tử có cùng loại dữ liệu, đặc biệt là Tensor có thể tính toán trên GPUs (điều này làm việc tính toán trở nên nhanh hơn). Hình 2.1 mô tả một quy trình cơ bản của Pytorch gồm các module quan trọng và được kết nối với nhau qua từng giai đoạn.



Hình 2.1. Workflow của Pytorch [3]

Các module quan trọng và cơ bản chúng tôi sẽ đề cập trong bài báo cáo này là `torch.utils`, `torch.nn`, `torch.optim`, `torch.autograd`.

Trong một bài toán học sâu, để giải quyết ta cần biết đầu vào đầu ra của mô hình do đó việc tạo và xử lý data đầu vào là bước đầu tiên của bài toán và mô-đun `torch.utils.data` của Pytorch hỗ trợ việc này. Mặc khác ta còn có thể sử dụng `torch.nn.DataParallel` và `torch.distributed` khi muốn huấn luyện bằng nhiều GPUS. Sau khi đã tạo và xử lý đầu vào thì việc tiếp theo là tạo ra mô hình học sâu. Pytorch hỗ trợ mô-đun `torch.nn` để làm việc này. Nó cung cấp các lớp Neural Network phổ biến như ‘fully connected’, ‘convolution’, ngoài ra còn có các hàm activation và hàm mất

mát,... Và có nhiều hàm mới hay kiến trúc mới thì nó sẽ phụ thuộc vào phiên bản Pytorch đang sử dụng.

Khi đã hoàn thành kiến trúc mạng và đã có dữ liệu, thì chúng ta tiến hành huấn luyện mô hình để có thể học được dữ liệu. Lúc này chúng ta sẽ sử dụng mô-đun `torch.optim` để cập nhật trọng số. Ngoài ra chúng ta có thể sử dụng mô-đun `torch.autograd` để tự động tính gradient. Sau khi mô hình được huấn luyện, nó có thể sử dụng để dự đoán các ngõ ra khi có ngõ vào. Ngoài ra chúng ta cũng có thể chuyển đổi từ framework Pytorch sang các framework khác nếu cần thiết.

### **2.1.3. Máy học (Machine Learning) [4]**

Máy học là một nhánh của trí tuệ nhân tạo và khoa học máy tính, máy học tập trung vào việc sử dụng dữ liệu và thuật toán để có thể bắt chước cách con người học và dần dần cải thiện độ chính xác của nó. Các thuật toán của máy học thường được tạo ra bằng cách sử dụng các khuôn khổ giúp tăng độ phát triển sản phẩm như Tensorflow hoặc Pytorch.

### **2.1.4. Học sâu (Deep Learning) [5]**

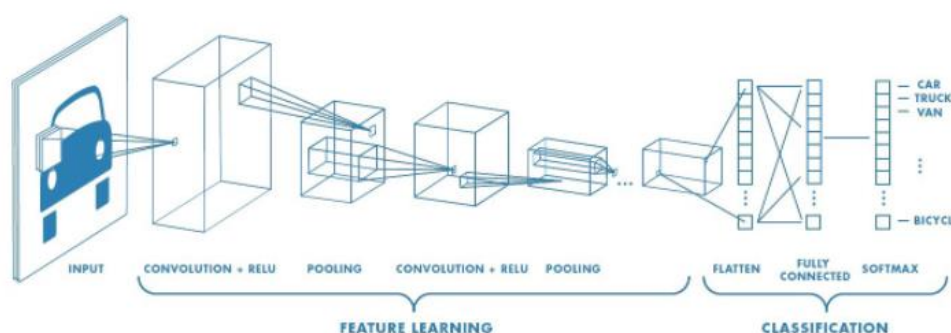
Học sâu là một nhánh của máy học, giúp cho máy tính tự huấn luyện để thực hiện các tác vụ phức tạp. Học sâu hoạt động giống với các mạng lưới thần kinh sâu, việc thiết kế này dựa trên cách con người suy nghĩ và học hỏi. Các phương pháp về thị giác máy tính trong nhận diện vật thể chỉ hiệu quả đối với các đặc trưng được chỉ định và biết trước, nhưng trong thực tế các đặc trưng này bị ảnh hưởng của rất nhiều yếu tố khác nhau như là góc nhìn, ánh sáng, che khuất, ... Học sâu ra đời đã khắc phục những hạn chế này và nó đã được ứng dụng mạnh mẽ trong thị giác máy tính.

### **2.1.5. Mạng Noron tích chập (Convolutional Neural Network)**

Convolution Neural Network (CNN) [6] là một trong các loại mô hình nổi tiếng của học sâu, dùng để xử lý các loại dữ liệu dạng lưới, chẳng hạn như hình ảnh. Một mô hình mạng nơ ron tích chập thường sẽ có các thành phần bao gồm hình ảnh đầu vào sẽ đi qua các lớp tích chập (Convolution Layer) và các lớp gộp (Pooling Layer) tiếp theo đến các lớp kết nối đầy đủ (Fully Connected Layer) rồi đến kết quả dự đoán của mô hình. Hai lớp đầu tiên, convolution và lớp pooling dùng để trích xuất đặc trưng từ đầu vào, và lớp

thứ ba fully connected layers sẽ liên kết các lớp đặc trưng này thành đầu ra. Lý do mạng nơ ron tích chập là do vấn đề của mạng nơ ron trước đây, trích đặc trưng dựa trên từng pixel. Ví dụ khi đầu vào là một ảnh máy có kích thước là  $32 \times 32$  được biểu diễn ở dạng tensor là  $32 \times 32 \times 3$ . Để hiển thị hết nội dung của bức ảnh thì cần truyền vào input layer tất cả pixel ( $32 \times 32 \times 3 = 3072$ ). Nghĩa là input layer sẽ có 3072 notes. Giả sử số lượng note ở trong hidden layer đầu tiên là 1000. Thì lúc này số lượng trọng số (weight  $W$ ) giữa input và hidden là  $3072 \times 1000 = 3072000$  và số lượng bias là 1000. Đây mới chỉ là layer đầu tiên và trong một mô hình còn nhiều layer nữa và nếu kích thước ảnh tăng thì số lượng tham số tăng cực kỳ nhanh. Bên cạnh đó, ta có thể thấy rằng các pixel ở cạnh nhau thường liên kết với nhau hơn là những pixel ở xa. Do đó, chúng ta sẽ sử dụng tích chập nhằm giải quyết vấn đề về số lượng tham số lớn nhưng vẫn trích xuất được đặc trưng của ảnh.

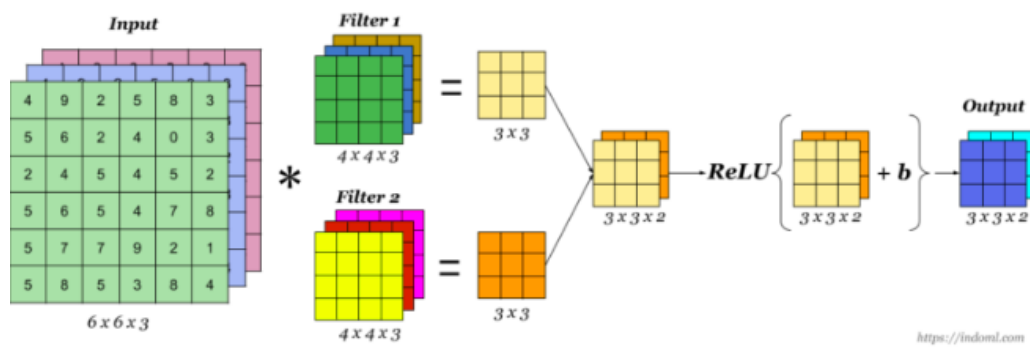
Trong hình 2.2 thể hiện cấu trúc cơ bản của một mạng CNN, ảnh đầu vào sẽ được đi qua nhiều lớp tích chập với các bộ lọc gọi là kernel, sau đó đến các lớp pooling, rồi tiếp theo được kết nối đầy đủ (fully connected layers) và cuối cùng sẽ đến các hàm softmax để phân loại đối tượng dựa vào xác suất từ 0 đến 1.



Hình 2.2. Kiến trúc cơ bản của một Convolution Neural Network [7]

#### 2.1.5.1. Lớp tích chập (Convolution Layer)

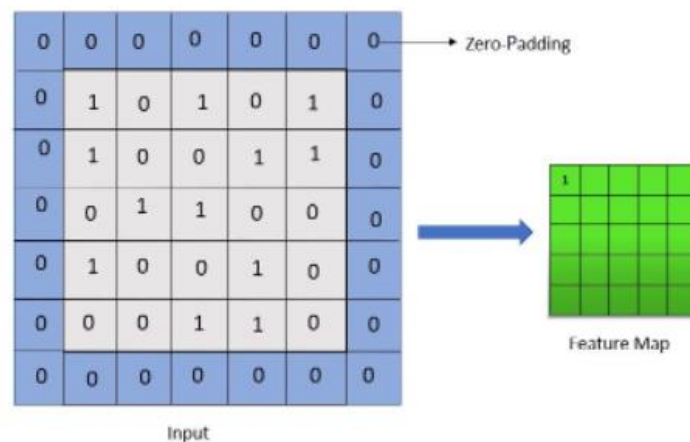
Bắt đầu là tích chập có nhiệm vụ trích xuất các đặc trưng từ hình ảnh, tham số này bao gồm nhiều bộ lọc có thể học được. Các bộ lọc này thường là  $3 \times 3$ ,  $5 \times 5$ , ... có độ sâu bằng độ sâu của đầu vào. Bằng cách trượt các bộ lọc theo chiều ngang và dọc trên ảnh, chúng ta thu được một bản đồ đặc trưng chứa các đặc trưng trích xuất hình ảnh đầu vào. Một convolution layer được mô tả như hình 2.3.



Hình 2.3. Minh họa Convolution Layer [8]

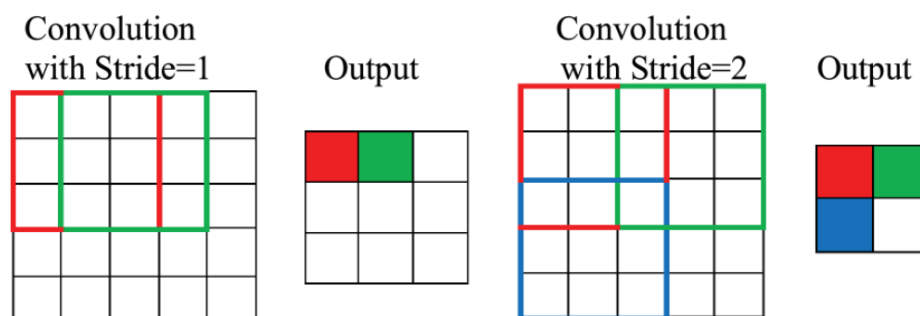
Quá trình trượt các bộ lọc có các giá trị cần lưu ý bao gồm:

- **Padding:** Thuộc tính padding khi sử dụng sẽ thêm chèn vào biên của hình ảnh chiều rộng hoặc chiều cao tùy vào hướng sử dụng padding. Padding thường được sử dụng khi kernel không phù hợp với ảnh đầu vào và ta sẽ chỉnh sửa lại ảnh đầu vào, hoặc có thể dùng để giữ nguyên kích thước đầu vào. Ví dụ padding như ở hình 2.4.



Hình 2.4. Padding [9]

- **Stride:** được minh họa như ở hình 2.5, được hiểu là bước dịch chuyển của bộ lọc theo pixel. Nếu stride bằng 1 thì bộ lọc mỗi lần dịch sẽ dịch qua 1 pixel tại mỗi thời điểm và nếu stride là 2 thì bộ lọc sẽ di chuyển 2 pixel một lúc. Giá trị stride càng lớn thì kết quả càng nhỏ và ngược lại.

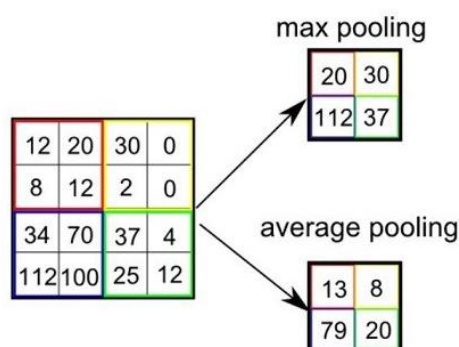


Hình 2.5. Stride [10]

Với mỗi kernel khác nhau chúng sẽ được học các đặc trưng khác nhau, do đó ta có thể sử dụng nhiều loại kernel để có thể học được nhiều loại đặc trưng của đầu vào. Vì mỗi kernel cho ra ngõ ra là một ma trận nên k kernel sẽ cho ra k ma trận đầu ra. Ta kết hợp k ma trận này lại thành 1 tensor 3 chiều có chiều sâu k. Và đầu ra này sẽ qua hàm kích hoạt trước khi vào lớp tích chập tiếp theo.

#### 2.1.5.2. Lớp gộp (Pooling Layer)

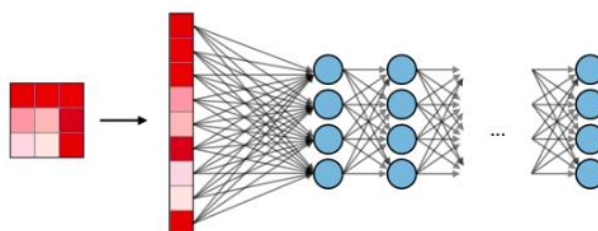
Pooling layer thường được dùng giữa các lớp tích chập, để giảm kích thước dữ liệu nhưng vẫn giữ được các đặc trưng quan trọng. Có nghĩa là, sau khi qua các lớp này đặc trưng sẽ được loại bỏ các thông tin không cần thiết. Trong quá trình này thì padding và stride được sử dụng như phép tính convolution trên ảnh. Thông thường có hai dạng là Max Pooling và Average Pooling. Trong đó, Max Pooling chọn giá trị lớn nhất là hàm đầu ra, còn Average Pooling là lấy trung bình cộng. Trong 2.6 là ví dụ về 2 loại pooling với bộ lọc có kích thước 2x2 và stride = 2.



Hình 2.6. Max pooling và Average Pooling [11]

### 2.1.5.3. Lớp kết nối đầy đủ ( Fully Connected Layer)

Sau khi ảnh đã được truyền qua nhiều lớp tích chập và lớp gộp thì lúc này đặc trưng đã học được tương đối của ảnh đầu vào, và lúc này đầu ra sẽ được làm phẳng thành vector và đưa vào một lớp kết nối đầy đủ như mạng nơ-ron. Cuối cùng dùng softmax hay sigmoid để phân loại đầu ra. Hình 2.7 mô tả lớp kết nối đầy đủ cơ bản.



Hình 2.7. Minh họa về Fully Connected Layer [12]

### 2.1.6. Hàm kích hoạt (Activation Function)

Hàm kích hoạt (activation function) là những hàm phi tuyến được áp dụng vào đầu ra của các nơ-ron trong tầng ẩn của một mô hình mạng ( có nhiệm vụ là chuẩn hoá output của neural) và được sử dụng làm input data cho tầng tiếp theo.

Trong đó, hàm kích hoạt sẽ bao gồm các hàm cơ bản như: Sigmoid, Softmax, Relu, Leaky Relu.

### 2.1.7. Các loại tích chập thay thế tích chập truyền thống

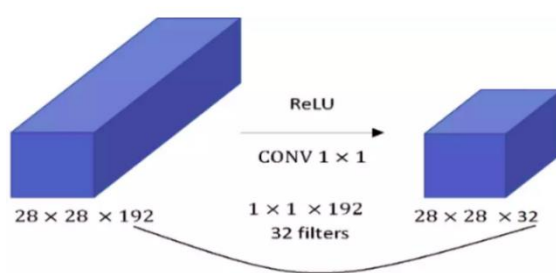
Như được giới thiệu ở trên, lớp tích chập có chức năng trích xuất đặc trưng. Để bài toán đạt được độ chính xác cao thì các mô hình cần phải có lượng tham số lớn do đó yêu cầu khả năng tính toán cao chính vì vậy các thiết bị nhỏ, bị giới hạn bởi khả năng tính toán sẽ gặp nhiều khó khăn. Từ đó có một số loại tích chập có thể thay thế tích chập truyền thống được ra đời để giảm nhiều thông số như là pointwise convolution, grouped convolution, depthwise separable convolution.

#### 2.1.7.1. Pointwise Convolution [13]

Vấn đề thường gặp phải khi xây dựng một mô hình học sâu là số lượng channel sẽ tăng theo chiều sâu của mạng, điều đó dẫn tới số lượng tham số cũng tăng theo dẫn đến tình trạng học quá mức. Do đó chúng ta có thể sử dụng một số phương pháp

để giảm kích thước như là pooling được giới thiệu ở trên, tuy nhiên lớp này gặp phải một vấn đề là không thể điều chỉnh số lượng channel theo mong muốn.

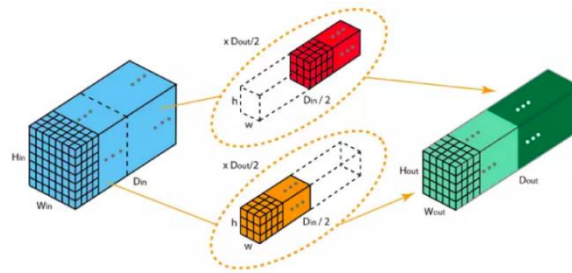
Pointwise convolution là một convolution có hai chiều đầu tiên bằng 1, chiều thứ 3 có kích thước bằng với số channel của input. Ví dụ như ở hình 2.8, ta có đầu vào có kích thước là  $28 \times 28 \times 192$ , ta sẽ có pointwise convolution có kích thước  $1 \times 1 \times 192$ . Ta nhân đầu vào với số filter chính là số channel mà chúng ta muốn ví dụ là 32, ta sẽ có đầu ra có kích thước là  $28 \times 28 \times 32$ . Kích thước rộng cao không thay đổi, chỉ thay đổi mỗi channel điều này giúp điều khiển số channel như chúng ta muốn.



Hình 2.8. Pointwise convolution [13]

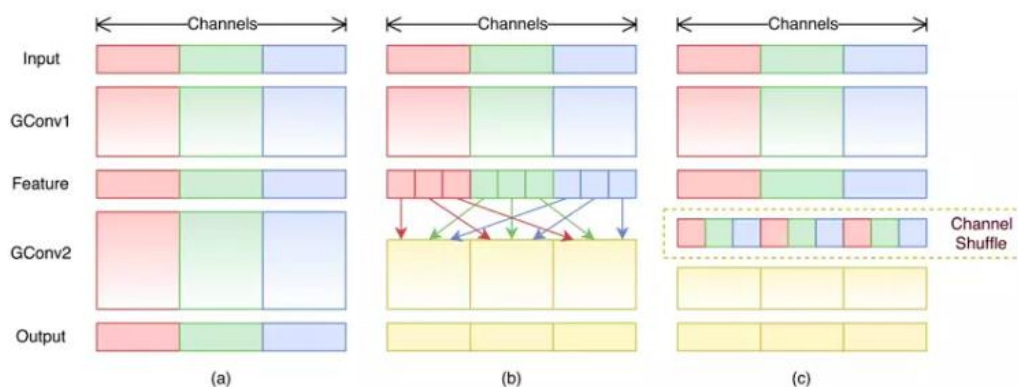
#### 2.1.7.2. Grouped Convolution [13]

Grouped convolution sẽ chia bản đồ đặc trưng thành nhiều nhóm nhỏ để nhân tích chập thay vì toàn bộ kênh như tích chập truyền thống. Mỗi nhóm ở đây sẽ có số kênh cố định sao cho tổng của các nhóm sẽ bằng số kênh của bản đồ đặc trưng ban đầu. Sau đó ta sẽ thực hiện nhân tích chập từng nhóm nhỏ, và mỗi nhóm sẽ có một kernel riêng, sau đó có một lớp concatenate nối chúng lại với nhau để cho số kênh ngõ ra không khác gì so với nhân tích chập truyền thống. Ví dụ như ở Hình 2.9, ta có  $D_{in}$  chính là số channel ở input. Ta sẽ chia input ra thành 2 nhóm, mỗi nhóm có số kênh là  $D_{in} / 2$ . Rồi ta thực hiện nhân tích chập với từng group, ta thu về hai ma trận có kích thước  $h \times w \times D_{out}/2$ . Sau đó chúng ta sử dụng layer concatenate chồng hai ma trận bên trên lại với nhau, ta sẽ thu được ma trận kết quả có kích thước  $h \times w \times D_{out}$ .



Hình 2.9. Minh họa Grouped Convolution [13]

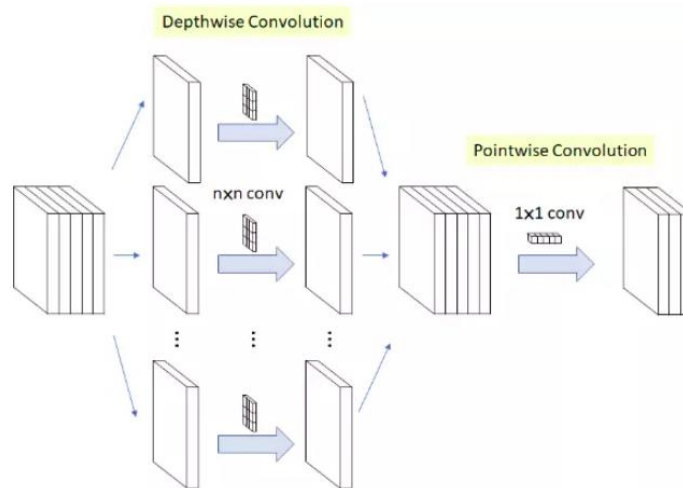
Tuy nhiên có một nhược điểm đó chính là một convolution chỉ nhận một đầu vào có số kênh cố định thay vì toàn bộ số kênh do đó nó chỉ học được một phần nhỏ của đầu ra làm giảm hiệu suất. Do đó Channel Shuffle được sinh ra để giải quyết vấn đề này. Channel Shuffle giúp cho group convolution có thể nhận đầu vào chứa nhiều dữ liệu ngẫu nhiên từ những group khác nhau nên một group sẽ học được toàn bộ đặc trưng từ input, giải quyết hạn chế của group convolution. Minh họa Channel Shuffle ở hình 2.10.



Hình 2.10. Minh họa Channel Shuffle [13]

### 2.1.7.3. Depthwise Separable Convolution [13]

Hình 2.11 minh họa cho Depthwise Separable Convolution và mục đích của convolution này cũng giống grouped convolution là làm giảm số lượng tham số.



Hình 2.11. Minh họa cho Depthwise Separable Convolution [13]

Depthwise Separable Convolution bao gồm hai phần theo thứ tự là:

- Depthwise Convolution: sẽ chia số kênh đầu vào thành các nhóm có số channel cố định là bằng 1 thay vì chia ngẫu nhiên như grouped convolution.
- Pointwise Convolution: dùng để tùy chỉnh số lượng kênh ngõ ra theo mong muốn.

### 2.1.8. Tăng cường dữ liệu (Data Augmentations)

Hiện nay trong các bài toán về học sâu thì vấn đề về dữ liệu đóng vai trò vô cùng quan trọng. Do đó nếu có quá ít dữ liệu dùng để huấn luyện thì model khó có thể tạo ra được kết quả tốt trong việc dự đoán. Vì vậy, kỹ thuật tăng cường dữ liệu được ra đời nhằm tạo ra nhiều kiểu dữ liệu mới dựa trên những dữ liệu mà chúng ta có giúp cho model có thể học đa dạng hơn. Ví dụ như hình 2.12, đây là các hình được tạo ra thêm từ một ảnh gốc ban đầu. Có các loại tăng cường cơ bản như là: chuyển đổi màu, thay đổi độ bão hoà, cân bằng sáng, lật, cắt ngẫu nhiên, thêm nhiễu, thay đổi độ tương phản, xoay, thay đổi kích thước, biến dạng, ...



Hình 2.12. Ví dụ về tăng cường dữ liệu

### 2.1.9. Bài toán phát hiện vật thể (Object Detection)

Bài toán nhận diện vật thể là bài toán nổi bật của thị giác máy tính vì nó được ứng dụng trong nhiều lĩnh vực như: phát hiện khuôn mặt, phát hiện xe, xe không người lái, truy vấn sự kiện, ...

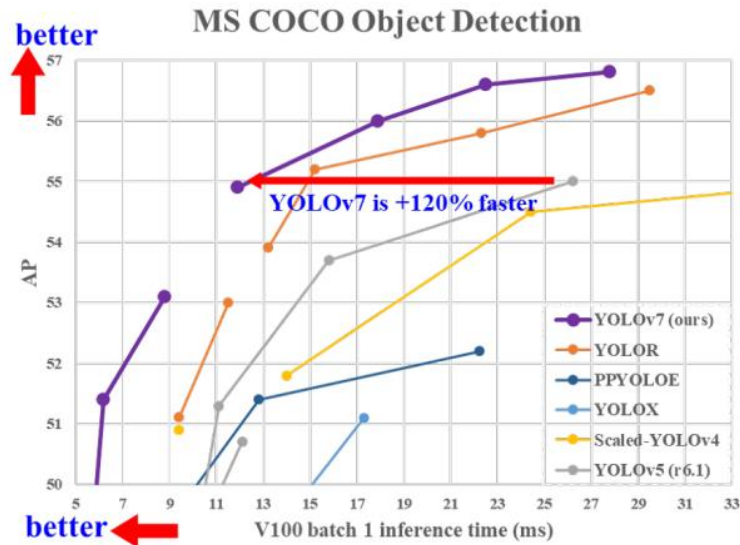
Ta cần xác định rõ nhiệm vụ của bài toán phát hiện vật thể và so sánh với hai khái niệm phân loại hình ảnh, định vị vật thể. Phân loại hình ảnh là dự đoán lớp của một đối tượng nào đó trong một ảnh. Định vị vật thể là xác định vị trí của một hoặc nhiều đối tượng trong một hình và vẽ bounding box xung quanh chúng cuối cùng là phát hiện đối tượng là kết hợp hai nhiệm vụ trên và thực hiện cho một hoặc nhiều đối tượng trong hình ảnh. Chúng ta có thể phân loại ba nhiệm vụ này dễ dàng qua Input và Output của chúng:

- Phân loại hình ảnh:
  - Input: Hình ảnh chỉ chứa một đối tượng.
  - Output: Nhãn của đối tượng đó.
- Định vị đối tượng:
  - Input: Hình ảnh có chứa một hoặc nhiều đối tượng.
  - Output: Một hoặc nhiều bounding box của các đối tượng.
- Phát hiện đối tượng:
  - Input: Hình ảnh có chứa một hoặc nhiều đối tượng.
  - Output: Một hoặc nhiều bounding box và nhãn cho mỗi bounding box.

Dựa vào cách hoạt động của mô hình mà các thuật toán phát hiện vật thể được chia là mạng One-Stage (một giai đoạn) và mạng Two-Stage (hai giai đoạn). Mạng hai giai đoạn như R-CNN [14], Fast-RCNN [14], Faster-RCNN [14],... giai đoạn đầu tiên là phát hiện vùng có chứa vật thể còn có thể gọi là vùng được quan tâm bằng cách sử dụng các mô hình đặc biệt như RPN, và dựa vào vùng chứa vật thể mô hình cố gắng phân loại và xác định vị trí của vật. Bởi vì phải qua hai giai đoạn mà mạng sẽ có độ chính xác về vị trí và độ chính xác về nhận diện cao, tuy nhiên việc trải qua hai giai đoạn làm mạng chậm đi và không phù hợp với các ứng dụng yêu cầu thời gian thực. Đó là lý do vì sao các mạng một giai đoạn như các phiên bản YOLO [15], SSD [16], Nanodet [17],... ra đời, mạng này phát hiện vị trí và phân loại vật thể đồng thời. YOLO [15] chia ảnh đầu vào thành các lưới SxS thay vì các mạng RPN khác, điều này giúp tăng tốc độ phát hiện vật thể một cách đáng kể. Tuy nhiên các mạng này quá nặng nề cho các ứng dụng điện thoại hay các hệ thống nhúng và do đó hiện nay các phiên bản sau của YOLO [15] nhẹ hơn như gần đây nhất là YOLOv7, ngoài ra áp dụng thêm các kỹ thuật “nén model” như Quantization [18], Pruning [18],... .Thì việc triển khai trên các thiết bị nhúng không còn là vấn đề.

#### **2.1.9.1. Kiến trúc YOLOv7 [19]**

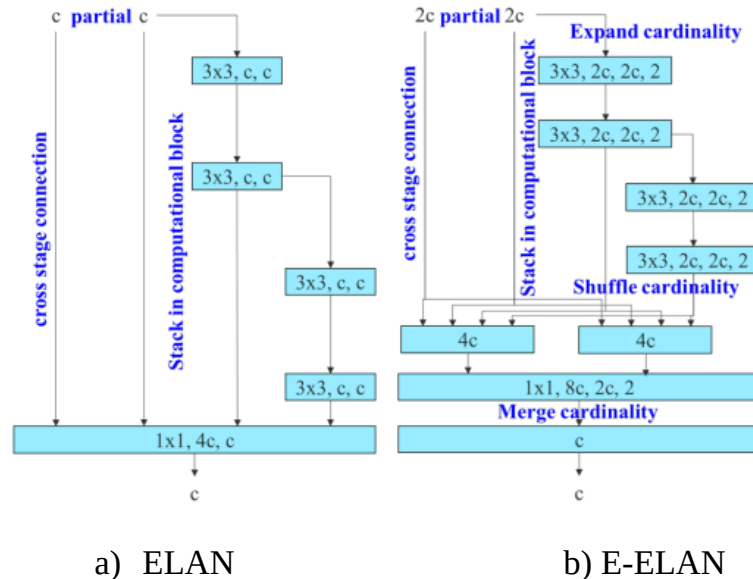
Dựa vào hình 2.13 được tác giả công bố, YOLOv7 [19] vượt qua mọi model Object Detection trong cả tốc độ và độ chính xác từ 5 FPS tới 160 FPS và đạt độ chính xác cao nhất với 56.8% AP trong số toàn bộ các model Object Detection Real-time, có tốc độ 30 FPS hoặc hơn trên GPU V100. YOLOv7-E6 (56 FPS trên V100, 55.9% AP) vượt qua cả backbone nhà Transformer là SWIN-L Cascade-Mask R-CNN (9.2 FPS trên A100, 53.9% AP) với 509% về tốc độ và 2% về AP, hay là cả các backbone CNN cao cấp như ConvNeXt-XL Cascade-Mask R-CNN (8.6 FPS trên A100, 55.2% AP) với 551% về tốc độ và 0.7% về AP. Và đương nhiên là YOLOv7 cũng vượt qua cả: YOLOR, YOLOX, Scaled-YOLOv4, YOLOv5, DETR, Deformable DETR, DINO-5scale-R50, ViT-Adapter-B cũng như là rất nhiều các mạng Object Detection khác cả về mặt tốc độ cũng như là độ chính xác. Hơn nữa, YOLOv7 được train trên COCO từ đầu mà không sử dụng bất kì pretrained nào.



Hình 2.13. So sánh YOLOv7 với những phiên bản YOLO khác

YOLOv7 [19] sử dụng:

- Backbone: ELAN (hình 2.14a), E-ELAN (hình 2.14b)
- Neck: CSP-SPP và (ELAN, E-ELAN)-PAN
- Head: YOLOR [20] + Auxiliary Head [21]
- Khái niệm mới: Label Assignment sẽ được trình bày rõ hơn ở phần 2.1.9.2



Hình 2.14. Backbone

### 2.1.9.2. Label Assignment

Các model Object Detection mạnh từ trước tới nay đều sử dụng kỹ thuật Anchor Box [22]. Tuy nhiên, với sự ra đời của FCOS (Fully Convolutional One-Stage Object Detection) [23], đạt độ chính xác khá cao mà không cần sử dụng đến Anchor Box, đã có khá nhiều sự chú ý dành cho việc loại bỏ Anchor Box [22] ra khỏi các model Object Detection. Ta gọi các dạng model đó là Anchor-free. Anchor-free giúp việc huấn luyện nhanh hơn, dùng ít GPU/CPU hơn là sử dụng Anchor Box (Anchor-based).

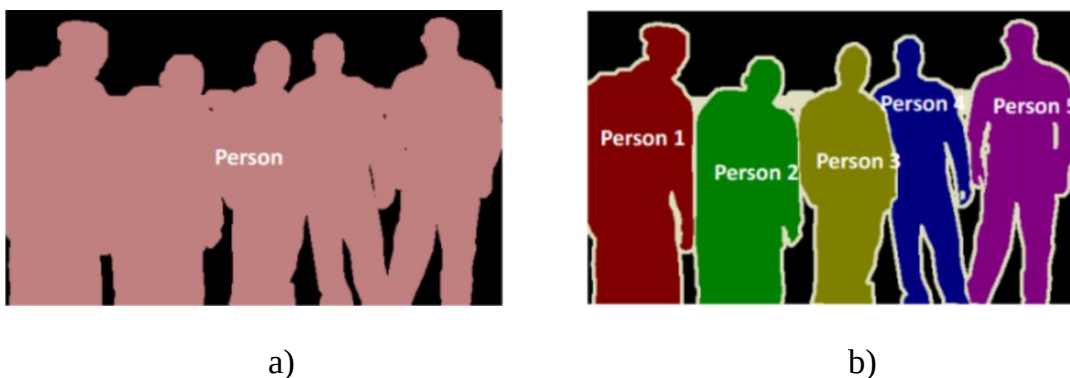
Tuy nhiên, cho tới tận FCOS thì những model Anchor-based vẫn cho ra kết quả tốt hơn Anchor-free. Cho tới tận ATSS (Adaptive Training Sample Selection) [24] thì các phương pháp Anchor-free mới có thể đạt độ chính xác ngang bằng với Anchor-based. ATSS [24] cho rằng sự khác biệt giữa Anchor-free và Anchor-based thực chất nằm ở cách chúng ta định nghĩa thế nào là một positive sample (foreground/FG), thế nào là một negative sample (background/BG). Và nếu positive samples và negative samples được định nghĩa tốt thì độ chính xác của các phương pháp Anchor-free có thể sánh ngang với Anchor-based. Và quá trình tìm ra định nghĩa cho Positive và Negative samples này gọi là quá trình Label Assignment.

Tuy nhiên, YOLOv7 vẫn chọn sử dụng Anchor Box và áp dụng thêm chiến lược Label Assignment là SimOTA (Simple OTA) [25] vào. Về việc không sử dụng Anchor-free, tác giả YOLOv7 giải thích rằng dù SimOTA đã rất tốt rồi nhưng đối với những ảnh có kích cỡ to thì Anchor-based vẫn hội tụ tốt và nhanh hơn, nên tác giả quyết định không sử dụng Anchor-free.

### 2.1.10. Mô hình phân loại hình ảnh (Image Segmentation)

Phân đoạn hình ảnh là bài toán sẽ cho ta kết quả là một hình ảnh đã được phân chia thành nhiều vùng khác nhau và gán nhãn cho chúng. Phân đoạn vật thể yêu cầu độ chính xác cao hơn so với phát hiện vật thể nó cần dự đoán từng pixel, và nhờ vào vùng pixel này ta sẽ biết vị trí của vật thể, hình dạng của chúng. Có hai dạng bài toán phân đoạn hình ảnh chính:

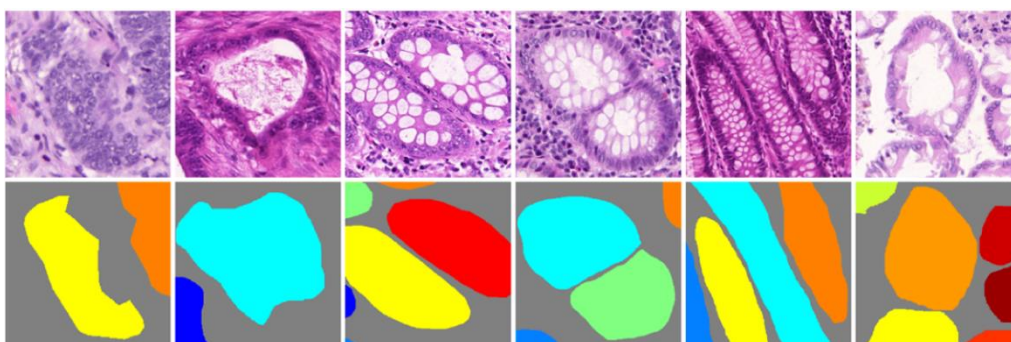
- Phân đoạn ngữ nghĩa (Semantic segmentation): sẽ phân đoạn các vùng ảnh theo từng ảnh khác nhau mà không cần quan tâm giữa các đối tượng trong ảnh đó, ví dụ như hình 2.15 a) chúng ta phân biệt được pixel nào thuộc về người và pixel nào thuộc về background. Tuy nhiên trong bức ảnh xuất hiện 5 người thì không thể xác định được pixel nào thuộc người nào.
- Phân đoạn cá thể (Instance segmentation): chúng ta phân đoạn từng đối tượng trong mỗi ảnh. Ví dụ: ở hình 2.15 b) đối với ảnh người sẽ được phân chia chi tiết đến từng người 1, 2, ... , 5.



Hình 2.15. Phân đoạn ngữ nghĩa và phân đoạn cá thể [26]

Phân đoạn hình ảnh có ứng dụng rất nhiều trong y học, xe tự hành:

- Y học: Hỗ trợ chẩn đoán bệnh thông qua X-Quang. Ưu điểm: Biết được cả vị trí và hình dạng của khối u đó thông qua đó bác sĩ có thể dự đoán được bệnh. Ví dụ như hình 2.16.



Hình 2.16. Phát hiện khối U-Net [26]

- Xe tự hành: Xe tự hành là bài toán yêu cầu hệ thống phải xử lý thông tin liên tục trong môi trường. Và đây là bài toán yêu cầu độ chính xác cao vì tính an toàn nên một hệ thống xe tự hành cần phải hoạt động tốt các tác vụ

sau như xác định các vật thể xuất hiện khi tham gia giao thông, nhận diện biển báo, phát hiện làn đường, .... Ví dụ như hình 2.17



Hình 2.17. Xe tự hành [26]

Bởi vì sự thành công của mạng học sâu, các phương pháp tích chập được đề xuất rộng rãi trong mạng phân đoạn ngữ nghĩa. Unet [27] là một trong những mô hình nổi bật trong số đó về hiệu suất. Unet là một bộ mã hóa tự động có các kết nối từ bộ mã hóa và giải mã. Các loại liên kết này giúp mạng kết hợp các đặc điểm từ các vùng không gian khác nhau của hình ảnh từ đó giúp việc phân đoạn chính xác, mặc dù thiết kế này nhanh hơn các cách tiếp cận phân loại từng pixel nhưng chúng vẫn không phù hợp với hệ thống thời gian thực. Vì vậy kiến trúc Enet ra đời, kiến trúc này ít hơn hai lần tham số so với Enet [28]. Ý tưởng chính đằng sau Enet [28] là việc sử dụng bộ mã hóa lớn hơn và một bộ giải mã nhỏ hơn so với các công trình trước đây. Với thiết kế như vậy, các tác giả hy vọng có nhiều tính năng hữu ích hơn trong bộ mã hóa, trong khi bộ giải mã chỉ chịu trách nhiệm lấy mẫu đầu ra của bộ mã hóa và tinh chỉnh lại các chi tiết. Bên cạnh đó việc sử dụng các tích chập giãn, chập không đối xứng và các phương pháp tiếp cận khác ảnh hưởng đến sự phân bố trọng lượng của các lớp học sâu và cũng giúp tăng hiệu quả của mô hình.

### 2.1.11. Các phương pháp đánh giá

#### 2.1.11.1. IoU

IoU – Intersection over Union [29] là một chỉ số dùng để đánh giá phần trăm trùng lặp giữa nhãn dán và đầu ra kết quả dự đoán và được tính theo công thức

(2.1). Nếu ta gọi A là tổng số điểm ảnh của mặt nạ được dán nhãn; B là tổng số điểm ảnh của mặt nạ đầu ra dự đoán. IoU được sử dụng để đánh giá độ chính xác của phân đoạn.

$$IoU = \frac{A \cap B}{A \cup B} \quad (2.1)$$

#### 2.1.11.2. Precision, Recall, AP, mAP [29]

Trong bài toán nhận diện vật thể thì mô hình cần dự đoán bounding box nào trong ảnh là có (Positive) hoặc không có (Negative) vật thể. Và có thể mô hình dự đoán đúng (True) hoặc sai (False) so với thực tế. Vì thế có 4 trạng thái của một kết quả được mô tả trong bảng 2.1.

Bảng 2.1. Phân loại đánh giá kết quả dự đoán của mô hình

|         |           | Mô hình dự đoán |              |
|---------|-----------|-----------------|--------------|
|         |           | Positive - P    | Negative - N |
| Thực tế | True – T  | TP              | TN           |
|         | False - F | FP              | FN           |

Precision được tính theo công thức (2.2) là thông số đánh giá độ tin cậy của kết quả model dự đoán được. Recall được tính theo công thức (2.3) là thông số đánh giá khả năng tìm kiếm toàn bộ kết luận đúng của model trong kết luận đúng thực tế.

Hiện nay AP và mAP là chỉ số đánh giá mô hình nhận diện vật thể được sử dụng phổ biến nhất. Nó tính toán dựa vào 3 thông số IoU, Precision, Recall trên cả tập dữ liệu. AP, mAP được tác giả giới thiệu chi tiết tại [29].

$$precision = \frac{TP}{TP+FP} \quad (2.2)$$

$$recall = \frac{TP}{TP+FN} \quad (2.3)$$

#### 2.1.12. Phương pháp tiếp cận đa tác vụ

Multitask learning là một phương pháp học máy sử dụng cùng một mô hình để học nhiều tác vụ khác nhau cùng một lúc. Lợi thế chính của multitask learning là:

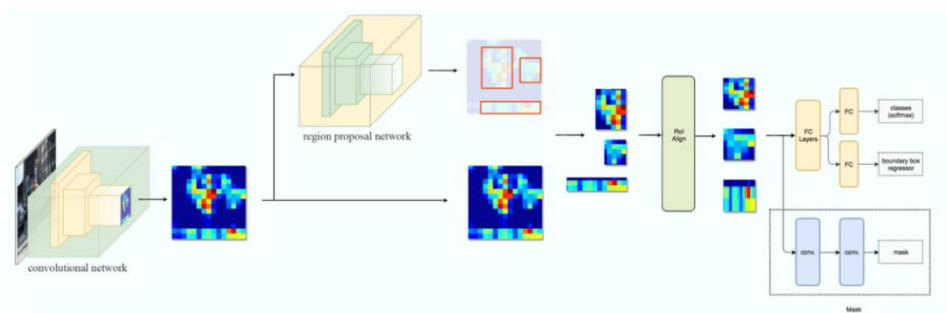
1. Tiết kiệm tài nguyên: sử dụng một mô hình duy nhất cho nhiều tác vụ giúp tiết kiệm tài nguyên và chi phí hơn.

2. Tăng khả năng generalization: học nhiều tác vụ cùng một lúc giúp mô hình có khả năng generalization tốt hơn trong các tác vụ khác nhau.

3. Tăng độ chính xác: học nhiều tác vụ cùng một lúc giúp mô hình có thể học được thông tin chung giữa các tác vụ, do đó tăng độ chính xác trong các tác vụ khác nhau.

4. Tăng khả năng sử dụng thông tin: học nhiều tác vụ cùng một lúc giúp mô hình có thể sử dụng thông tin giữa các tác vụ để học tốt hơn.

Mục đích của việc học đa tác vụ là học tốt hơn bằng cách cho phép mạng được đào tạo từ nhiều nhiệm vụ khác nhau nhưng có liên quan. Vì vậy nhiều mạng đa tác vụ được đề xuất gần đây để kết hợp phát hiện vật thể và phân đoạn hình ảnh. Ví dụ mạng Mask RCNN [30] như hình 2.18 mở rộng từ Faster RCNN [14] bằng cách thêm một nhánh dự đoán mặt nạ của vật thể cái mà kết hợp nhiệm vụ phân đoạn và phát hiện vật thể. Bằng cách thiết kế này, hai nhiệm vụ có thể thúc đẩy hiệu suất của nhau một cách hiệu quả.



Hình 2.18. Kiến trúc Mask R-CNN [31]

Nguyên lý hoạt động của mạng qua các bước :

- Đầu tiên là cần một backbone có tác dụng trích lọc đặc trưng ảnh, sau đó output là một feature map ở layer cuối.
- Feature map tiếp tục được huấn luyện qua một mạng RPN để tìm các ROI ( là những vùng có khả năng cao chứa vật thể)
- ROI sẽ được cắt từ feature map và sử dụng ROI pooling layer để reshape ảnh thành một khối cùng kích thước vì lớp sau sẽ là lớp fully connected.
- Khối ROI tiếp tục truyền qua các lớp fully connected và tách làm 2 nhánh, một là nhánh dự báo bounding box và một nhánh dự báo nhãn.

Mặc dù Mask RCNN [29] đem lại độ chính xác cao, Tuy nhiên, được phát triển từ Faster RCNN [14] vì thế tốc độ bị chậm và đầu vào của nhánh dự đoán mặt nạ chỉ có vùng quan tâm của vật thể điều này không phù hợp cho việc phát hiện làn đường.

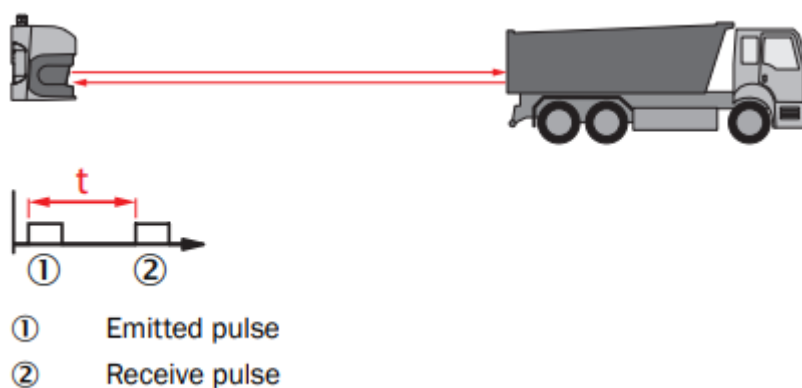
## 2.2. LIDAR

LiDAR là cảm biến 2D (hoặc 3D) được dùng để đo khoảng cách bằng chùm tia laser. Cảm biến được áp dụng rộng rãi trong các xe tự hành AGV vận hành ở các nhà máy sản xuất hiện đại, đo kích thước, nhận dạng xe ô tải, xe container tại các trạm kiểm soát hay hệ thống kiểm soát hàng hóa ở các khu vực hàng hóa cảng.

### 2.2.1. Nguyên lý hoạt động

Nguyên tắc hoạt động của thiết bị là sử dụng nguồn phát một chùm tia laser (với bước sóng tầm vài trăm nano mét) qua một tấm gương được gắn mô-tơ quay 360 độ để truyền đến vật thể và sử dụng một đi-ốt quang để nhận tín hiệu chùm tia phản xạ. Với bước sóng dài, chùm tia laser tới vật thể sẽ không đi xuyên qua mà được vật thể hấp thụ hoặc phản xạ lại thiết bị.

Dựa vào thời gian tia đi tới và tia phản xạ, bộ phận xử lý của LiDAR tiến hành tính toán khoảng cách đến vật thể như hình 2.19.



Hình 2.19. Nguyên lý tính toán khoảng cách của thiết bị [29]

### 2.2.2. Ứng dụng trên LIDAR

#### 2.2.2.1. SLAM

SLAM (viết tắt của simultaneous localization and mapping) là phương pháp dùng trong lĩnh vực xe tự hành, trong đó thuật toán này hỗ trợ xây dựng bản đồ đồng thời

xác định được vị trí hiện tại của ô tô trên bản đồ đó. Các phần mềm dùng thông tin vị trí đó để thực hiện các tác vụ của xe tự hành như thiết lập các con đường xe cần đi (Path-Planning) và tránh vật cản trên đường đi.

Hiện tại có rất nhiều hình thức để áp dụng cho việc thực thi thuật toán SLAM bao gồm sử dụng camera (Visual SLAM), sử dụng LiDAR (LiDAR SLAM),... Mỗi một thiết bị có điểm mạnh và yếu riêng. Riêng với LiDAR SLAM, thuật toán sử dụng các chùm tia laser của thiết bị LiDAR để quét môi trường xung quanh, sau đó dùng phần mềm xử lý, tính toán để xây dựng nên bản đồ.

#### **2.2.2.2. DATMO (Detection and tracking of moving object)**

Thuật toán sử dụng chùm tia laser quét môi trường xung quanh. Chùm tia quét được phân ra thành nhiều phần (segment) khác nhau. Điểm đặc biệt trong thuật toán này là khi phát hiện có đối tượng (đối tượng được quy ước từ trước) trong vùng quét thì tần số quét chùm tia laser được tính toán sao cho tập trung vào đối tượng đó là nhiều nhất. Nhờ đặc điểm trên mà thuật toán có thể tính toán, xác định được vị trí hiện tại và tốc độ di chuyển của đối tượng so với cảm biến LiDAR là bao nhiêu. Tính năng này được áp dụng vào trong hệ thống Adaptive Cruise Control trên xe ô tô hay dùng để điều khiển tránh vật cản trên xe tự hành.

### **2.3. Hệ thống thông tin**

#### **2.3.1. Giới thiệu về mạng giao tiếp trên ô tô**

Mạng giao tiếp trên ô tô là một hệ thống bao gồm các hộp ECU có chức năng riêng biệt như ECM, TCM, BCM, ABS v.v. Hộp điều khiển này kết nối và giao tiếp với nhau để trao đổi thông tin, mỗi hộp chỉ tiếp nhận thông tin cần thiết. Mạng giao tiếp trên các phương tiện giao thông khác cũng tối ưu hóa việc điều khiển và hạn chế dây dẫn. Mạng giao tiếp trên ô tô sử dụng rất ít dây dẫn nhưng vẫn truyền tải dữ liệu nhiều với tốc độ cao.

Các loại mạng giao tiếp phổ biến trên ô tô:

❖ Mạng giao tiếp CAN là một giao thức kết nối phổ biến trên ô tô hiện nay với tốc độ truyền tải lên đến 1Mb/s, độ chính xác cao và ít bị nhiễu.

- ❖ Mạng giao tiếp LIN là mạng giao tiếp phổ biến thứ 2, thường được sử dụng trong hệ thống body với tốc độ truyền thấp và giao tiếp 2 chiều.
- ❖ Mạng giao tiếp MOST là mạng truyền cáp quang với tốc độ truyền tải tối đa 150 Mb/s, thường được sử dụng trong mạng thông tin giải trí trên xe.
- ❖ Mạng giao tiếp FlexRay là một giao thức truyền thông mạng nội bộ cho ô tô được phát triển bởi FlexRay Consortium. Nó được thiết kế để nhanh hơn và đáng tin cậy hơn giao thức CAN, tuy nhiên giá thành của FlexRay cao hơn.

### 2.3.2. Tổng quan về mạng CAN ứng dụng chuẩn giao tiếp trên ô tô

#### 2.3.2.1. Các khái niệm cơ bản về mạng CAN

Mục này trình bày các thuật ngữ và khái niệm cơ bản trước khi đi sâu phân tích chi tiết chuẩn BUS CAN:

*Node (Station)*: là những thành phần độc lập có thể xử lý truyền nhận dữ liệu trên BUS CAN, một Node thường có 3 thành phần cơ bản là vi điều khiển MCU, chip điều khiển CAN (CAN controller) và chip thu-phát (CAN transceiver).

*Thông điệp (Message)*: Thông tin trên BUS CAN được gửi dưới dạng các thông điệp có định dạng cố định. Các thông điệp có thể khác nhau nhưng độ dài (số bit trong một thông điệp) là có giới hạn. và được giới hạn về độ dài. Khi BUS rảnh (IDLE) thì bất kỳ Node nào trên BUS đều có thể bắt đầu truyền một thông điệp mới. Thông điệp được truyền thông qua 4 loại khung (frame) khác nhau là *khung dữ liệu (Data frame)*, *khung yêu cầu hay khung điều khiển (remote frame)*, *khung báo lỗi (Error frame)* và *khung báo quá tải (Overload frame)*.

*Tốc độ bit (Bit rate)*: Tốc độ bit của CAN có thể khác nhau trong các hệ thống khác nhau nhưng trong một hệ thống cho trước thì tốc độ bit đồng nhất và cố định. Tốc độ bit còn tùy thuộc vào chiều dài đường truyền. Tốc độ tối đa có thể lên đến 1 Mbit/s. Sau đây là bảng 2.2 thể hiện một số thông số tham khảo thực tế:

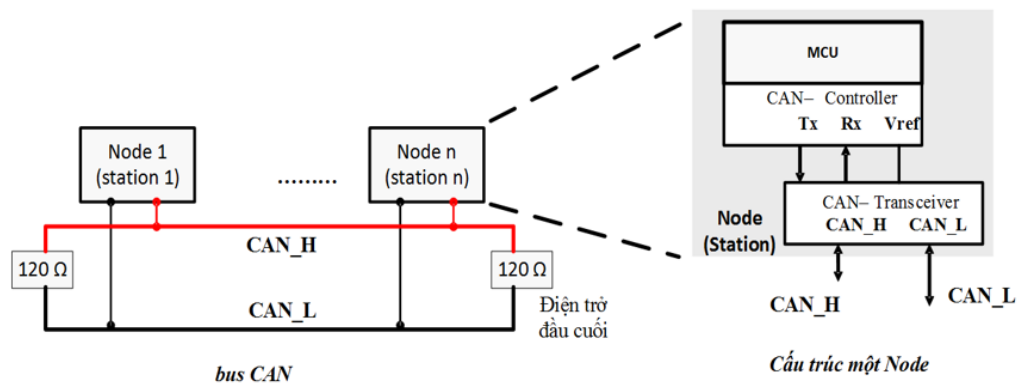
Bảng 2.2. Bảng tốc độ truyền bit

| Tốc độ bit<br>(Bit Rate – kbit/s) | Chiều dài BUS<br>(BUS Length - m) | Thời gian bit danh định<br>(Nominal Bit-time - $\mu$ s) |
|-----------------------------------|-----------------------------------|---------------------------------------------------------|
| 1000                              | 30                                | 1                                                       |
| 800                               | 50                                | 1.25                                                    |
| 500                               | 100                               | 2                                                       |
| 250                               | 250                               | 4                                                       |
| 125                               | 500                               | 8                                                       |
| 62.5                              | 1000                              | 20                                                      |
| 20                                | 2500                              | 50                                                      |
| 10                                | 5000                              | 100                                                     |

### 2.3.2.2. Cấu trúc và các thành phần trong CAN BUS

#### 2.3.2.2.1. Cấu trúc mạng CAN

Cấu trúc mạng CAN (hình 2.20) gồm các thành phần cơ bản sau:



Hình 2.20. Cấu trúc BUS và Node của mạng CAN

BUS CAN là BUS vi sai bao gồm:

Dây cáp (hình 2.21) gồm hai dây riêng biệt được gọi là CAN\_H (CAN High) và CAN\_L (CAN Low). Tùy vào mỗi ứng dụng mà hai dây này có thể là dây xoắn kép hay cáp quang. Ngoài ra, dây cáp còn có vỏ bọc chống nhiễu, vỏ này được nối đến một điện áp tham khảo hoặc GND. CAN\_H có điện áp cao, thường là 0V khi BUS không truyền, nhận dữ liệu. CAN\_L có điện áp thấp, thường là

+5V khi BUS không truyền, nhận dữ liệu. Như đã nói, BUS CAN là BUS vi sai, khi dữ liệu được truyền trên BUS, điện áp trên CAN\_H và CAN\_L sẽ thay đổi, mức logic 0/1 sẽ được xác định căn cứ trên sự chênh lệch (sai lệch) điện áp giữa CAN\_H và CAN\_L. Điện trở đầu cuối của đường dây là **120Ω**.



Hình 2.21. Dây cáp BUS CAN [8]

Node (station): là một thành phần kết nối đến BUS CAN thông qua 2 dây CAN\_H và CAN\_L. Node này là các bo mạch hoặc Module điều khiển.

#### 2.3.2.2.2. Cấu trúc Node hay Station

*Microcontroller - MCU* thực thi các chức năng chính, điều khiển chính của một Node. Thực hiện cấu hình hoạt động cho CAN controller, phân phối dữ liệu cần truyền đến CAN controller, lấy dữ liệu nhận từ CAN controller để sử dụng cho hoạt động của Node.

*CAN controller* thực thi các xử lý về truyền nhận dữ liệu, báo lỗi, tính toán thời gian bit, ... theo chuẩn CAN quy định; phát dữ liệu cần truyền dạng số (theo mức logic 0/1) ra chân TX; nhận dữ liệu dạng số qua chân RX. CAN controller có thể là một chip (vi mạch) độc lập.

*CAN transceiver* hoạt động như bộ chuyển đổi từ tín hiệu số (mức logic 0/1) trên đường TX thành tín hiệu tương tự trên BUS CAN và ngược lại, chuyển đổi từ tín hiệu tương tự trên BUS CAN (CAN\_H và CAN\_L) thành tín hiệu số trên đường RX.

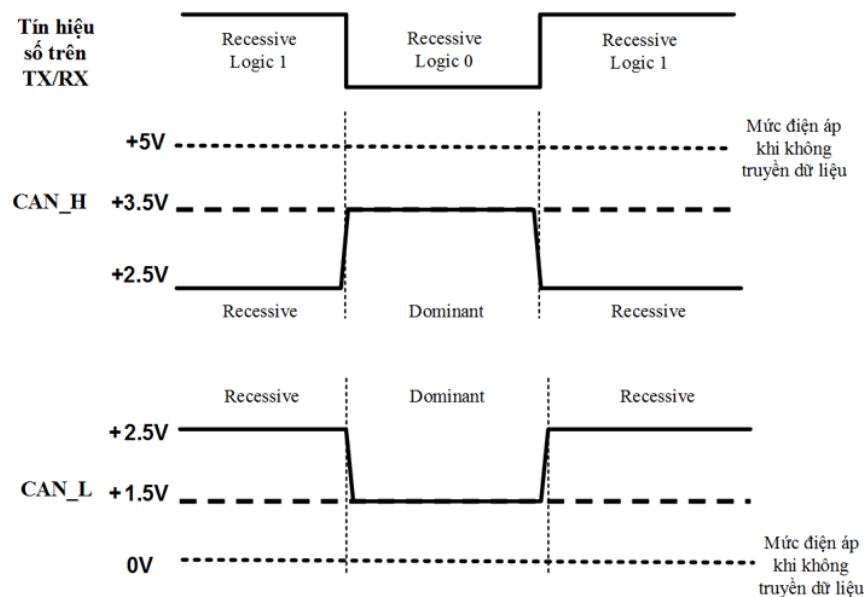
#### 2.3.2.2.3. Tín hiệu trên BUS CAN

Như đã trình bày, CAN transceiver chuyển đổi qua lại giữa tín hiệu số (giao tiếp với CAN controller) và tương tự (giao tiếp với BUS CAN). Trong giao thức CAN (hình 2.22):

Mức dominant, nghĩa là "mức trội" hay "mức chiếm ưu thế", là mức logic 0

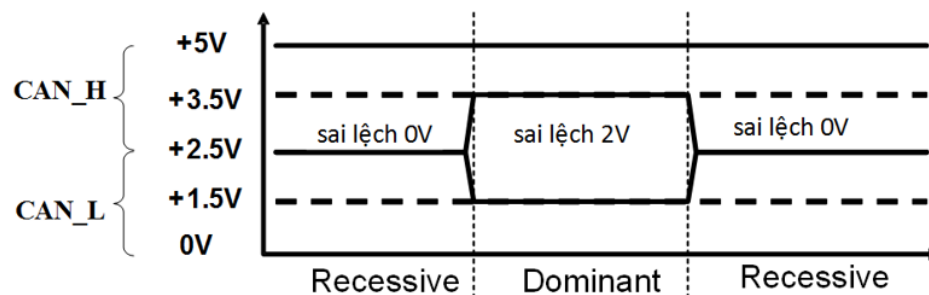
Mức recessive, nghĩa là "mức lặn" hay "mức ẩn", là mức 1

Điều \_L, đến +7V trên CAN\_H nhưng được sử dụng phổ biến là 0V trên CAN\_L và +5V trên CAN\_H khi dùng ở tốc độ cao 1Mbit/s. Đường CAN\_H có mức áp +5V khi ở trạng thái nghỉ và sẽ sụt áp còn +3.5V khi đang hoạt động. Lúc này, mức +3.5V được quy định là mức “dominant” và +2.5V được quy định là mức “recessive”. Đường CAN\_L có mức áp 0V khi ở trạng thái nghỉ và sẽ tăng lên +1.5V khi hoạt động. Lúc này, mức +1.5V được quy định là mức “dominant” và +2.5V được quy định là mức “recessive”.



Hình 2.22. Mức điện áp trên đường CAN\_H và CAN\_L khi truyền dữ liệu

Trạng thái BUS dựa vào sự sai lệch áp giữa CAN\_H và CAN\_L như hình 2.23. Cụ thể, nếu sai lệch điện áp thấp hơn điện áp ngưỡng tối thiểu thì đó là mức recessive (mức 1), thường dưới +0.5V. Nếu sai lệch điện áp cao hơn điện áp ngưỡng tối thiểu thì đó là mức dominant (mức 0), thường trên +0.9V. Nếu sai lệch áp rơi vào khoảng từ 0.5V đến 0.9V thì không thể phân biệt được mức logic và gây sai dữ liệu.



Hình 2.23. Xác định mức điện áp logic qua sự sai lệch CAN\_L và CAN\_H

### 2.3.2.3. Cấu trúc các khung dữ liệu trong CAN [32]

Giao thức CAN sử dụng 4 loại khung (frame) khác nhau để truyền tải dữ liệu và điều khiển. Bốn loại khung này gồm:

Khung dữ liệu (Data frame) là khung mang dữ liệu từ một bộ truyền đến các bộ nhận. Khung này có vùng để mang các byte dữ liệu.

Khung yêu cầu hay khung điều khiển (Remote frame) là khung được truyền từ một Node để yêu cầu Node khác truyền khung dữ liệu có ID (IDENTIFIER) trùng với khung yêu cầu.

Khung báo lỗi (Error frame) là khung được truyền bởi bất kỳ Node nào khi Node đó phát hiện lỗi BUS.

Khung báo quá tải (Overload frame) được sử dụng để tạo thêm độ trễ giữa các khung dữ liệu hoặc khung yêu cầu. Mỗi Node trong BUS CAN có thể truyền bất kỳ khi nào nếu phát hiện BUS rảnh, nếu một Node nhận quá nhiều dữ liệu, nó có thể dùng khung này để ngăn sự truyền tiếp theo.

### 2.3.3. Các toán tử bitwise

Toán tử AND(&) (bảng 2.3): Đây là toán tử thao tác bit lấy 2 toán hạng nhị phân có chiều dài bằng nhau và thực hiện phép toán lý luận AND trên mỗi cặp bit tương ứng bằng cách nhân chúng lại với nhau. Nhờ đó, nếu cả hai bit ở vị trí được so sánh đều là 1, thì bit hiển thị ở dạng nhị phân sẽ là 1 ( $1 \times 1 = 1$ ); ngược lại thì kết quả sẽ là 0 ( $1 \times 0 = 0$ ).

Bảng 2.3. Bảng chân trị cho AND

| <b>A</b> | <b>B</b> | <b>A&amp;B</b> |
|----------|----------|----------------|
| 0        | 0        | 0              |
| 0        | 1        | 0              |
| 1        | 0        | 0              |
| 1        | 1        | 1              |

Ta có  $A = 5$ ,  $B = 3$  khi đó  $A \& B \Rightarrow 0101 \& 0011 = 0001 \Rightarrow A \& B = 1$

Toán tử OR(|) (bảng 2.4): Phép toán trên thao tác bit OR lấy hai dãy bit có độ dài bằng nhau và thực hiện phép toán lý luận bao hàm OR trên mỗi cặp bit tương ứng. Kết quả ở mỗi vị trí sẽ là 0 nếu cả hai bit là 0, ngược lại thì kết quả là 1.

Bảng 2.4. Bảng chân trị của OR

| <b>A</b> | <b>B</b> | <b>A B</b> |
|----------|----------|------------|
| 0        | 0        | 0          |
| 0        | 1        | 1          |
| 1        | 0        | 1          |
| 1        | 1        | 1          |

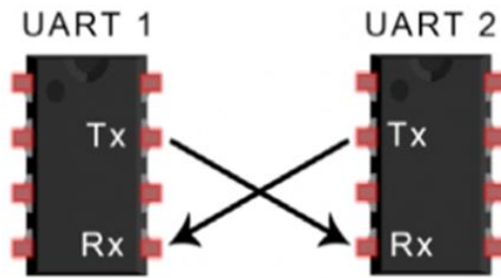
Thao tác dịch bit sang phải (>>): phép dịch chuyển số học sang phải  $n$  lần của một giá trị bù 2 thì tương đương với việc chia cho  $2^n$  và làm tròn về phía âm vô cùng. Nếu số nhị phân được xem là bù 1, thì phép dịch chuyển sang phải tương tự sẽ cho kết quả bằng với việc chia số đó cho  $2^n$  và làm tròn về phía 0.

Thao tác dịch bit sang trái (<<): Dịch chuyển số học bên trái  $n$  lần tương đương nhân với  $2^n$  (nếu giá trị đó không gây tràn bộ nhớ).

### 2.3.4. Các chuẩn giao tiếp khác

#### 2.3.4.1. Chuẩn giao tiếp UART [33]

UART hay bộ thu-phát không đồng bộ đa năng là một trong những hình thức giao tiếp kỹ thuật số giữa thiết bị với thiết bị đơn giản và lâu đời nhất. Các UART giao tiếp giữa hai nút riêng biệt bằng cách sử dụng một cặp dẫn và một nối đất chung.



Hình 2.24. Sơ đồ giao tiếp Uart

Trong một sơ đồ giao tiếp UART (hình 2.24):

- Chân Tx (truyền) của một chip kết nối trực tiếp với chân Rx (nhận) của chip kia và ngược lại. Thông thường, quá trình truyền sẽ diễn ra ở 3.3V hoặc 5V. UART là một giao thức một master, một slave.
- Dữ liệu truyền đến và đi từ UART song song với thiết bị điều khiển (ví dụ: CPU).
- Khi gửi trên chân Tx, UART đầu tiên sẽ dịch thông tin song song này thành nối tiếp và truyền đến thiết bị nhận.
- UART thứ hai nhận dữ liệu này trên chân Rx của nó và biến đổi nó trở lại thành song song để giao tiếp với thiết bị điều khiển của nó.

Quá trình truyền dữ liệu diễn ra dưới dạng các gói dữ liệu, bắt đầu bằng một bit bắt đầu, đường mức cao được kéo xuống đất. Sau bit bắt đầu, năm đến chín bit dữ liệu truyền trong khung dữ liệu của gói, theo sau là bit chẵn lẻ tùy chọn để xác minh việc truyền dữ liệu thích hợp. Cuối cùng, một hoặc nhiều bit dừng được truyền ở nơi đường đặt ở mức cao. Như vậy là kết thúc một gói.

#### 2.3.4.2. Chuẩn giao tiếp SPI [34]

Giao tiếp SPI là chuẩn truyền thông nối tiếp tốc độ cao được phát triển bởi Motorola. Đây là kiểu truyền thông Master-Slave, trong đó master điều khiển slave. Cấu hình đơn giản nhất của giao tiếp SPI là hệ thống một slave và một master duy nhất, phức tạp hơn thì một master có thể được cấu hình để điều khiển nhiều hơn một slave. Giao tiếp SPI cho phép truyền dữ liệu mà không bị gián đoạn và quá trình truyền và nhận có thể diễn ra đồng thời giữa master và slave.

Giao tiếp SPI (hình 2.25) sử dụng 4 dây để kết nối giữa slave và master nên giao tiếp SPI hay còn gọi là chuẩn truyền thông giao tiếp 4 dây bao gồm: SCK (Serial Clock), MISO (Master Input Slave Output), MOSI (Master Output Slave Input) và SS (Slave Select).

Đối với Arduino Uno các chân giao tiếp SPI Lần lượt là SS-10; MOSI-11; MISO-12; SCK-13.



Hình 2.25. Các chân giao tiếp SPI của Arduino Uno

## 2.4. GIAO DIỆN NGƯỜI DÙNG (UI)

UI viết tắt của User Interface là một khái niệm để nói tới nơi mà con người và máy móc cùng làm việc với nhau. Với sự ra đời của máy tính, UI có thể coi là những gì chúng ta nhìn thấy trên màn hình và tương tác với máy tính thông qua những câu lệnh được mã hóa. Đơn giản hơn UI chính là cách người dùng nhìn thấy thiết kế của chương trình trên desktop, laptop, máy tính cầm tay (table) hay smartphone.

Ví dụ: Trong một chiếc smartphone, màn hình người dùng của ứng dụng như Google Maps hay Facebook, thông qua đó người dùng có thể đưa vào thông tin và nhận phản hồi, đó chính là giao diện người dùng UI.

### 2.4.1. Ngôn ngữ front-end

Front-end [35] là một khái niệm trong lập trình web, nó chỉ ra phần của một trang web mà người dùng có thể thấy và tương tác trực tiếp, bao gồm HTML, CSS và JavaScript. Nó điều khiển cách trang web hiển thị và hoạt động. Các nghề nghiệp liên quan đến front-end development bao gồm web design và web development.

#### **2.4.1.1. Ngôn ngữ HTML**

HTML (Hypertext Markup Language) là ngôn ngữ đánh dấu được sử dụng để tạo các trang web và các thông tin khác có thể được hiển thị trong trình duyệt web. HTML được sử dụng để cấu trúc nội dung của trang web, bao gồm văn bản, hình ảnh và các yếu tố đa phương tiện khác.

HTML được sử dụng cùng với các công nghệ khác như CSS và JavaScript để tạo các trang web và ứng dụng web nâng cao hơn. CSS (Cascading Style Sheets) được sử dụng để xác định cách trình bày và bố cục của trang web, trong khi JavaScript được sử dụng để thêm tính tương tác và hành vi động cho trang web. Cùng với nhau, những công nghệ này giúp tạo ra trải nghiệm web tương tác, phong phú có thể chạy trên nhiều loại thiết bị.

#### **2.4.1.2. Ngôn ngữ CSS**

CSS (Cascading Style Sheets) là ngôn ngữ được sử dụng để mô tả giao diện và định dạng của nội dung được viết bằng HTML. CSS cung cấp thông tin về kiểu dáng chẳng hạn như màu sắc, phông chữ, bố cục và khoảng cách cho nội dung HTML, nếu không thì nội dung này sẽ chỉ là văn bản thuần túy. CSS tách phần trình bày của nội dung web khỏi cấu trúc của nó, giúp việc duy trì và sửa đổi giao diện của trang web trở nên dễ dàng hơn.

CSS có thể được áp dụng cho nhiều trang web cùng một lúc và có thể xác định các kiểu khác nhau cho các thiết bị khác nhau, chẳng hạn như điện thoại thông minh và máy tính để bàn. Điều này cho phép trình bày nhất quán trên các thiết bị và giúp giảm số lượng công việc cần thiết để duy trì trang web.

CSS cung cấp rất nhiều chức năng và tính năng, bao gồm khả năng đặt nền, thay đổi kích thước và màu sắc của văn bản, thay đổi khoảng cách giữa các phần tử, v.v. CSS cũng có thể được sử dụng để tạo hoạt ảnh, thiết kế đáp ứng và tính tương tác. Nói tóm lại, CSS đóng một vai trò quan trọng trong việc làm cho trang web trở nên hấp dẫn về mặt trực quan, thân thiện với người dùng và đáp ứng các thiết bị khác nhau.

### 2.4.1.3. Ngôn ngữ Java Script

Java Script là ngôn ngữ lập trình cấp cao, được gõ động, được diễn giải, được sử dụng rộng rãi để phát triển web và kịch bản phía người dùng (client). Nó được tạo ra vào năm 1995 bởi Brendan Eich và ban đầu chỉ được sử dụng để thêm tính tương tác và hành vi động cho các trang web.

Mã JavaScript được thực thi bởi trình duyệt web của khách hàng, có nghĩa là mã này có thể chạy tự nhiên trên thiết bị của người dùng, mang lại trải nghiệm người dùng nhanh và nhạy. Mã JavaScript được viết trong các tệp .js, sau đó các tệp này được liên kết với một trang web bằng cách sử dụng thẻ script trong HTML. JavaScript là ngôn ngữ lập trình quan trọng trong phát triển web được hỗ trợ bởi tất cả các trình duyệt chính. Nó được sử dụng để tạo các ứng dụng web phức tạp, phát triển web mặt trước và mặt sau, phát triển ứng dụng di động và viết mã phía máy chủ bằng Node.js.

### 2.4.2. API

API (Application Programming Interface) là một tập hợp các giao thức, quy trình và công cụ để xây dựng phần mềm và ứng dụng. API xác định cách các hệ thống phần mềm khác nhau giao tiếp với nhau và cho phép trao đổi dữ liệu giữa chúng. API cho phép nhà phát triển tương tác với các dịch vụ hoặc dữ liệu từ các nguồn bên ngoài, chẳng hạn như dịch vụ web, cơ sở dữ liệu hoặc các hệ thống khác, bằng cách cung cấp một cách chuẩn hóa để truy cập dữ liệu hoặc chức năng mà các hệ thống này cung cấp.

Ví dụ: API được áp dụng trong bài toán này để lấy các thông tin về xe, Lidar và Speed. Ngoài ra còn thực hiện các sự kiện giao tiếp giữa web và server. Sau đó, API sẽ trả về dữ liệu ở định dạng được chỉ định, chẳng hạn như JSON hoặc XML, sau đó ứng dụng có thể hiển thị cho người dùng.

API được sử dụng trong nhiều ngữ cảnh, bao gồm phát triển web, phát triển ứng dụng dành cho thiết bị di động và tích hợp hệ thống. Chúng cho phép các nhà phát triển xây dựng các ứng dụng và dịch vụ mới trên các hệ thống hiện có, giúp truy cập và tích hợp dữ liệu và chức năng từ nhiều nguồn dễ dàng hơn. Bằng cách cung cấp một cách tiêu

chuẩn để truy cập dữ liệu và chức năng, API giúp hợp lý hóa các quy trình phát triển và giảm số lượng mã tùy chỉnh cần phải viết.

### **2.4.3. FLASK**

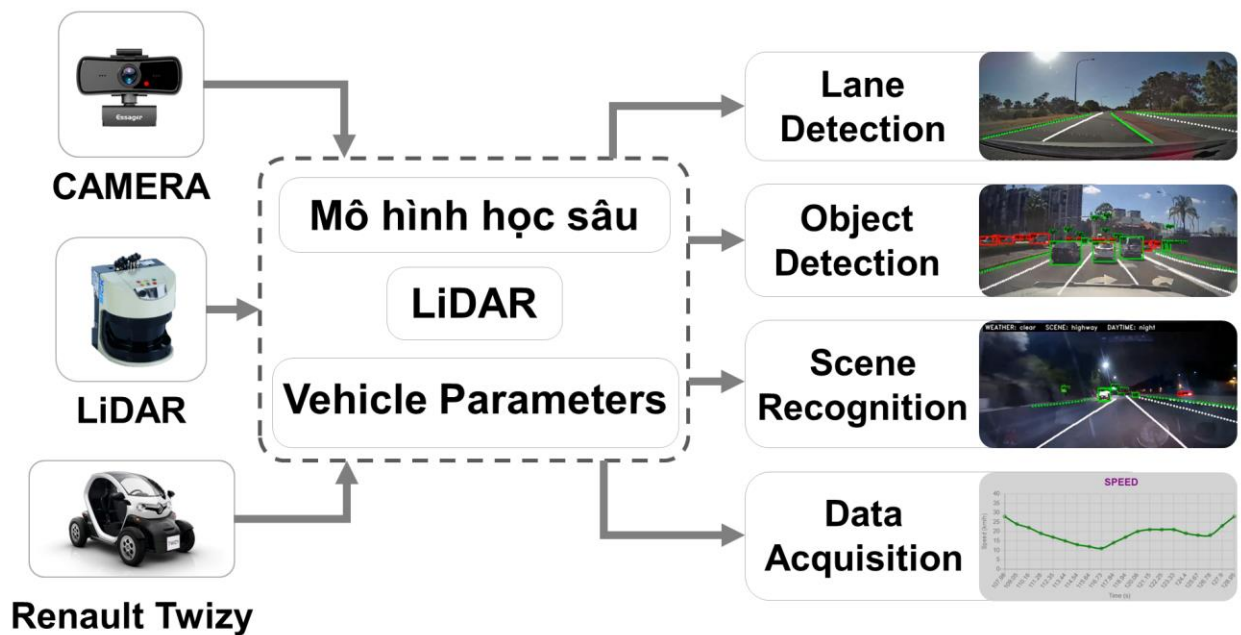
Flask là một khung web được viết bằng Python. Nó được thiết kế nhẹ và đơn giản, cung cấp chức năng cơ bản để xây dựng các ứng dụng web. Flask cho phép các nhà phát triển dễ dàng xác định các tuyến(route) hoặc URL mà ứng dụng sẽ phản hồi, cũng như cách xử lý các yêu cầu gửi đến và trả về các phản hồi.

Ngoài chức năng tích hợp sẵn, Flask có khả năng mở rộng và mô-đun cao, cho phép các nhà phát triển thêm chức năng bổ sung thông qua việc sử dụng các tiện ích mở rộng và thư viện của bên thứ ba. Điều này giúp dễ dàng thêm các tính năng như xác thực người dùng, tích hợp cơ sở dữ liệu, v.v.

Flask thường được sử dụng để xây dựng các ứng dụng web vừa và nhỏ, chẳng hạn như blog cá nhân hoặc trang web doanh nghiệp nhỏ. Nó cũng là một lựa chọn phổ biến để xây dựng nguyên mẫu hoặc bằng chứng về khái niệm, do tính đơn giản và dễ sử dụng của nó.

## CHƯƠNG 3. THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG

Trong chương này, chúng tôi sẽ trình bày thiết kế chi tiết của hệ thống đáp ứng những yêu cầu đặt ra. Hệ thống phải giải quyết được các nhiệm vụ một cách đồng thời với độ chính xác cao, ngoài ra hệ thống phải có khả năng chạy thời gian thực trên các hệ thống nhúng. Cụ thể cách tiếp cận bài toán của tôi được mô tả ở hình 3.1.



Hình 3.1. Pipeline hệ thống đa tác vụ xe tự hành

### 3.1. Thiết kế mô hình học sâu

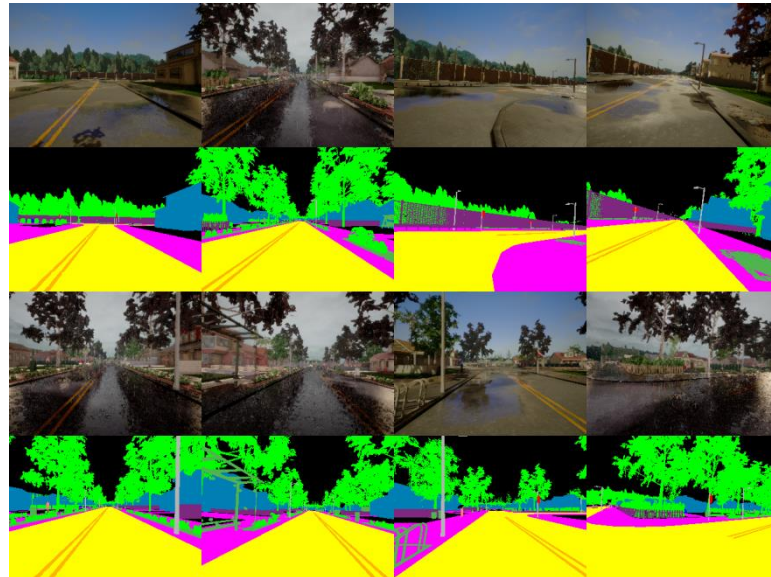
Trong phần thiết kế mô hình học sâu, để tăng độ tin cậy cho hệ thống cũng như có thể ứng dụng được trong thực tế, tôi chia bài toán thành 3 phần: mô phỏng, mô hình, thực tế (Renault Twizy) như hình 3.2 để có thể đánh giá một cách trực quan và chính xác nhất.



Hình 3.2. Pipeline bài toán xe tự hành

### 3.1.1. Tập dữ liệu

Chúng tôi sử dụng bộ dữ liệu của Carla [36]. và bộ dữ liệu của Unity để huấn luyện và đánh giá mô hình. Bộ dữ liệu của Carla bao gồm 15.000 ảnh với đa dạng về môi trường, thời tiết và vật thể (hình 3.3). Nó bao gồm 13 phân lớp với các đối tượng khác nhau trong môi trường thực nhưng chúng tôi chỉ quan tâm đến các phân lớp là đường, nền, và vật thể. Tuy nhiên bộ dữ liệu của Carla với số lượng dữ liệu ít và không cân bằng giữa các phân lớp điều này có thể làm cho mạng bị overfit hoặc bias. Vì lý do này, chúng tôi đã thu thập thêm dữ liệu bằng cách sử dụng mô phỏng Unity. Tập Unity cũng có 10.000 ảnh bao gồm các phân lớp biển báo, nền và phương tiện giao thông. Ngoài ra, để làm cho mô hình của chúng tôi thích nghi được trong các môi trường khác nhau, chúng tôi sử dụng tăng cường dữ liệu về biến dạng hình học và trắc quang trong quá trình đào tạo của mình. Đối với biến dạng trắc quang, chúng tôi điều chỉnh màu sắc, độ bão hòa và giá trị hình ảnh, độ sáng, độ tương phản và độ bão hòa. Đối với biến dạng hình học, chúng tôi sử dụng xoay ngẫu nhiên, chia tỉ lệ, cắt, kéo dài, tịnh tiến. Sau khi sử dụng tăng cường dữ liệu, chúng tôi có thêm 3.000 ảnh sau khi tăng cường. Do đó chúng tôi có tổng cộng 28.000 ảnh và chúng tôi chia chúng thành 2 phần 22.400 để huấn luyện và 5.600 để đánh giá.



Hình 3.3. Bộ dữ liệu Carla [36]

### 3.1.2. Ứng dụng trên phần mềm mô phỏng UNITY

Trong phần này tôi sẽ đánh giá trên môi trường:

- Phần cứng: thực nghiệm được chạy thử trên GeForce RTX 3090 and Intel core i9-10900K CPU 3.7GHz x20
- Phần mềm: Python 3.7, Torch 1.10.1, CUDA 11.3

### 3.1.2.1. Cài đặt môi trường

#### 3.1.2.1.1. Cài đặt CUDA

Đầu tiên, muốn tải được CUDA về máy thì phải xác định được card màn hình GPU đang sử dụng là bao nhiêu. Với phần cứng sử dụng là card màn hình GeForce RTX 3090.

Sau khi đã xác định được card màn hình GPU là bao nhiêu thì ta vào trang [https://en.wikipedia.org/wiki/CUDA#GPUs\\_supported](https://en.wikipedia.org/wiki/CUDA#GPUs_supported) để kiểm tra tham số (hình 3.4) Compute capability để xác định version của CUDA chính xác nhất.

|     |        |                                   |                                                                                                                                                                                             |                                                                                                                                                |                   |
|-----|--------|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|
| 8.6 | Ampere | GA102, GA103, GA104, GA106, GA107 | GeForce RTX 3090 Ti, RTX 3090, RTX 3080 Ti, RTX 3080 12GB, RTX 3080, RTX 3070 Ti, RTX 3070, RTX 3060 Ti, RTX 3060, RTX 3050, RTX 3050 Ti(mobile), RTX 3050(mobile), RTX 2050(mobile), MX570 | RTX A6000, RTX A5500, RTX A5000, RTX A4500, RTX A4000, RTX A2000<br>RTX A5000(mobile), RTX A4000(mobile), RTX A3000(mobile), RTX A2000(mobile) | A40, A16, A10, A2 |
|-----|--------|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|

Hình 3.4. Tham số phù hợp với GeForce RTX 3090

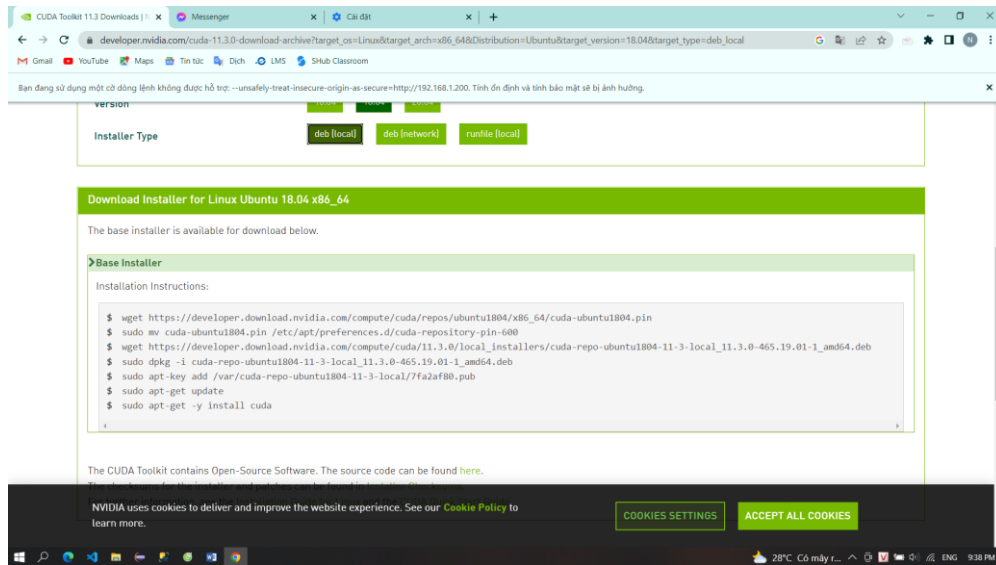
Theo hình 3.5, cột đầu tiên chính là tham số Compute capability chúng ta cần biết, đối với GeForce RTX 3090 thì tham số compute capability là 8.6. Sau khi đã xác định được compute capability của card màn hình, ta sẽ tìm được version thích hợp của CUDA, hình cho thấy đối với compute capability thì version CUDA thích hợp là CUDA SDK 11.1 – 11.7.

| <ul style="list-style-type: none"> <li>• CUDA SDK 10.0 – 10.2 support for compute capability 3.0 – 7.5 (Kepler, Maxwell, Pascal, Volta, Turing). Last version with support for compute capability 3.x (Kepler). 10.2 is the last official release for macOS, as support will not be available for macOS in newer releases.</li> <li>• CUDA SDK 11.0 support for compute capability 3.5 – 8.0 (Kepler (in part), Maxwell, Pascal, Volta, Turing, Ampere (in part)).<sup>[37]</sup></li> <li>• CUDA SDK 11.1 – 11.7 support for compute capability 3.5 – 8.6 (Kepler (in part), Maxwell, Pascal, Volta, Turing, Ampere).<sup>[38]</sup></li> </ul> |                    |      |         |             |                  |                      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|------|---------|-------------|------------------|----------------------|
| Compute capability (version)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Micro-architecture | GPUs | GeForce | Quadro, NVS | Tesla/Datacenter | Tegra, Jetson, DRIVE |

Hình 3.5. Xác định phiên bản CUDA

Ở đây, chúng tôi cài CUDA SDK version 11.3 cho máy Ubuntu 18.04. Để cài đặt CUDA, ta truy cập vào trang web của Nvidia để tải: <https://developer.nvidia.com/cuda-11.3.0-download-archive> và chọn hệ điều hành đang sử dụng, ở đây em chọn Linux x86\_64, hệ điều hành Ubuntu phiên bản

18.04. Đến đây, chọn cách thức để tải, chọn deb(local) để tải. Sau khi đã chọn, ta sẽ thấy các câu lệnh như hình 3.6 để chạy để cài đặt CUDA.



Hình 3.6. Các câu lệnh cần thiết để tải CUDA.

### 3.1.2.1.2. Cài đặt cuDNN

Đầu tiên ta cần phải cài đặt cuDNN phù hợp version của CUDA như hình 3.7, tiếp theo truy cập vào trang của nvidia để tải cuDNN phù hợp với CUDA :<https://developer.nvidia.com/rdp/cudnn-archive> (Lưu ý: Bạn có thể sẽ phải đăng nhập với tư cách Developer mới có thể tải được cuDNN). Sau đó, phải tìm kiếm phiên bản của cuDNN phù hợp với phiên bản của CUDA đã tải trước đó. Ở trước đó, đã tải phiên bản CUDA 11.3 cho nên cần phải tìm các cuDNN phù hợp với CUDA 11.3 hoặc 11.x.



Hình 3.7. Chọn phiên bản của cuDNN

### 3.1.2.2. Cách tạo phần mềm mô phỏng Unity

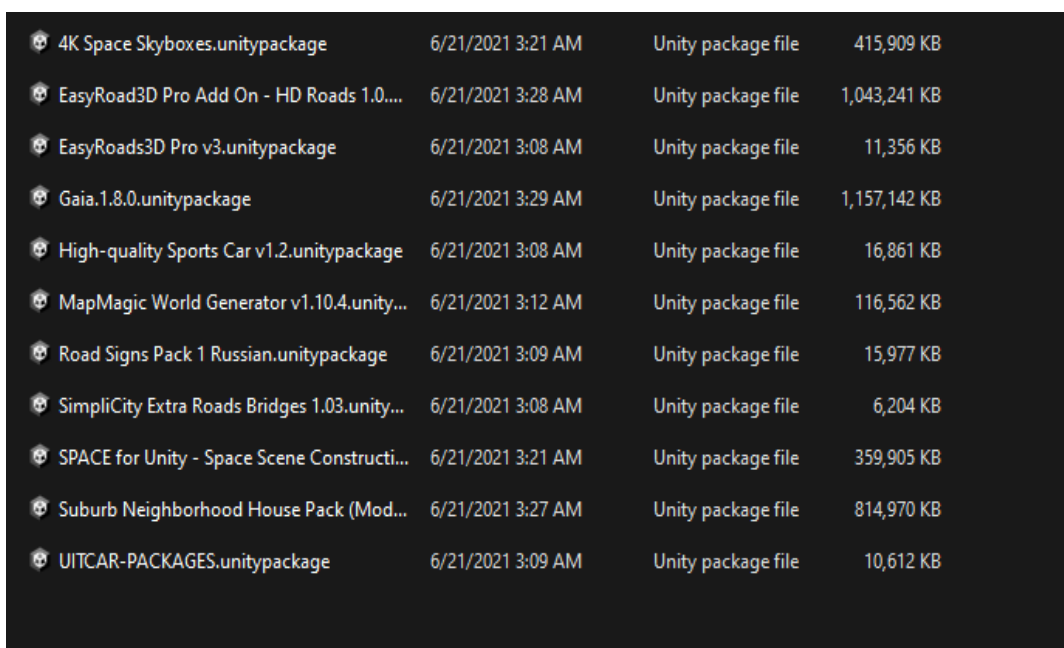
**B1:** Mở phần mềm Unity Hub và tạo project mới bằng các click New Project

**B2:** Tải thêm module Window và Linux để build map sau khi vẽ xong.

**B3:** Có thể sử dụng Unity Asset Store để tải xuống và import các assets cần thiết như địa hình, cây cối, lane đường, biển báo, xe.

Một số package (hình 3.8) cần thiết để tạo map, trong đó các package quan trọng nhất để tạo được map xe tự hành là:

- EasyRoad3D: chứa lane, ngã 3, ngã 4, địa hình cơ bản cho map.
- Road Signs Pack 1 Russian: chứa biển báo bao gồm 5 loại biển báo sử dụng chính (rẽ trái, rẽ phải, cấm trái, cấm phải, cấm thẳng)
- UITCAR-PACKAGES [37]: thư viện được phát triển bởi nhóm thiết kế map của trường ĐH UIT, chứa xe, checkpoint, hiển thị màn hình, code giao tiếp giữa map vs client.



|                                              |                   |                    |              |
|----------------------------------------------|-------------------|--------------------|--------------|
| 4K Space Skyboxes.unitypackage               | 6/21/2021 3:21 AM | Unity package file | 415,909 KB   |
| EasyRoad3D Pro Add On - HD Roads 1.0....     | 6/21/2021 3:28 AM | Unity package file | 1,043,241 KB |
| EasyRoads3D Pro v3.unitypackage              | 6/21/2021 3:08 AM | Unity package file | 11,356 KB    |
| Gaia.1.8.0.unitypackage                      | 6/21/2021 3:29 AM | Unity package file | 1,157,142 KB |
| High-quality Sports Car v1.2.unitypackage    | 6/21/2021 3:08 AM | Unity package file | 16,861 KB    |
| MapMagic World Generator v1.10.4.unity...    | 6/21/2021 3:12 AM | Unity package file | 116,562 KB   |
| Road Signs Pack 1 Russian.unitypackage       | 6/21/2021 3:09 AM | Unity package file | 15,977 KB    |
| SimpliCity Extra Roads Bridges 1.03.unity... | 6/21/2021 3:08 AM | Unity package file | 6,204 KB     |
| SPACE for Unity - Space Scene Constructi...  | 6/21/2021 3:21 AM | Unity package file | 359,905 KB   |
| Suburb Neighborhood House Pack (Mod...       | 6/21/2021 3:27 AM | Unity package file | 814,970 KB   |
| UITCAR-PACKAGES.unitypackage                 | 6/21/2021 3:09 AM | Unity package file | 10,612 KB    |

Hình 3.8. Một số package để tạo map mô phỏng

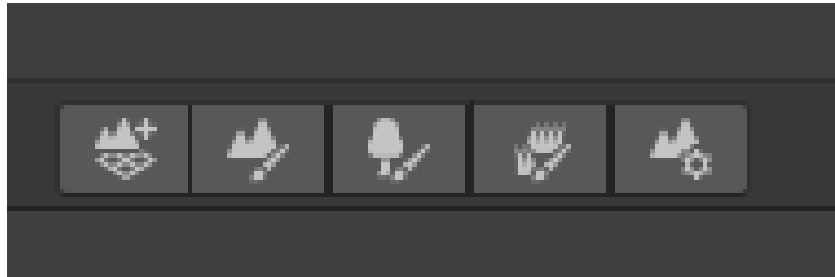
#### 3.1.2.2.1. Tạo và vẽ địa hình

Tạo địa hình: Vào Assets menu, Import Package Terrain Assets, sau khi Assets được import, chọn menu GameObject và chọn 3D object rồi chọn Terrain.

Vẽ địa hình: Sử dụng các công cụ địa hình để vẽ địa hình với các kết cấu khác nhau, nâng cao và hạ thấp địa hình, đồng thời thêm cây cối và thảm thực vật khác.

Các công cụ địa hình như hình 3.9 theo thứ tự từ trái qua phải:

- Thêm Terrain.
- Paint Terrain (nâng cao, hạ thấp địa hình)
- Paint Tree (thêm cây cối)
- Paint Details (thêm thảm thực vật)
- Terrain Settings (điều chỉnh các thông số của địa hình)



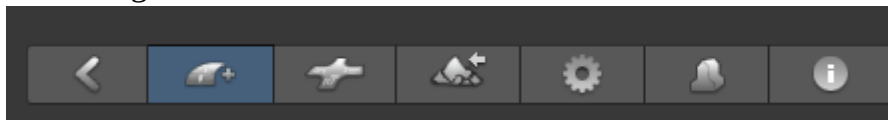
Hình 3.9. Các công cụ vẽ địa hình

#### 3.1.2.2.2. Tạo đường đi

Vào menu 3D Obejct -> EasyRoads3D -> New Road Network để tạo hệ thống lane mới.

Các công cụ chỉnh sửa làn đường như hình 3.10 theo thứ tự từ trái qua phải:

- Thêm lane mới
- Thêm ngã 3, ngã 4
- Cập nhật địa hình theo lane
- Chỉnh sửa các thông số của lane
- Thêm object viền cho lane
- Thông tin của assets

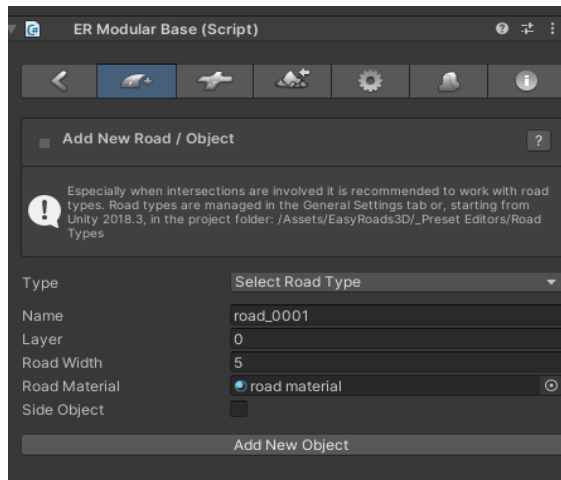


Hình 3.10. Các công cụ chỉnh sửa làn đường

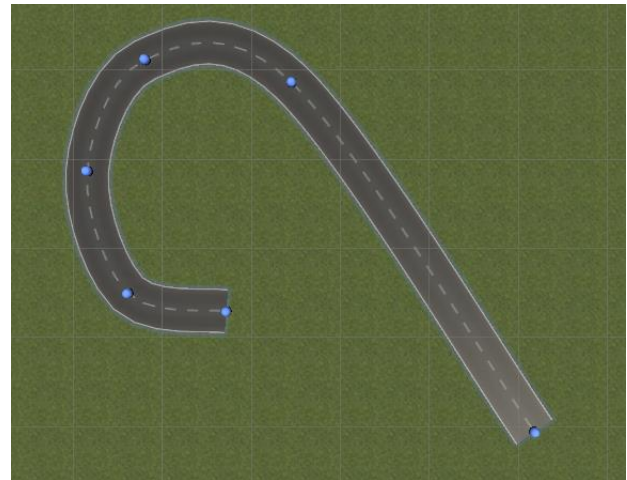
#### Các bước xây dựng đường đi:

**B1:** Điều chỉnh các thông số của road (hình 3.11a), sau đó click Add New Object để tạo Road mới. Sử dụng Shift + Click để tạo các điểm neo của Road (Các điểm chấm màu xanh là các điểm neo như hình 3.11b). Ngoài ra, có thể sử dụng

công cụ Move Tool để di chuyển qua lại các điểm neo để thay đổi hình dạng Road, di chuyển lên xuống để thay đổi độ cao của Road tạo dốc.



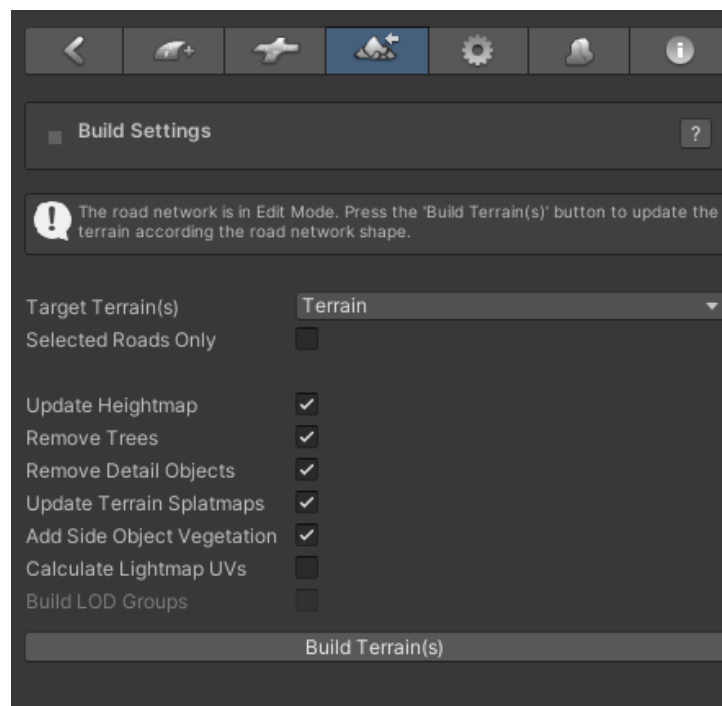
a) Các thông số của lane



b) Các điểm neo (màu xanh)

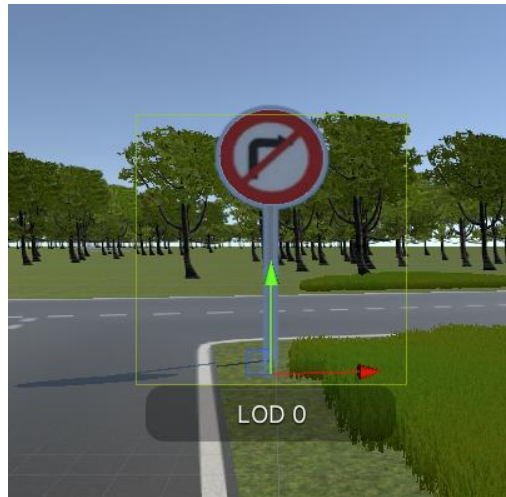
Hình 3.11. Thêm Lane mới

**B2:** Sau khi hoàn thành sang menu Build Terrain (hình 3.12) để cập nhật lại địa hình theo lane.



Hình 3.12. Cập nhật địa hình

**B3:** Thêm biển báo bằng cách kéo thả file perlab nằm trong thư viện biển báo vào map tại các vị trí muốn đặt biển báo, thay đổi kích thước sao cho phù hợp với map như hình 3.13.



Hình 3.13. Điều chỉnh biển báo

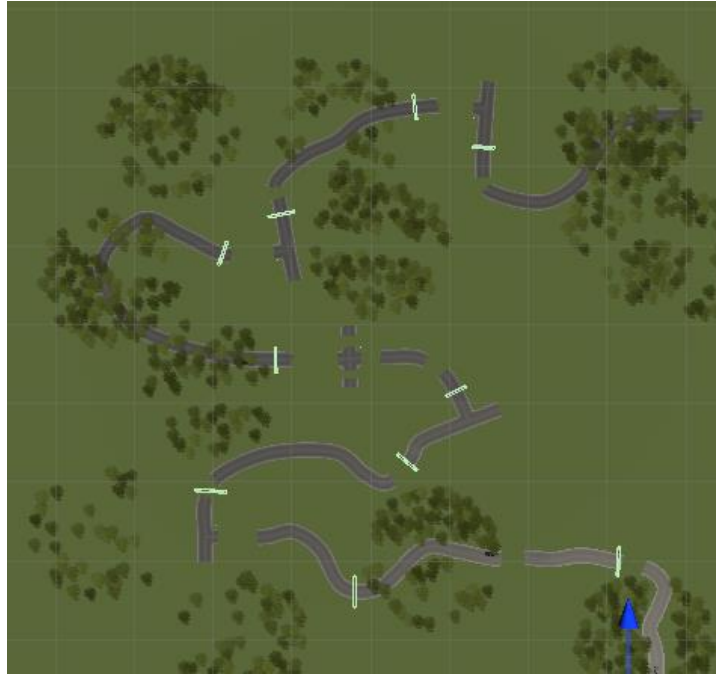
**B4:** Kiểm tra lại đường đi sau khi hoàn thành sẽ thu được map nhìn từ trên xuống như hình 3.14.



Hình 3.14. Map đường đi

#### 3.1.2.2.3. Tạo vật cản và checkpoints

Import package UITCAR, kéo thả file perlab xe vào map, sau đó sử dụng công cụ Move Tool, di chuyển các khối checkpoint đến vị trí muốn đặt checkpoint. Sau đó thêm hàng rào 2 bên Road, add Script Check out cho hàng rào này để làm checkout như hình 3.15.



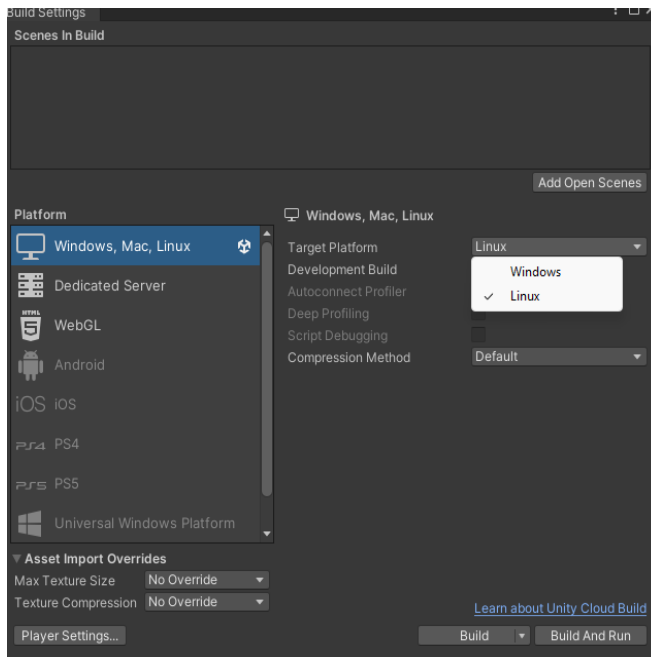
Hình 3.15. Tạo vật cản và Checkpoints

#### 3.1.2.2.4. Kiểm tra và xây dựng map

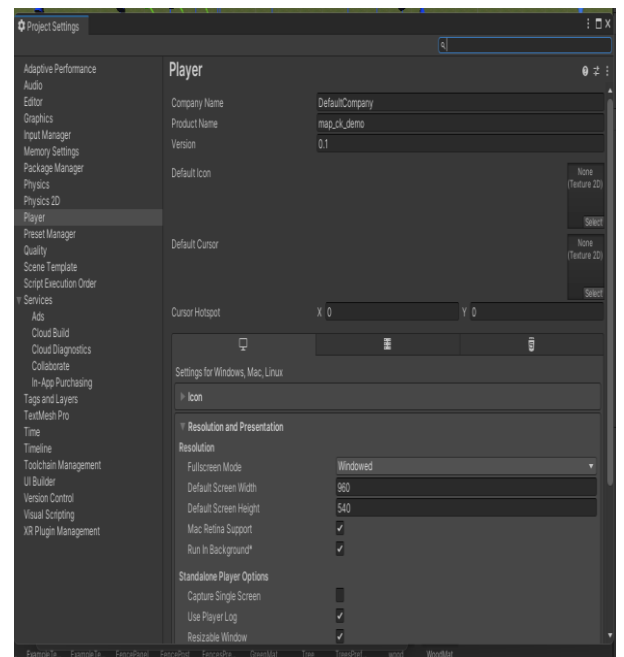
**Play and test:** nhấn nút Play để test map và có các điều chỉnh cần thiết.

**Build Map:** Vào menu File -> Build Setting

- Ở đây sẽ có 2 target platform (hình 3.16a) hỗ trợ build map là Windows và Linux sử dụng cho các hệ điều hành tương ứng.
- Có thể vào mục Player Settings (hình 3.16b) để điều chỉnh thêm các thông số liên quan tới độ phân giải, âm thanh, phím vật lí, ...
- Click Build để tiến hành tạo file map.



a) Target platform

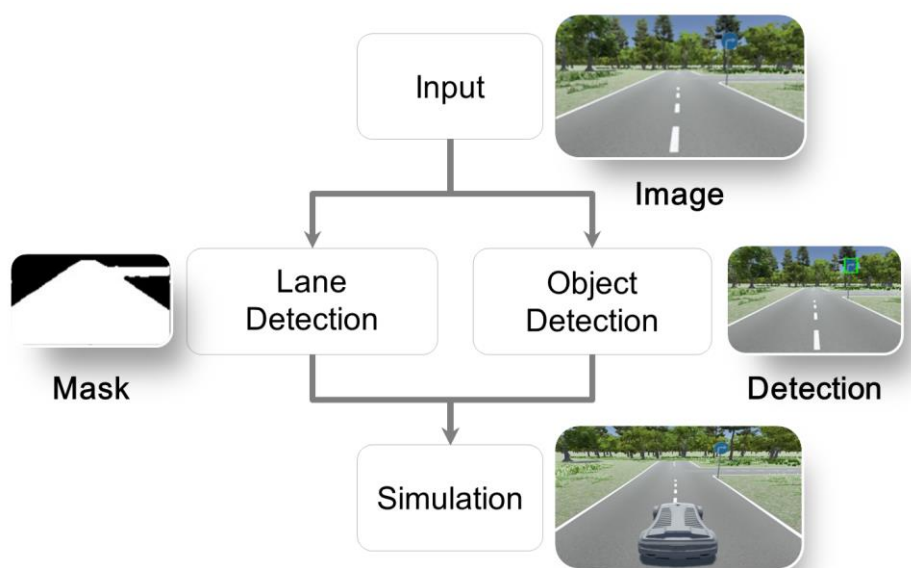


b) Player Settings

Hình 3.16. Nền tảng xây dựng map

### 3.1.2.3. Mô hình học sâu sử dụng

Trong bài toán xe không người lái, tôi đã chia nó thành hai bài toán con như hình 3.17 Lane Detection và Object Detection. Dựa trên hình ảnh, góc lái, tốc độ từ phần mềm mô phỏng, Lane Detection phát hiện và xác định vị trí của làn đường. Object Detection phát hiện và xác định vị trí của các vật cản và biển báo giao thông trên đường.



Hình 3.17. Phương pháp giải quyết bài toán xe tự hành trên phần mềm mô phỏng

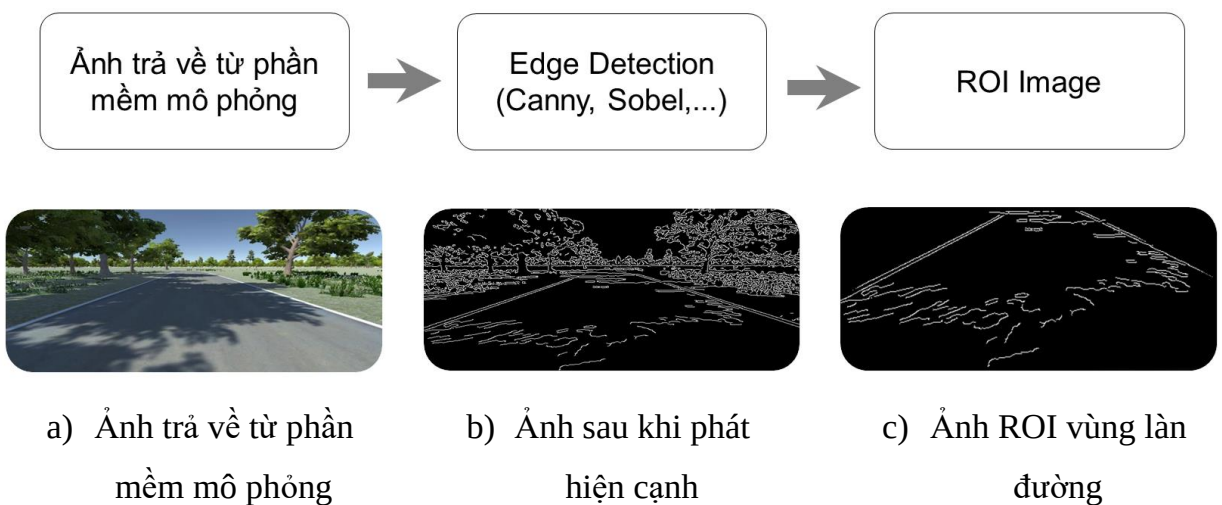
### 3.1.2.3.1. Lane Detection

#### 3.1.2.3.1.1. Phân tích bài toán

##### - Xử lý ảnh truyền thống:

Bằng việc nhận hình ảnh từ phần mềm mô phỏng trả về (hình 3.18a), để có thể phát hiện được làn đường, đầu tiên tôi sẽ phát hiện đường biên (hình 3.18b) và sau đó ROI vùng chỉ có làn đường (hình 3.18c) để xử lý (quy trình được mô tả như hình 3.18). Tuy nhiên, các phương pháp hiện đường biên trước đây thường được sử dụng bao gồm các phương pháp đạo hàm cạnh sử dụng các toán tử vi phân (các toán tử Prewitt, Canny, LOG, Sobel và Robert) có nhiều vấn đề bao gồm phạm vi phát hiện quá rộng và hiệu suất kém khi đối mặt với các điều kiện khác nhau hoặc các sự cố như bụi bẩn, ánh sáng và độ đặc biệt (VD: như bóng cây (hình c), đường có vũng nước,...). Để vượt qua những thách thức này, hiện nay đã có các mô hình học sâu sử dụng các mạng neuron để học các đặc trưng và quy tắc tự động của dữ liệu hình ảnh, cho phép nó để suy luận tự động trong các điều kiện khác nhau hoặc kể cả trong tình huống vạch kẻ đường bị mờ hoặc không có, làn đường bị nhiều bóng cây, đường bị ướt,...

Vì thế chúng tôi đề xuất bài toán Sematic Segmentation. Ưu điểm của nó là tốn ít chi phí tính toán, dễ thực hiện và không tốn thời gian huấn luyện mô hình nên cực kì thích hợp cho các bước hậu xử lý.



Hình 3.18. Phương pháp xử lý ảnh truyền thống

**- Semantic Segmentation:**

Semantic segmentation là một bài toán trong đó mục tiêu là phân chia một hình ảnh thành các khu vực có nghĩa và gán mỗi khu vực với một nhãn (label) xác định. Trong trường hợp xe không người lái, bài toán semantic segmentation có thể được sử dụng để phân chia các đối tượng trong hình ảnh như làn đường, bối cảnh xung quanh và gán mỗi đối tượng với một nhãn tương ứng từ đó giúp cho xe không người lái có thể tìm đường đi và thực hiện các hành động tương ứng. Tuy nhiên, nhược điểm lớn nhất của nó là mất rất nhiều chi phí tính toán nhưng với sự phát triển mạnh mẽ của máy tính thì việc này không còn là vấn đề.

Để huấn luyện một mô hình học sâu (hình 3.19), cần:

1. Tập dữ liệu: thu thập và sắp xếp dữ liệu đầu vào cho mô hình học sâu. Điều này có thể bao gồm việc tải dữ liệu từ các nguồn công cộng hoặc tạo ra dữ liệu từ đầu.
2. Đánh nhãn dữ liệu: Nếu tập dữ liệu chưa được đánh nhãn thì cần đánh nhãn theo yêu cầu bài toán.
3. Khảo sát mô hình: cần chọn một mô hình học sâu phù hợp với bài toán, phần cứng. Ví dụ, nếu bạn muốn phân đoạn hình ảnh (Unet hoặc Bisenet hoặc Enet) yêu cầu độ chính xác thì có thể chọn một mô hình Unet.
4. Huấn luyện mô hình: Sử dụng dữ liệu đầu vào và mô hình đã chọn, huấn luyện mô hình bằng cách sử dụng các thuật toán huấn luyện như stochastic gradient descent (SGD).
5. Đánh giá mô hình: Sau khi huấn luyện, cần đánh giá hiệu suất của mô hình bằng cách sử dụng tập dữ liệu kiểm tra.



Hình 3.19. Các bước huấn luyện một mô hình học sâu

### 3.1.2.3.1.2. Khảo sát

Trong bài toán phân đoạn hình ảnh xe không người lái, tôi đã huấn luyện và so sánh hiệu suất của ba mô hình khác nhau: Unet [27], ENet [28] và Bisenet [31]. Tôi sử dụng tập dữ liệu gồm khoảng 25k ảnh (15k ảnh từ bộ Carla và 10k ảnh lấy từ các map mô phỏng) với kích thước ảnh đầu vào là 80x160.

Unet [27] là một mô hình được tạo ra với mục đích để phân đoạn ảnh y tế, tuy nhiên nó lại cho kết quả khá tốt trong nhiều task khác, ENet [28] và Bisenet [31] là hai mạng phân đoạn tập trung vào tốc độ. Như bảng 3.1 cho thấy, kết quả mô hình UNet có độ chính xác cao hơn so với kiến trúc ENet [28] và BiseNet [31]. Cụ thể, IoU của Unet [27] cao hơn 2.7% so với Bisenet [31] và 4.6% so với ENet [28]. So sánh trực quan trong hình 3.20 cũng cho thấy rằng mô hình Unet [27] có hiệu suất tốt nhất trong việc phân đoạn các phần rìa bên cạnh làn đường và bóng cây, và tốc độ chạy 49.63 FPS cũng không phải là vấn đề trong việc chạy thời gian thực. Vì vậy, tôi chọn sử dụng mô hình Unet [27] cho bài toán phân đoạn hình ảnh này (hình 3.21).

Bảng 3.1. So sánh các mạng hiện đại về nhiệm vụ phân đoạn.

| Mô hình          | IoU(%)      | FPS (RTX 3090) |
|------------------|-------------|----------------|
| <b>UNet [24]</b> | <b>94.8</b> | <b>49.63</b>   |
| Bisenet [32]     | 92.1        | 105.80         |
| ENet [25]        | 90.2        | 98.26          |



a) Ảnh gốc

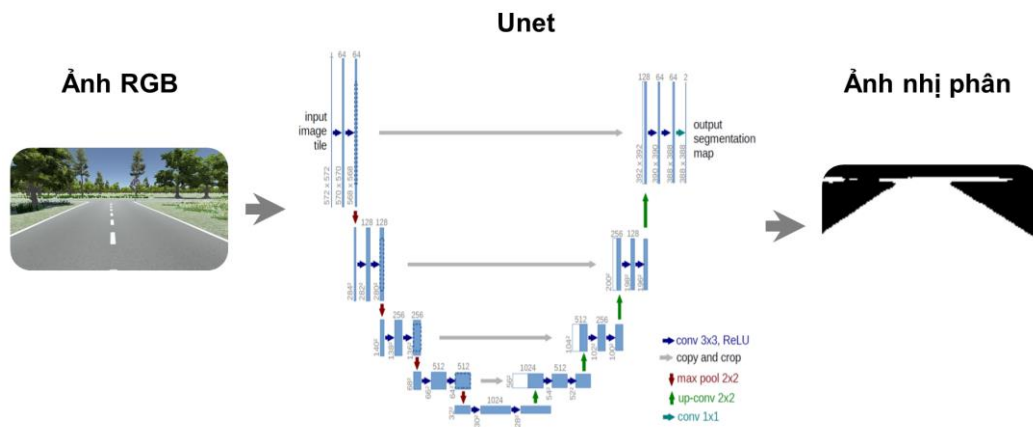


b) Kết quả Bisenet



c) Kết quả Unet

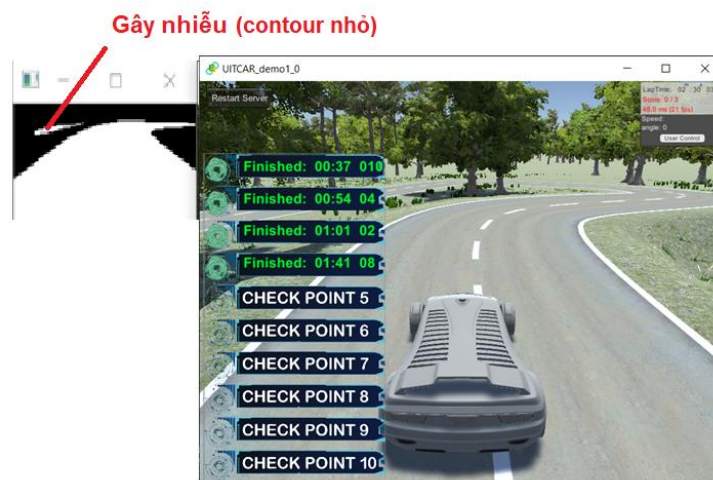
Hình 3.20. So sánh kết quả giữa Bisenet [31] và Unet [27]



Hình 3.21. Sử dụng mô hình Unet cho bài toán phân đoạn làn đường

### 3.1.2.3.1.3. Vấn đề gặp phải

Do Unet rất chính xác nên mạng bắt được những đoạn đường rất xa gây nhiễu cho việc điều khiển (hình 3.22) => Tôi xử lý bằng cách loại bỏ những vùng contours có diện tích nhỏ thì nó sẽ xóa đi các chỗ nhiễu và chỉ giữ lại làn đường.



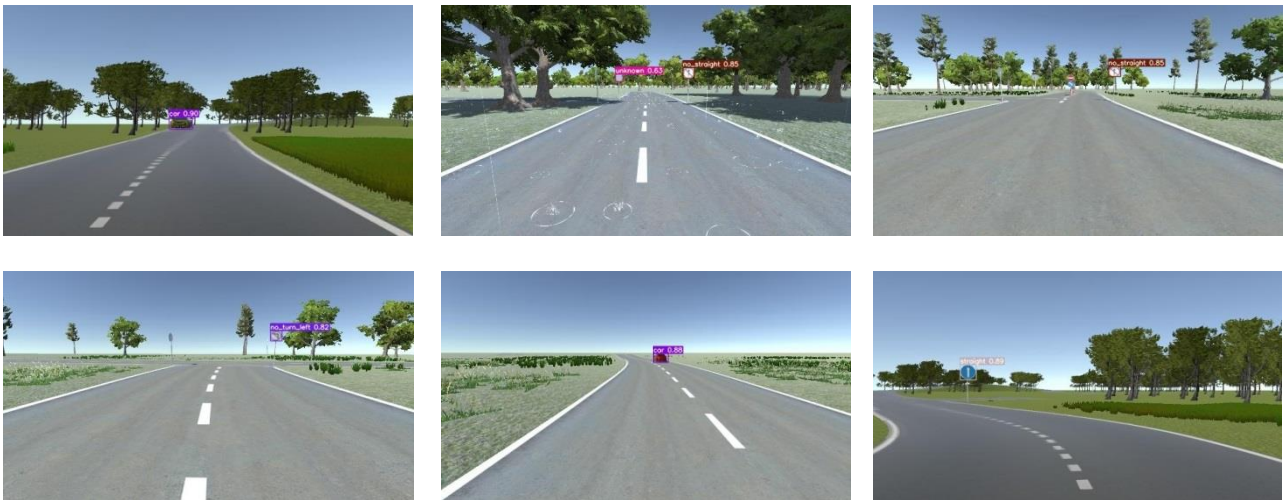
Hình 3.22. Bắt được những đoạn đường rất xa gây nhiễu cho việc điều khiển

### 3.1.2.3.2. Object Detection

Bài toán Object detection trong xe không người lái là một bài toán trong lĩnh vực nhận dạng đối tượng đặc thù cho xe không người lái. Nhiệm vụ của bài toán này là tìm và xác định vị trí các đối tượng quan trọng trong hình ảnh hoặc video từ camera trên xe không người lái, ví dụ như xe máy, xe tải, người đi bộ, v.v. Điều này có thể giúp cho xe không người lái điều khiển và tránh các tình huống xảy ra tai nạn. Các mô hình Object detection hiện đại được sử dụng cho bài toán này bao gồm Faster R-CNN [14], YOLO [15], v.v.

#### 3.1.2.3.2.1. Phân tích bài toán

Trong bài toán xe không người lái, hai vấn đề chính mà tôi đối mặt là yêu cầu chạy thời gian thực và phát hiện đối tượng nhỏ (small objects) như hình 3.23. Để giải quyết vấn đề đó, tôi sử dụng kỹ thuật học sâu và các kỹ thuật hậu xử lý. Đặc biệt, tôi chọn sử dụng mô hình mạng học sâu như YOLO [15], SSD [16] để phát hiện vị trí và phân loại đối tượng một giai đoạn, để đáp ứng yêu cầu chạy thời gian thực. Tôi cũng sử dụng YOLO [15] để giải quyết vấn đề phát hiện các đối tượng nhỏ, vì YOLO [15] được đánh giá cao trong việc nhận dạng đối tượng nhỏ với tốc độ và độ chính xác cao. Sử dụng mô hình có thể phát hiện được các đối tượng nhỏ sẽ rất hữu ích với trường hợp xe chạy với tốc độ cao và cần có thông tin sớm để cho thể đưa ra quyết định. Kết quả kiểm chứng trên tập dữ liệu mô phỏng Unity chứng minh tính hiệu quả và độ chính xác của mô hình được đề xuất.

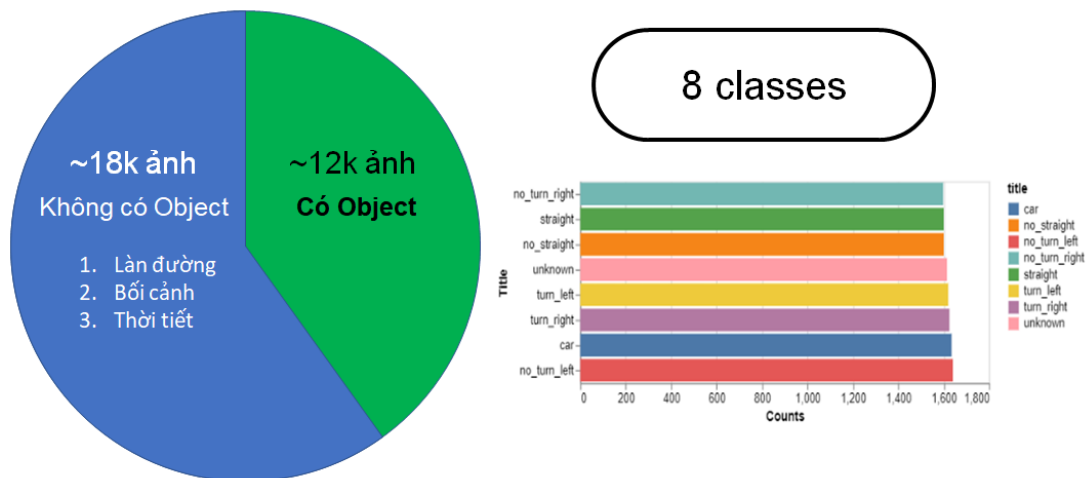




Hình 3.23. Các trường hợp đối tượng nhỏ

Về phần dữ liệu cho bài toán Object Detection chúng tôi lấy từ bộ dữ liệu đã chuẩn bị ở phần 3.1.1, sau đó chia thành 8 class riêng với tổng cộng hơn 12.000 đối tượng (hình 3.24) và được thống kê như sau:

- 1598 ảnh cấm quẹo phải
- 1600 ảnh đi thẳng
- 1600 ảnh cấm đi thẳng
- 1614 ảnh người
- 1620 ảnh quẹo trái
- 1625 ảnh quẹo phải
- 1608 ảnh xe (vật cản)
- 1640 ảnh cấm quẹo trái



Hình 3.24. Trực quan hoá dữ liệu

### 3.1.2.3.2.2. Khảo sát

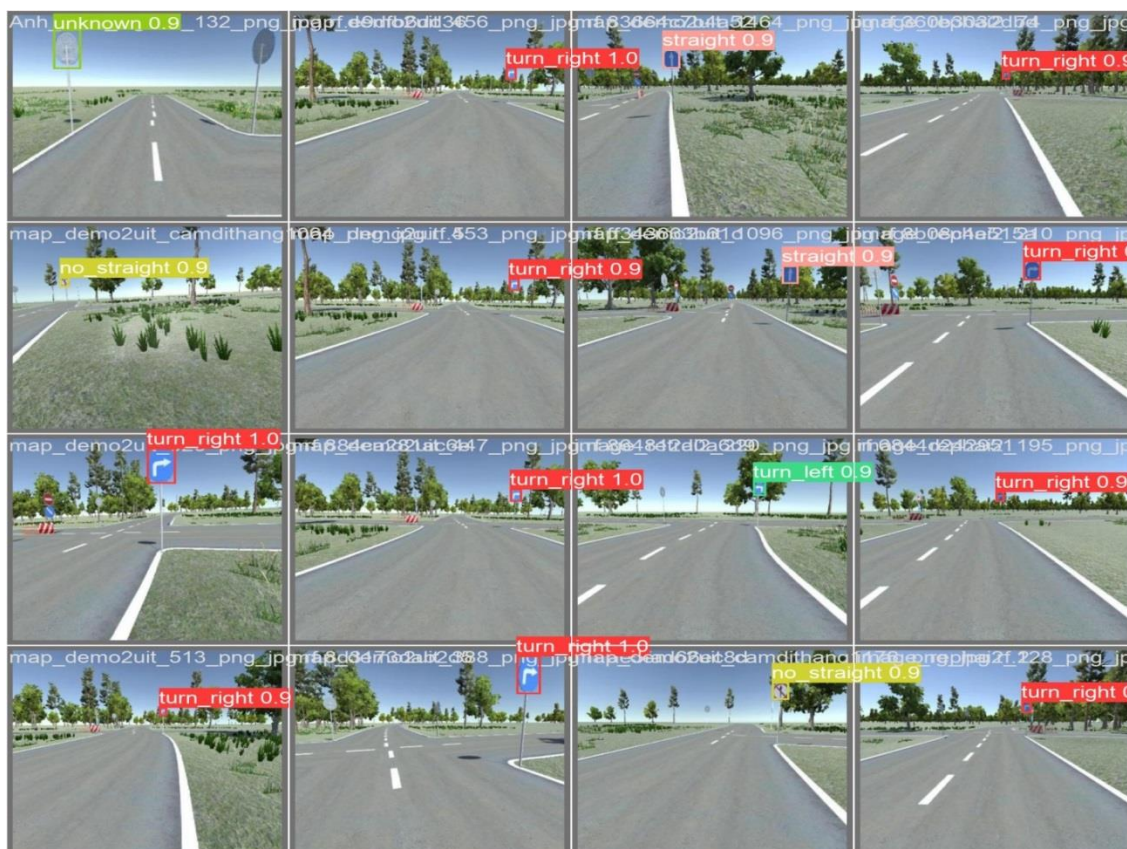
Một trong những lợi thế chính của YOLO [15] là thời gian xử lý nhanh. Nó cho phép xử lý hình ảnh ở tốc độ khung hình cao, giúp nó thích hợp với các ứng dụng như giám sát video, lái xe tự động và robot. YOLO [15] đã được cải tiến từ lần đầu tiên được phát hành, với các phiên bản YOLOv2, YOLOv3, YOLOv4, YOLOv5 [38], YOLOv6 [39] và YOLOv7 [19] được phát triển. Mỗi phiên bản đều có những ưu điểm và hạn chế riêng, với cải tiến trong độ chính xác, tốc độ và kích thước mô hình. Tuy nhiên, YOLO [15] cũng có một số hạn chế như khó khăn trong việc nhận diện đối tượng nhỏ và trong các cảnh đông đúc. Nó cũng cần một lượng lớn dữ liệu để huấn luyện và có thể không hoạt động tốt trên các bộ dữ liệu có dữ liệu hạn chế.

Điều đáng chú ý là YOLOv7 [19] được coi là phiên bản chính xác hơn YOLOv5 [38] nhưng chậm hơn một chút. Tuy nhiên, sự khác biệt về FPS giữa YOLOv5 [38] và YOLOv7 [19] là không đáng kể và RTX 3090 sẽ có thể xử lý cả hai phiên bản có FPS cao. Như bảng 3.2 so sánh giữa các phiên bản YOLO được huấn luyện trên tập dữ liệu lấy từ bộ Carla và phần mềm mô phỏng Unity, chỉ số  $\text{map}@0.5:0.95$  của YOLOv7 [19] đang cao nhất cụ thể lớn hơn YOLOv5 [38] là 0.023 và YOLOv6 [39] là 0.028 và với FPS là 243.90, điều này hoàn toàn có thể giải quyết được vấn đề chạy thời gian thực. Vì vậy, chúng tôi quyết định sử dụng YOLOv7 [19] cho đề tài này.

Bảng 3.2. So sánh các phiên bản YOLO về phát hiện và nhận diện vật thể.

| Model   | Size | Precision | Recall | mAP@0.5 | mAP@0.5:0.95 | FPS (RTX 3090) |
|---------|------|-----------|--------|---------|--------------|----------------|
| YOLOv5m | 640  | 0.995     | 0.997  | 0.993   | 0.861        | 256.41         |
| YOLOv6n | 640  | 0.994     | 0.99   | 0.996   | 0.856        | 307.69         |
| YOLOv7  | 640  | 0.995     | 0.997  | 0.993   | 0.884        | 243.90         |

Kết quả đạt được sau khi ứng dụng mạng YOLOv7 [19] trong nhận diện và phát hiện biển báo và vật cản (hình 3.25).

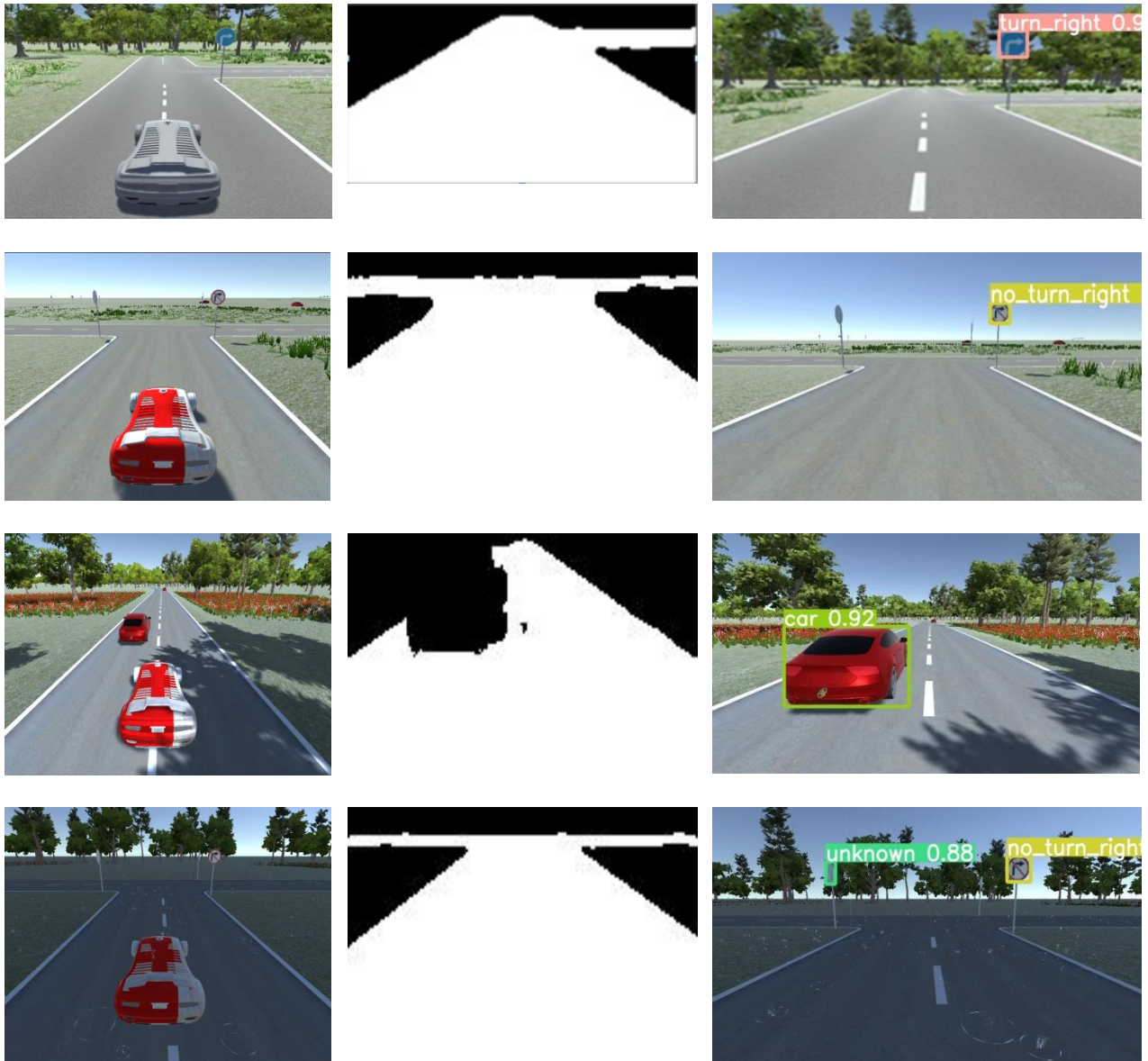


Hình 3.25. Kết quả Yolov7

#### 3.1.2.4. Đánh giá

Mô phỏng xe tự hành là một phương pháp để kiểm tra và đánh giá các hệ thống xe tự hành trước khi triển khai trên đường thực tế. Nó cho phép nhà phát triển kiểm tra các giải pháp trong môi trường điều kiện giả định và thử nghiệm các tình huống đặc biệt mà không có rủi ro gây hại cho người dùng cuối. Mô phỏng xe tự hành có thể giúp các nhà phát triển đánh giá độ chính xác của các thuật toán trong việc phát hiện và tránh các đối tượng, đánh giá tốc độ chạy của hệ thống, và đánh giá tính năng của hệ thống trong các tình huống khác nhau.

Kết quả kiểm chứng như hình 3.26 qua hai cuộc thi “**Giải nhì** UIT Car Racing 2021 - bảng chuyên nghiệp” và “**Giải nhì** cuộc thi Xe tự hành UTE 2022” chứng minh phương pháp của chúng tôi đề xuất có tính hiệu quả và độ chính xác.



a) Phần mềm mô phỏng

b) Phân đoạn làn đường

c) Phát hiện đối tượng

Hình 3.26. Kết quả mô phỏng xe tự hành

Tuy nhiên, mô phỏng xe tự hành cũng có một số hạn chế, chẳng hạn như khó khăn trong việc tạo ra môi trường mô phỏng chính xác, khó khăn trong việc mô phỏng các tình huống khó khăn và khó xử lý, và khó khăn trong việc kiểm chứng kết quả mô phỏng với thực tế. Vì thế trước khi triển khai trên môi trường thực tế tôi đã ứng dụng trên xe mô hình để có thể kiểm chứng hệ thống với một môi trường thực tế nhỏ.

### 3.1.3. Ứng dụng trên mô hình

Trong Việt Nam, công nghệ xe tự hành đã phát triển mạnh với sự dẫn đầu của các công ty lớn như Tập đoàn Phenikaa và Vinfast. Các nhà nghiên cứu trong nước cũng đang rất tích cực tham gia vào phát triển công nghệ này. Hàng năm, có rất nhiều cuộc thi như Cuộc Đua Số FPT, UIT Car Racing, Bosch Future Mobility Challenge với các xe mô hình tỷ lệ 1/7 hoặc 1/10 như hình 3.27 để kích thích tư duy sáng tạo của sinh viên và học sinh. Bằng việc ứng dụng các mô hình học sâu, thuật toán điều khiển và công nghệ định vị để điều khiển xe tự động trên sa bàn (hình 3.28). Sử dụng xe mô hình giúp giảm chi phí và tăng tốc độ trong quá trình phát triển và kiểm chứng các thuật toán, giúp cho việc phát triển và tối ưu hóa các thuật toán trở nên dễ dàng hơn.



a) FPT



b) UIT



c) Bosch

Hình 3.27. Xe mô hình



Hình 3.28. Sa bàn xe tự hành

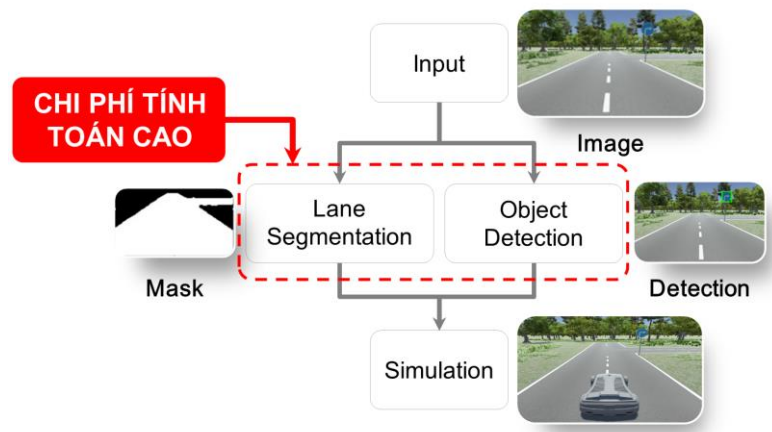
### 3.1.3.1. Những thách thức

Tuy nhiên, trong bài toán ứng dụng trên xe mô hình gặp những thách thức:

- Sử dụng máy tính nhúng (cụ thể là Jetson Nano) để triển khai các ứng dụng AI trên xe mô hình là vì máy tính nhúng có kích thước nhỏ, tiết kiệm điện năng và có khả năng tích hợp với các thiết bị cảm biến và phần cứng khác, giúp cho việc triển khai hệ thống AI trên xe mô hình trở nên dễ dàng và hiệu quả hơn. Ngoài ra, máy tính nhúng còn có thể chạy được trong môi trường có điều kiện khó khăn và có thể tích hợp vào các thiết bị di động, giúp cho việc triển khai AI trên xe mô hình trở nên linh hoạt và tiện lợi hơn. Tuy nhiên, do hạn chế của về tốc độ tính toán của Jetson Nano nên đã gây thách thức cho việc ứng dụng các mô hình học sâu (hình 3.29).
- Việc sử dụng camera nên hình ảnh trả về không đẹp như trên mô phỏng và do kết cấu của xe dẫn đến khi xe vận hành camera bị rung nên kết quả dự đoán của mô hình không được chính xác.
- Ngoài ra, trong năm 2016 một tai nạn xe không người lái Tesla Model S đầu tiên trên thế giới là do hệ thống hỗ trợ lái của xe không nhận diện được một cầu thang trên đường cao tốc.

Để giải quyết các vấn đề trên tôi thực hiện như sau:

- Thu thập thêm dữ liệu với môi trường thực tế.
- Thiết kế mô hình học sâu dựa trên các mô hình Unet và Yolov7 đã được ứng dụng trong phần mềm mô phỏng. Chúng tôi còn sử dụng các kỹ thuật nén mô hình để giảm kích thước mô hình và tăng tốc độ chạy, để cho hệ thống có thể chạy trên thiết bị nhúng như Jetson Nano.
- Ngoài ra, chúng tôi còn thêm tác vụ phân loại điều kiện lái xe dựa vào thời tiết, bối cảnh, thời gian nhằm cung cấp cho thông tin cho các thuật toán điều khiển với điều kiện bối cảnh khác nhau.

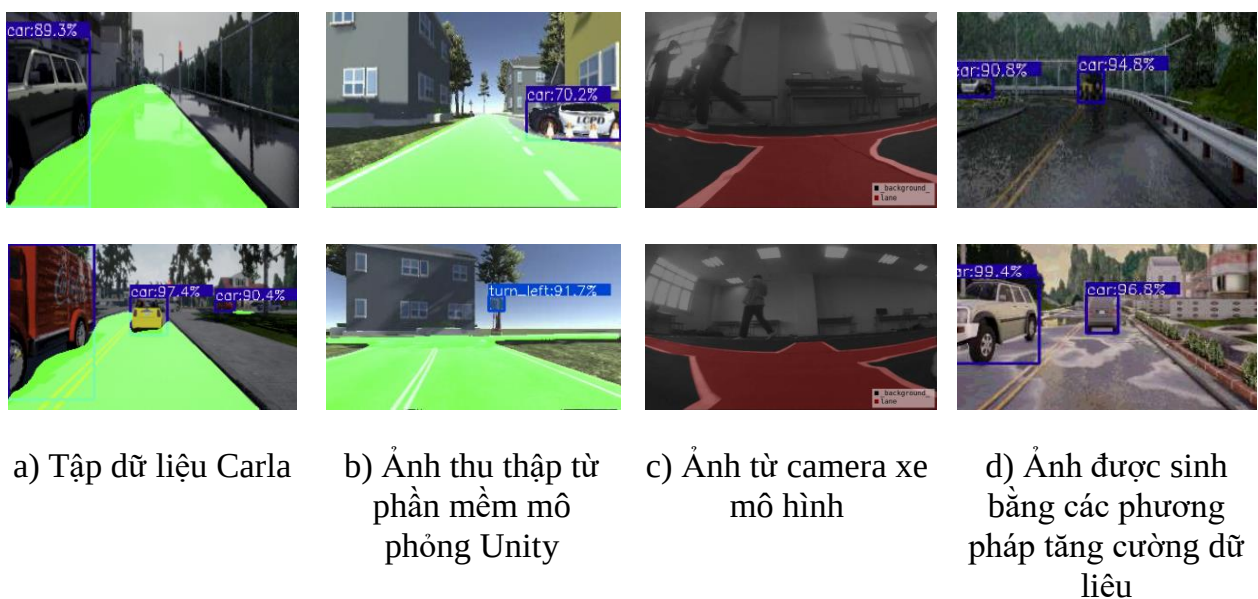


Hình 3.29. Vấn đề trong việc triển khai các mô hình học sâu lên các máy tính nhúng

### 3.1.3.2. Thu thập thêm dữ liệu

Nguyên nhân cụ thể của tai nạn xe không người lái Tesla đầu tiên mà bạn đang hỏi là do hệ thống hỗ trợ lái tự động (Autopilot) của xe không nhận diện được một cầu thang trên đường cao tốc, khiến Joshua Brown không thể tránh khỏi va chạm với cầu thang đó.

Vì vậy, chúng tôi sẽ thu thập thêm 2.000 ảnh được lấy từ camera trên xe mô hình và kết hợp với bộ dữ liệu ở phần 3.1.1 (15.000 ảnh bộ Carla, 10.000 ảnh phần mềm mô phỏng Unity và 3.000 ảnh tăng cường dữ liệu). Sau khi thu thập thêm dữ liệu, chúng tôi có 24.000 hình ảnh để đào tạo và 6.000 hình ảnh để thử nghiệm được mô tả như hình 3.30.



a) Tập dữ liệu Carla

b) Ảnh thu thập từ phần mềm mô phỏng Unity

c) Ảnh từ camera xe mô hình

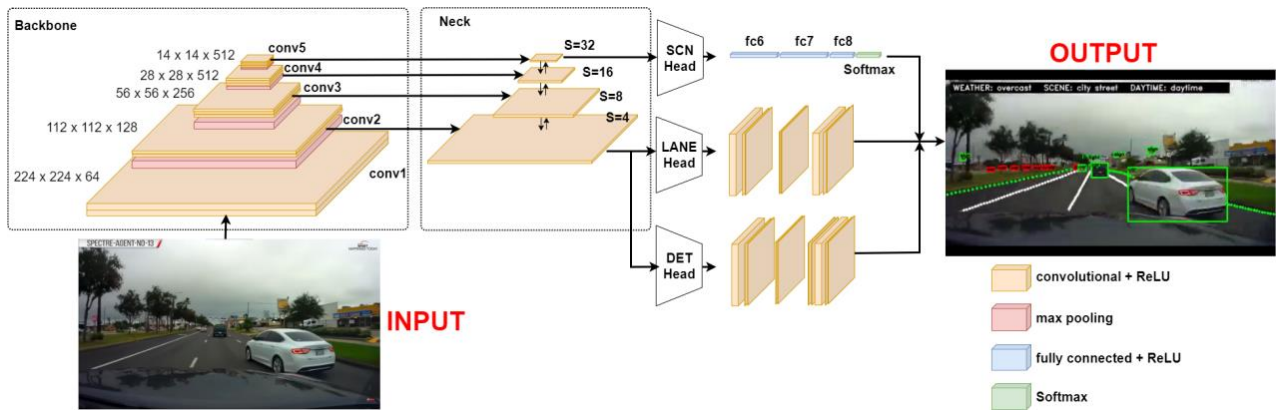
d) Ảnh được sinh bằng các phương pháp tăng cường dữ liệu

Hình 3.30. Bộ dữ liệu huấn luyện mô hình

### 3.1.3.3. Thiết kế mô hình học sâu

Các tác vụ phổ biến trong bài toán xe không người lái bao gồm phát hiện người đi đường, cũng như xác định ranh giới làn đường và phân loại điều kiện lái xe. Trong vài năm qua, rất nhiều mô hình Deep Learning mạnh mẽ đã được đề xuất để giải quyết các nhiệm vụ riêng lẻ về nhận thức ô tô dựa trên camera với hiệu suất đáng kinh ngạc. Tuy nhiên do khả năng hạn chế của các của các máy tính nhúng nên không thể tính toán để chạy một mô hình nặng cho từng tác vụ riêng lẻ.

Trong phần này, tôi đề xuất một mô hình đa tác vụ có thể đồng thời giải quyết các nhiệm vụ 1) phát hiện đối tượng, 2) phân đoạn làn đường và 3) phân loại hình ảnh cho thời tiết (nắng, mưa, mây vv), cảnh điều khiển xe (đường cao tốc, đường thành phố vv) và thời gian trong ngày (sáng, đêm vv). Như ở hình 3.31, mạng của chúng tôi bao gồm một nhánh mã hóa và ba nhánh giải mã để giải quyết các nhiệm vụ riêng. Bằng cách sử dụng một nhánh mã hóa, chúng tôi tạo một cầu nối chung để các nhiệm vụ có thể chia sẻ và bổ sung cho nhau trong quá trình đào tạo, đối với bộ giải mã (Heads), mỗi nhánh sẽ giải quyết các vấn đề khác nhau.



Hình 3.31. Kiến trúc của mạng

#### 3.1.3.3.1. Mã hóa

Bộ mã hóa bao gồm phần backbone và phần neck kết hợp lại với nhau.

##### A. Backbone

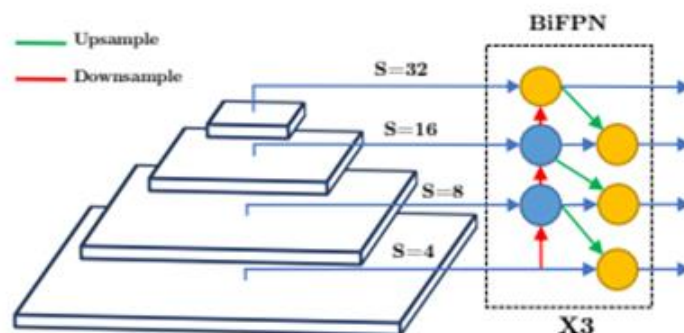
Backbone có nhiệm vụ trích xuất đặc trưng từ ảnh đầu vào. chúng tôi đã thử nghiệm với ba nhóm mô hình khác nhau: 1) Resnet [40], với kiến trúc đơn giản và dễ hiểu của nó, là lựa chọn an toàn nhất; 2) Mobilenet [41] thay vào đó là

một ví dụ được chấp nhận của mô hình tích hợp với bộ nhớ và chi phí tính toán giảm; 3) cuối cùng là EfficientNet [42] là một ví dụ mới hơn của một mô hình với chi phí thực thi thấp, tận dụng các giải pháp hiện đại.

Chúng tôi cố tình tránh sửa đổi các xương sống hoặc quá chuyên sâu về mặt tính toán để có hiệu quả trong mục tiêu triển khai dự định của chúng tôi hoặc được xây dựng với các lớp và lựa chọn kiến trúc không phổ biến [43] sẽ khiến việc chuyển đổi sang khung suy luận khó hơn và có khả năng không chính xác. Tất cả các xương sống được thử nghiệm đều được đào tạo trước trên bộ dữ liệu phân loại ImageNet [44].

## B. Neck

Neck có chức năng kết hợp các đặc trưng được lấy từ xương sống và để xử lý các đặc trưng nhận được, chúng tôi thử nghiệm với hai khối necks khác nhau. Khối neck đơn giản chỉ lấy feature map cuối cùng (stride 32) và tăng dần độ phân giải không gian đến stride 4, cần thiết cho việc phát hiện đối tượng và làn đường, với các lớp xen kẽ của 2D Convolution và Transposed convolution với stride 2 và kernel size 4 (upsampling 2 lần cho mỗi lớp).



Hình 3.32. Cấu trúc BiFPN [45]

Khối necks phức tạp hơn nhưng nhẹ hơn, dựa trên Bi-directional Feature Pyramid Network (BiFPN) giới thiệu trong [45]. Khối này là một sự tiến bộ của Feature Pyramid Layer (FPL [46]), nó cho phép hợp nhất các bản đồ đặc trưng với các tầm khác nhau sử dụng cả đường dẫn từ trên xuống (lấy mẫu lên) và đường dẫn từ dưới lên (lấy mẫu xuống) để cho phép thông tin đổi chiều, như được minh họa trong hình 3.32.

### 3.1.3.3.2. Giải mã (Heads)

Như đã nói ở trên, hệ thống chúng tôi xử lý ba nhiệm vụ: 1) phát hiện đối tượng (biển báo giao thông và vật cản), 2) phân đoạn làn đường và 3) phân loại điều kiện lái xe (thời tiết, cảnh điều khiển xe và thời gian trong ngày).

Heads theo một thiết kế tối giản nhằm mục đích giữ cho sự phức tạp của từng nhiệm vụ riêng lẻ ở mức tối thiểu. Đầu tiên cho việc phân đoạn làn đường và đầu tiên cho việc nhận diện đối tượng cùng chia sẻ cùng một đặc tính đầu vào và cùng một cấu trúc: mỗi kết quả mong muốn được đạt được bằng cách áp dụng một lớp convolutional  $3 \times 3$  đến bản đồ đặc tính, sau đó là một kích hoạt ReLU, theo sau là một lớp convolution cuối cùng  $1 \times 1$  để sản xuất số lượng kênh đầu ra mong muốn.

### 3.1.3.3.3. Hàm mất mát (Loss Function)

Multitask deep learning models thường khó để huấn luyện do không ổn định và một số nhiệm vụ chiếm ưu thế trong gradient của loss. Do đó, cẩn thận trong việc chọn trọng lượng của các nhiệm vụ và thường cần các chiến lược huấn luyện đa giai đoạn. Tuy nhiên, mô hình của chúng tôi có thể được huấn luyện end-to-end chủ yếu dựa trên hàm mục tiêu cơ bản, trừ cho hàm loss sử dụng cho việc hồi quy heatmaps. Tổng cộng, chúng tôi sử dụng ba hàm loss khác nhau được thể hiện ở công thức (3.1):

$$L_{\text{all}} = L_{\text{det}} + \alpha L_{\text{seg}} + \beta L_{\text{cls}}, \quad (3.1)$$

trong đó  $\alpha, \beta$  là hệ số để kiểm soát cân bằng giữa các nhiệm vụ.  $L_{\text{det}}$  bao gồm ba hàm mất mát: mất mát về bounding box  $L_{\text{bbox}}$ , mất mát về chất lượng  $L_{\text{qfl}}$  và mất mát về phân phối  $L_{\text{dfl}}$  như hiển thị ở công thức (3.2). Mất mát về hộp giới hạn chính là mất mát về IoU được sử dụng để giảm thiểu sự khác biệt giữa bounding box và ground truth. Mất mát về chất lượng và phân phối được thiết kế để xử lý các nhãn liên tục và rủi ro mất cân bằng trong khi đào tạo với đại diện chung phân loại IoU như đã nói ở trên. Đối với mất mát phân đoạn  $L_{\text{seg}}$  và phân loại  $L_{\text{cls}}$ ,

Cross Entropy Loss [47] được sử dụng để giảm thiểu lỗi phân loại giữa các pixel của đầu ra và nhãn.

$$L_{\text{det}} = L_{\text{bbox}} + \beta L_{\text{qfl}} + \gamma L_{\text{dfl}}, \quad (3.2)$$

Trong đó  $\beta$  và  $\gamma$  là các trọng số dùng để điều chỉnh sự cân bằng giữa các hàm mất mát. Chúng sẽ được xác định bằng thực nghiệm.

### 3.1.3.4. Đánh giá

Tất cả các thử nghiệm được báo cáo trong bảng 3.3 đều được chạy với cùng một bộ siêu tham số: cụ thể là chúng tôi đào tạo trong 75 epochs với batch size là 64 bằng cách sử dụng trình tối ưu hóa Adam [48]. Để so sánh trong bảng 3.3, chúng tôi báo cáo kết quả của sáu kiến trúc khác nhau với các xương sống khác nhau: Resnet (34, 50 và 101), MobilenetV2 và Efficientnet-b2. Đối với necks, chúng tôi báo cáo một thử nghiệm duy nhất bằng cách sử dụng phiên bản đơn giản, do tính hiệu quả hơn của BiFPN [45].

Đánh giá hiệu suất: Chúng tôi đánh giá hiệu suất cho các tác vụ Phát hiện đối tượng bằng chỉ số mAP@0,5 [49] và Accuracy

Để đánh giá phân đoạn làn đường bằng chỉ số IoU. Cuối cùng, đối với tác vụ phân loại điều kiện lái xe, chúng tôi đánh giá riêng ba tác vụ phân loại nhiều lớp để tính điểm F1 trên tất cả các lớp

Bảng 3.3. Kết quả của các cấu hình khác nhau cho tất cả các tác vụ.

| Model      | Backbone | Neck    | Object Detection |                 | Lane Segment (IoU) | Scene Classification (F1) |       |             | FPS (Jetson Nano) |
|------------|----------|---------|------------------|-----------------|--------------------|---------------------------|-------|-------------|-------------------|
|            |          |         | mAP (Boxes)      | Accuracy (Occl) |                    | Weather                   | Scene | Time of Day |                   |
| RN34-sim   | Resnet34 | Simpl e | 0.810            | 0.913           | 0.940              | 0.815                     | 0.790 | 0.932       | 4.86              |
| RN34-bifpn | Resnet34 | BiFP N  | 0.865            | 0.910           | 0.944              | 0.817                     | 0.791 | 0.932       | 9.62              |

|              |                 |       |       |       |       |       |       |       |       |
|--------------|-----------------|-------|-------|-------|-------|-------|-------|-------|-------|
| RN50-bifpn   | Resnet50        | BiFPN | 0.874 | 0.911 | 0.948 | 0.822 | 0.794 | 0.935 | 7.69  |
| RN101-bifpn  | Resnet101       | BiFPN | 0.881 | 0.912 | 0.950 | 0.825 | 0.794 | 0.934 | -     |
| EnB2-bifpn   | Efficientnet b2 | BiFPN | 0.872 | 0.907 | 0.947 | 0.821 | 0.790 | 0.937 | 7.39  |
| Mobbv2-bifpn | Mobilenet V2    | BiFPN | 0.830 | 0.9   | 0.933 | 0.813 | 0.790 | 0.934 | 14.63 |

Để đánh giá hiệu quả của mô hình khi triển khai trên thực tế, chúng tôi sử dụng nền tảng máy tính nhúng Jetson Nano với 0,5 TOP và mức tiêu thụ điện năng 5W. Chúng tôi đã cài đặt hệ điều hành OS phiên bản 4.6.1 SDK Jetpack và nén mô hình với nén mô hình bằng cách sử dụng TensorRT (phiên bản 8.2.1) với kiểu dữ liệu 16 bit. Để đảm bảo tính ổn định và tiêu thụ tài nguyên thấp, chúng tôi sử dụng mô hình Mobbv2-bifpn để giải quyết các tác vụ với tốc độ **14.63 FPS**.

Kết quả kiểm chứng “*Giải nhất UIT Car Racing 2022 – bảng chuyên nghiệp*” (hình 3.33) chứng minh tính hiệu quả và độ chính xác của phương pháp đề xuất.





a) Ảnh từ Camera

b) Ảnh đã được dự đoán

Hình 3.33. Kết quả ứng dụng trên xe mô hình

### 3.1.4. Thử nghiệm trên xe điện RENAULT TWIZY

Cùng với thành công khi thiết kế mô hình đa tác vụ và triển khai trên Jetson Nano, chúng tôi quyết định tiếp tục triển khai mô hình trên môi trường thực tế trên xe Renault Twizy tại trường Đại học Sư phạm Kỹ thuật TPHCM (hình 3.34).



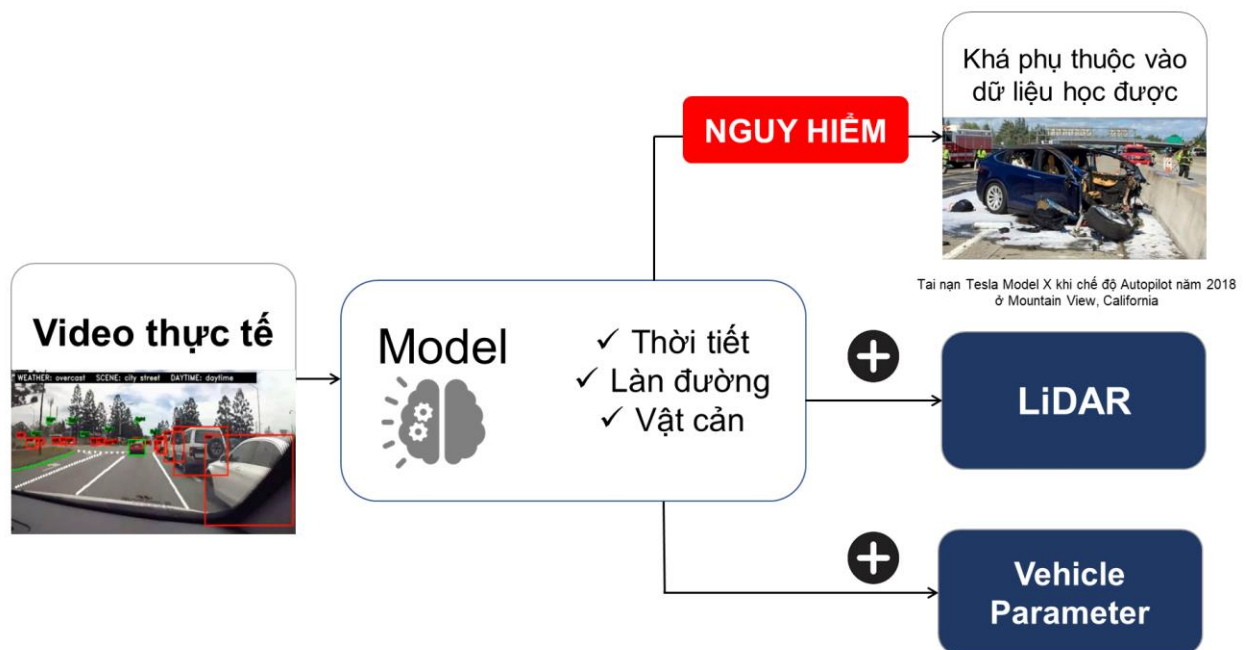
Hình 3.34. Model xe điện thử nghiệm Renault Twizy

Nhưng trước hết chúng tôi thử nghiệm trên trước trên các video thực tế và nhận thấy vẫn còn vấn đề liên quan đến tính đạo đức AI, nhất là với hệ thống Autopilot của Tesla. Năm 2018, hệ thống đã bị chỉ trích vì không nhận dạng và điều khiển được các đối tượng giao thông khác nhau, đặc biệt là xe máy, xe tải và người đi bộ. Nó cũng chưa tốt trong việc xử lý các tình huống khó khăn như trời tối, mưa, tuyết và không có dấu vết

đường. Nghiên cứu cũng cho thấy rằng hệ thống có thể gây ra sự lãng phí trong việc quản lý tình trạng của người lái xe. Bên cạnh đó, tỷ lệ phát hiện được vật cản chưa đạt đến 100%.

Với các vấn đề trên, tôi đề xuất bổ sung hai tác vụ như hình 3.35 nhằm tăng độ tin cậy cho hệ thống là:

- 1/ Phát hiện vật cản (LIDAR) được trình bày ở phần 3.2.
- 2/ Thu thập thông số xe được trình bày ở phần 3.3.



Hình 3.35. Bổ sung hai tác vụ LIDAR và CANBUS cho hệ thống

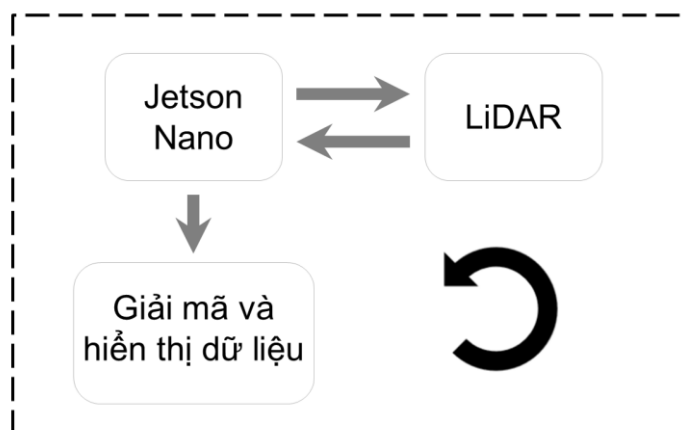
### 3.2. LIDAR

LiDAR (Light Detection and Ranging) là một công nghệ đo khoảng cách bằng cách sử dụng tia laser để xác định vị trí của đối tượng. Có nhiều ưu điểm khi sử dụng LiDAR, bao gồm:

1. Chính xác cao: LiDAR có thể đo độ chính xác cao và có thể đo khoảng cách tới đối tượng xa.
2. Đo trong điều kiện tối: LiDAR có thể hoạt động trong điều kiện tối, nắng, mưa và bụi bẩn, cho phép đo khoảng cách tới đối tượng trong môi trường khó khăn.

3. Đo khoảng cách tới đối tượng diện rộng: LiDAR có thể đo khoảng cách tới đối tượng diện rộng, từ các đối tượng gần đến xa.
4. Tạo bản đồ 3D: LiDAR có thể tạo ra bản đồ 3D chi tiết của môi trường xung quanh, được sử dụng trong nhiều lĩnh vực như xây dựng, giao thông, quản lý tài nguyên.
5. Giảm tình trạng vô lăng: LiDAR có thể hoạt động tự động và không cần sự giám sát trực tiếp của con người.

Với ưu điểm trên, chúng tôi sử dụng thiết bị đo khoảng cách LMS 511-PRO thuộc nhà phát hành SICK đến từ Đức. Khả năng hoạt động của LMS 511-PRO khá tốt, khắc phục được những yếu điểm của họ thiết bị đo khoảng cách bằng laser, vì thế chúng tôi chọn LMS 511-PRO làm thiết bị đo khoảng cách trong đề tài này với sơ đồ giải thuật như hình 3.36.



Hình 3.36. Sơ đồ giải thuật LiDAR

### 3.2.1. Thông số kỹ thuật [50]

LMS 511-PRO là một cảm biến thông minh với nhiều chức năng hỗ trợ đo khoảng cách được lập trình trong thiết bị. Tầm hoạt động lên đến 80m và góc quét lên đến 190 độ là điểm mạnh khắc phục các nhược điểm của dòng cảm biến đo khoảng cách bằng laser (hình 3.37).

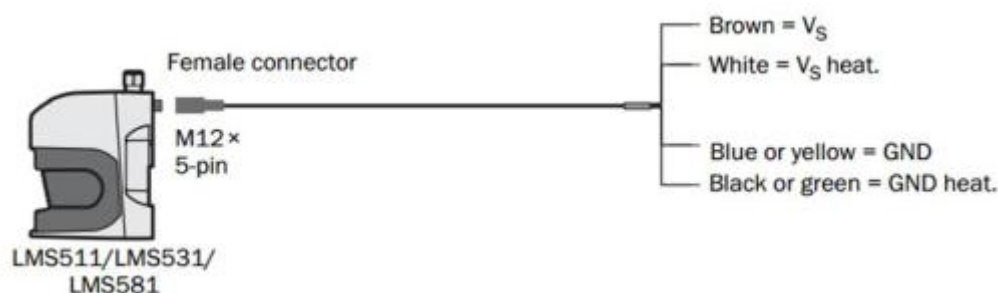
Với tần số quét từ 25Hz lên đến 100Hz, thiết bị có khả năng quét môi trường xung quanh nhanh chóng, thích hợp với lấy dữ liệu ở thời gian thực. Độ phân giải góc từ 1 độ xuống còn 0.1667 độ đảm bảo không bỏ sót vật thể khi quét.

|                                    |                                                                        |
|------------------------------------|------------------------------------------------------------------------|
| Scanning frequency                 | 25 Hz / 35 Hz / 50 Hz / 75 Hz / 100 Hz                                 |
| Angular resolution                 | 0.042°<br>0.083°<br>0.1667°<br>0.25°<br>0.333°<br>0.5°<br>0.667°<br>1° |
| Heating                            | Yes                                                                    |
| Working range                      | 1 m ... 80 m                                                           |
| Scanning range<br>At 10% remission | 40 m                                                                   |
| Spot size                          | 11.9 mrad                                                              |
| Amount of evaluated echoes         | 5                                                                      |
| Fog correction                     | Yes                                                                    |

Hình 3.37. Thông số kỹ thuật LMS511-PRO

Ngoài ra, thiết bị còn có chức năng lọc nhiễu với 5 bước giúp cho kết quả đo khoảng cách chính xác hơn. Ở chức năng lọc nhiễu này, có thể chọn lấy dữ liệu ở tầng đầu tiên (First Echo), tầng cuối cùng (Last Echo) hay lấy dữ liệu ở tất cả các tầng (All Echo).

Thiết bị sử dụng nguồn điện một chiều **24V**, như hình 3.38. Với hai dây dương  $V_S$  và  $V_S$  heat (nguồn cung cấp dùng để sưởi thiết bị trong trường hợp thiết bị hoạt động trong môi trường lạnh) và hai dây âm GND và GND heat.



Hình 3.38. Sơ đồ dây nguồn LMS511

Thiết bị sử dụng nhiều giao thức giao tiếp như USB, CAN Bus, Serial hay Ethernet (hình 3.39). Giao thức truyền tốc độ cao như Ethernet với tốc độ truyền lên đến 10/100 Mbit/s thích hợp với hoạt động thời gian thực, hay giao thức truyền đơn giản Serial (RS-232 hoặc RS-422) cũng có tốc độ truyền tương đối ổn từ 9.6 đến 500kBaud.

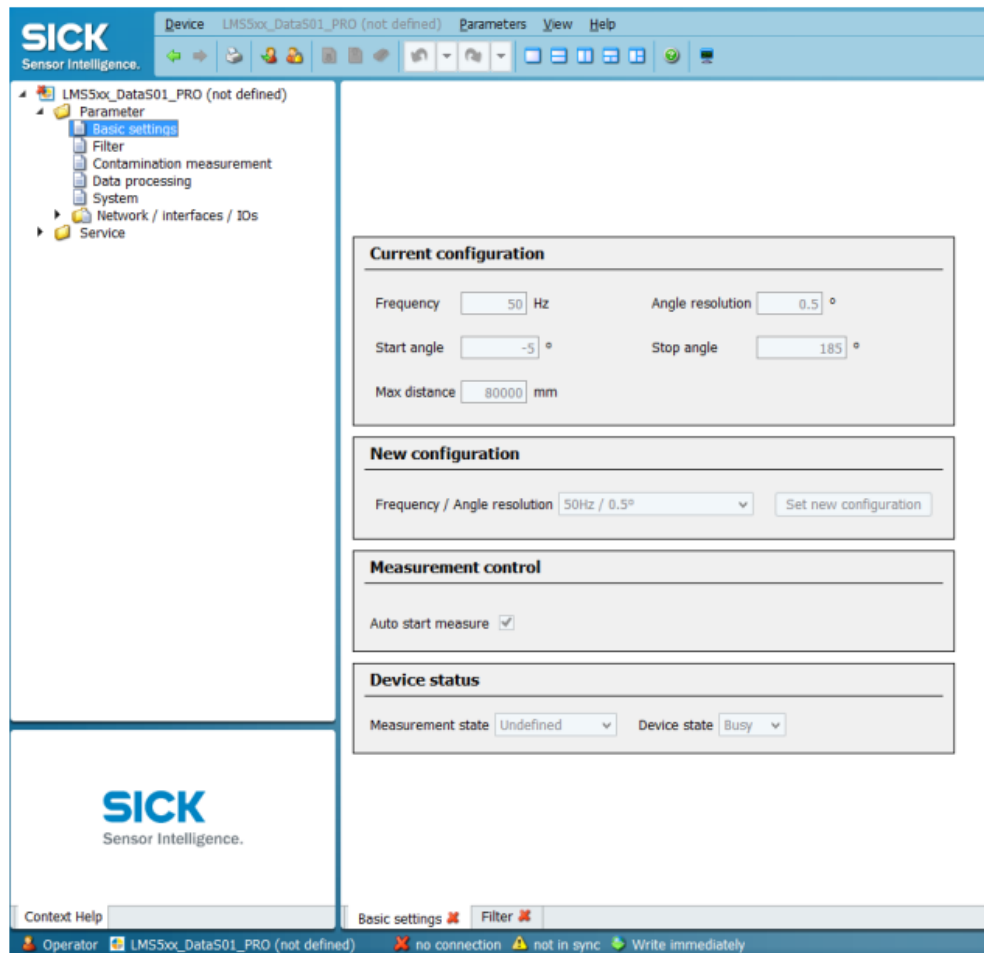
|                        |                         |
|------------------------|-------------------------|
| <b>Ethernet</b>        | ✓ , TCP/IP              |
| Remark                 | OPC                     |
| Function               | Host                    |
| Data transmission rate | 10/100 MBit/s           |
| <b>Serial</b>          | ✓ , RS-232, RS-422      |
| Function               | Host                    |
| Data transmission rate | 9.6 kBaud ... 500 kBaud |
| <b>CAN</b>             | ✓                       |
| Function               | Extension of outputs    |
| <b>USB</b>             | ✓                       |
| Remark                 | Mini-USB                |
| Function               | AUX                     |
| Data transmission rate | 9.6 kBaud ... 500 kBaud |

Hình 3.39. Giao thức giao tiếp của LMS511

### 3.2.2. Cấu hình thiết bị bằng phần mềm SOPAS ET

Các thành phần cần thiết lập (hình 3.40) bao gồm: Tần số quét (Frequency), độ phân giải góc (Angle resolution), góc bắt đầu quét (Start angle), góc kết thúc quét (Stop angle). Mặc định thiết bị quét ở tần số 50Hz, với độ phân giải góc 0.5 độ, góc bắt đầu quét là -5 độ và góc kết thúc quét là 185 độ, khoảng cách quét tối đa là 80m. Nhà cung cấp chỉ định các thông số góc bắt đầu quét, góc kết thúc quét và khoảng cách quét tối đa không thể thay đổi, chúng tôi chỉ có thể thay đổi các thông số là tần số quét, độ phân giải góc.

Đầu ra của thiết bị là khoảng cách theo góc, nếu thiết bị quét 190 độ với độ phân giải 0.5 độ thì chúng tôi có 381 đầu ra. Thiết bị quét 190 độ nhưng dữ liệu đầu ra của thiết bị có thể lựa chọn nhỏ hơn. Vì góc quét của camera là 60 độ nên chúng tôi thiết lập lấy dữ liệu đầu ra ở góc bắt đầu 55 độ, góc kết thúc 125 độ và với độ phân giải 0.5 nên chúng ta có **141** giá trị (**Measurements**).



Hình 3.40. Thiết lập cấu hình hoạt động LMS 511

Bước thiết lập cuối cùng là thiết lập giao thức truyền dữ liệu. Trong dự án này chúng tôi chọn giao thức giao tiếp là Serial nên chúng tôi chỉ thiết lập ở phần Serial trong mục Network/ interfaces/ Ios. Các thông số cần được thiết lập bao gồm: Baudrate, kiểu giao tiếp Serial RS-232, độ dài dữ liệu truyền đi Databits / Parity. Do hệ thống yêu cầu chạy thời gian thực nên chúng tôi thiết lập Baudrate là **230400** để đạt được **28.87 FPS** dựa vào công thức tính tốc độ từ nhà cung cấp [51] trang 107 như sau:

$$\begin{aligned}
 \text{Data of one Telegram} &= \text{Header} + \text{Measurements} + \text{end of telegram} \\
 \Rightarrow \text{Data of one Telegram} &= 81 \text{ Byte} + 141 \text{ Measurements} + 12 \text{ Byte} \\
 \Rightarrow \text{telegram size} &= 81 \text{ Byte} + (141 \times 5 \text{ Byte}) + 12 \text{ Byte} = 798 \text{ Byte per Telegram}
 \end{aligned}$$

Ta có:  $\text{Number of telegrams per second} = \text{Actual Speed} \div \text{telegram size}$

Với Actual Speed là tốc độ truyền thực tế (Byte/s) được tra trong bảng 3.4.

$$\Rightarrow \text{Actual Speed} = 23040$$

$$\Rightarrow \text{Number of telegrams per second} = \frac{23040}{798} = 28.87$$

Bảng 3.4. Bảng tốc độ truyền của chuẩn giao tiếp RS-232 [52]

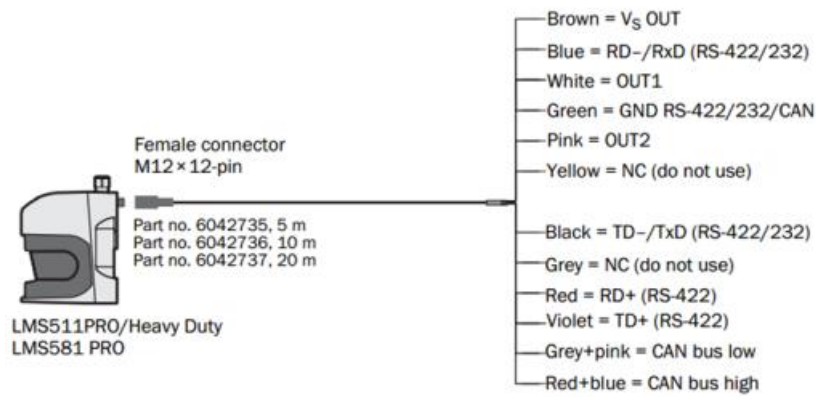
| Bauds  | Speed (bytes/s) | Actual Speed<br>(bytes/s) | Actual duration |
|--------|-----------------|---------------------------|-----------------|
| 9600   | 1200            | 960                       | 1.042 ms        |
| 115200 | 14400           | 11520                     | 86.806 $\mu$ s  |
| 230400 | 28800           | 23040                     | 43.403 $\mu$ s  |
| 576000 | 72000           | 57600                     | 17.361 $\mu$ s  |

Chúng tôi lưu lại các thiết lập cấu hình thiết bị ở mục Parameters -> Save permanently và đăng xuất khỏi thiết bị. Các thiết lập của cấu hình được lưu lại tại ổ nhớ EEPROM của thiết bị do đó chúng tôi không cần phải thiết lập lại cấu hình với mỗi lần sử dụng.

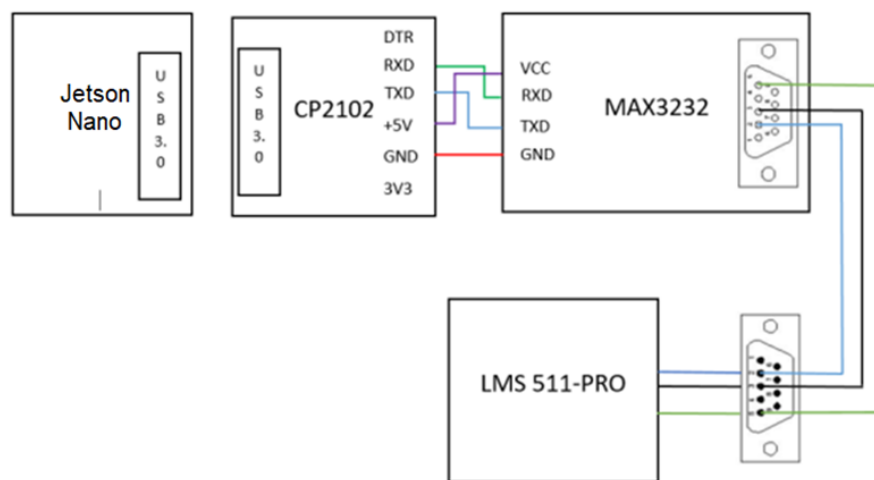
### 3.2.3. Lấy dữ liệu từ LMS 511-PRO

#### 3.2.3.1. Phần cứng

Như đã đề cập ở phần trước, chúng tôi giao tiếp với LMS 511-PRO qua giao thức RS-232 do đó chúng tôi sử dụng dây dữ liệu LMS 511-PRO 12 chân ra được miêu tả như trong hình 3.41 trong đó các dây thuộc giao thức RS-232 bao gồm các dây RXD ( xanh nước biển), TXD (đen), GND (xanh lá cây). Các dây này sẽ được kết nối với giắc đực DB9 (cổng RS-232 loại 9 chân) để kết nối với mô-đun MAX3232, các chân TXD, RXD, GND, VCC sẽ kết nối với các chân TXD, RXD, GND và 5V thuộc mô-đun CP2102, mô-đun CP2102 sẽ kết nối với Jetson Nano qua cổng USB 3.0 như hình 3.42.



Hình 3.41. Sơ đồ dây giao tiếp dữ liệu LMS511



Hình 3.42. Mạch giao tiếp LMS 511-PRO

Để lấy dữ liệu từ thiết bị, chúng tôi sẽ truyền lệnh lấy dữ liệu thiết bị từ Jetson Nano trên nền môi trường Python. LiDAR giao tiếp qua ngôn ngữ ColaB – một kiểu ngôn ngữ riêng được quy định bởi nhà phát hành. Lệnh này được truyền từ Jetson Nano qua module CP2102 và đến LiDAR theo đường TXD. LiDAR nhận được mã lệnh từ Jetson Nano và sẽ phản hồi dữ liệu đo đạc qua đường RXD và trở về lại Jetson Nano. Để đọc dữ liệu trả về, chúng tôi tham khảo những điện tín ASCII từ nhà phát hành [51] trang 90-91 và tiến hành giải mã, hiển thị dữ liệu trên môi trường Python.

### 3.2.3.2. Giải mã

Khi truyền một bức điện tín từ Jetson Nano cho thiết bị, thiết bị sẽ nhận biết và truyền bức điện tín phản hồi về lại Jetson Nano, cấu trúc của điện tín [51] được quy định bởi nhà sản xuất như sau:

<STX>(Command type){SPC}(Command){SPC}(Parameter)<ETX>

Trong đó:

- STX: Start of text (ký tự bắt đầu một điện tín), ví dụ STX ở điện tín ASCII hay 0x02 ở điện tín HEX.
- Command type: gồm có các chế độ sRN, sWN, sMN, sEN ở điện tín ASCII.
- Command tương ứng với Command type.
- ETX End of text (ký tự kết thúc một điện tín), ví dụ ETX ở điện tín ASCII hay 0x03 ở điện tín HEX.

Để thu thập được dữ liệu trong chế độ quét từ LiDAR, chúng tôi gửi cấu trúc mã điện tín theo nhà phát hành với chế độ SRN LMDscandata (hình 3.43). Vấn đề khi gửi điện tín xuất hiện từ các ký tự <STX> và <ETX>, vì khi gửi điện tín ASCII, các ký tự <STX> và <ETX> không được dịch chính xác, nên thiết bị sẽ không hiểu mã lệnh được yêu cầu. Để khắc phục vấn đề này, chúng tôi chuyển các ký tự sang mã HEX, với <STX> tương đương với 0x02 và <ETX> tương đương với 0x03.

|        |        |                                                                         |
|--------|--------|-------------------------------------------------------------------------|
| CoLa A | ASCII  | <STX>sRN[SPC]LMDscandata<ETX>                                           |
|        | Hex    | 02 73 52 4E 20 4C 4D 44 73 63 61 6E 64 61 74 61 03                      |
| CoLa B | Binary | 02 02 02 02 00 00 00 0F 73 52 4E 20 4C 4D 44 73 63 61 6E 64 61 74 61 05 |

Hình 3.43. Điện tín yêu cầu gửi dữ liệu về máy tính

Sau đó trong mẫu dữ liệu phản hồi từ thiết bị (ví dụ như hình 3.44), dữ liệu đo đạc bắt đầu sau ký tự DIST1, tiếp đến là hai thông số Scale factor (ví dụ: 3F800000) và Scale Factor offset (ví dụ: 00000000), thông số tiếp đến là góc bắt đầu quét - Start angle, (ví dụ: 186A0), bước nhảy góc quét - Steps, (ví dụ: 1388), số lượng dữ liệu đo lường - Amount of data, (ví dụ: 15), và cuối cùng dữ liệu đo lường là các ký

tự từ 8A1 đến 906, các ký tự 0 0 0 0 0 cuối cùng là các thiết lập đầu ra của dữ liệu, mặc định các ký tự đó là 0 0 0 0 0 và không đổi trong quá trình hoạt động.

Tiếp theo chúng tôi sẽ giải mã dữ liệu đã phân tích ở trên. Các ký tự trả về là điện tín ASCII nhưng các dữ liệu lại có cấu trúc thuộc về mã HEX (hệ 16) và để giải mã dữ liệu chúng tôi chuyển các dữ liệu từ mã HEX về mã Decimal (hệ 10).

Ví dụ ở thành phần góc bắt đầu quét (Start angle), dữ liệu trả về là 186A0 (hệ 16) nếu chúng tôi đổi chuỗi này về hệ 10 thì chúng tôi được dãy số 100000 tức là 100 độ. Tiếp theo chúng tôi đến thành phần bước nhảy góc (Steps), dữ liệu trả về là 1388 (hệ 10) chúng tôi chuyển về hệ 10 và được dãy số 5000 tức là 0.5 độ. Tiếp đến là thành phần số lượng dữ liệu (Amount of data), dữ liệu trả về là 15 (hệ 16) và chúng tôi chuyển qua hệ 10, kết quả thu được dãy số 21 tức là 21 dữ liệu đo lường. Sau thành phần số lượng dữ liệu chính là dữ liệu đo lường từ thiết bị, ví dụ ở vị trí dữ liệu đầu tiên, chuỗi trả về là 8A1 (hệ 16) chúng tôi chuyển về hệ 10 và thu được được dãy số 2209 tức là 2209mm.

Các bước ấy được tiến hành lặp đi lặp lại từ chuỗi dữ liệu ở góc đầu tiên (Start angle) đến chuỗi dữ liệu ở góc kết thúc (Stop angle). Kết quả chúng tôi thu được dãy dữ liệu khoảng cách theo từng góc.

#### ASCII

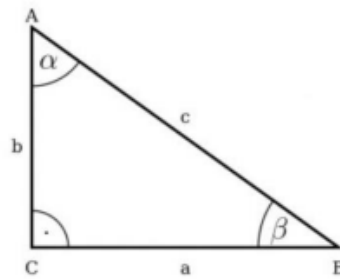
```
sRA LMDscandata 1 1 89A27F 0 0 343 347 27477BA9 2747813B 0 0 7 0 0
1388 168 0 1 DIST1 3F800000 00000000 186A0 1388 15 8A1 8A5 8AB 8AC
8A6 8AC 8B6 8C8 8C2 8C9 8CB 8C4 8E4 8E1 8EB 8E0 8F5 908 8FC 907 906 0
0 0 0 0 0
```

Hình 3.44. Mẫu dữ liệu phản hồi từ thiết bị

#### 3.2.4. Hiển thị dữ liệu

Để hiển thị các điểm trên hai trục tọa độ x,y. Chúng tôi áp dụng công thức lượng trong tam giác vuông (hình 3.45), cụ thể khoảng cách trích xuất từ dữ liệu thiết bị tương đương với cạnh AB, và góc ứng với khoảng cách đang xét tương đương với góc  $\beta$ . Như vậy, để biểu diễn điểm A trên đồ thị ta sử dụng công thức lượng trong tam giác vuông

để tìm tọa độ x,y tương ứng các cạnh CB và AC trong tam giác. Cụ thể, cạnh  $CA=AB*\sin(\beta)$  và cạnh  $CB=AB*\cos(\beta)$ .

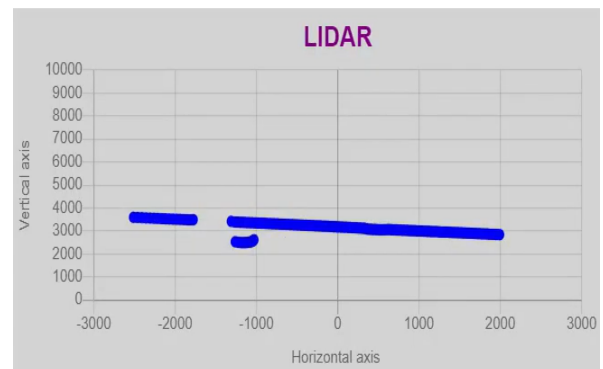


Hình 3.45. Tam giác vuông

Tiếp đến ta vẽ các điểm với tọa x,y vừa tìm được, kết quả hiển thị dữ liệu thiết bị LMS 511 được miêu tả trong hình hình 3.46.



a) Đầu vào

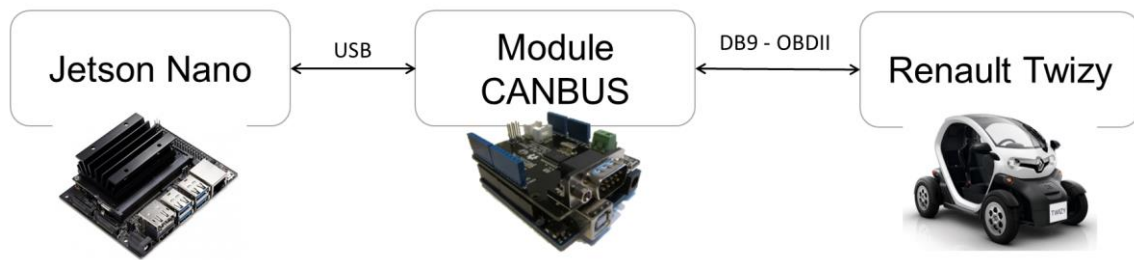


b) Trực quan hoá dữ liệu từ LiDAR

Hình 3.46. Kết quả thiết bị LMS 511-PRO

### 3.3. Thu thập thông số xe

Để có thể thu thập được dữ liệu thông qua cổng OBD II female của xe, chúng tôi thiết kế phần cứng như hình 3.47 có một cổng là cổng DB9 male cổng này sẽ kết nối với xe thông qua cáp chuyển đổi OBD II male – DB9 female. Đồng thời có một đầu là cổng USB type A female để kết nối với Jetson Nano dùng cho việc truyền nhận dữ liệu tích hợp vào hệ thống và từ đó thiết kế giao diện hiển thị dữ liệu nhận được.



Hình 3.47. Sơ đồ tổng quan về phương pháp thu thập dữ liệu

### 3.3.1. Môi trường nghiên cứu

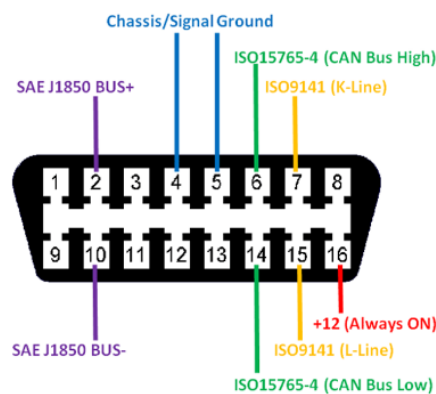
Phần mềm: Arduino IDE

Phần cứng: Arduino Uno [53], Module CAN BUS SHIELD MCP 2515 [54], Cáp OBD II male – DB9 Female

### 3.3.2. Phương pháp thu thập dữ liệu CAN Model xe điện RENAULT TWIZY

#### 3.3.2.1. Kiểm tra tín hiệu mạng CAN bằng máy Hantek 6022BE

Để có thể đo được tín hiệu CAN ta cần tải Driver HT6022 và Software HT6022 về laptop và sau đó xác định chân trên cổng OBD II (hình 3.48), hai đầu dây tín hiệu kết nối với 2 chân CAN Bus High/Low và một dây nối với Signal Ground trên cổng OBD II.



Hình 3.48. Các chân trên cổng OBD

Sau đó mở phần mềm Hantek6022BE, ta set thông số Time/DIV khoảng 5 hoặc 10  $\mu$ s để dễ dàng bắt xung. Lưu ý ta cần chỉnh 2 kênh CH1 và CH2 là 5V và kéo xung về giữa để dễ dàng phóng to xem xung.



Hình 3.49. Kết quả xung đo được

Như trong hình 3.49 ta thấy rằng xung màu vàng là CANH và xung màu xanh lá là CANL vì cạnh lên của xung màu vàng là cạnh xuống của xung màu xanh lá và xung màu vàng bắt cạnh lên trước. Với CAN high: điện áp dao động từ 2.5 - 3.75 V và CAN low: điện áp dao động từ 1.25 - 2.5 V như hình 3.50.

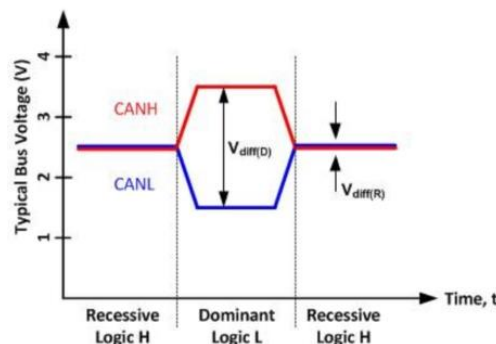


Figure 2: CAN bus signal levels

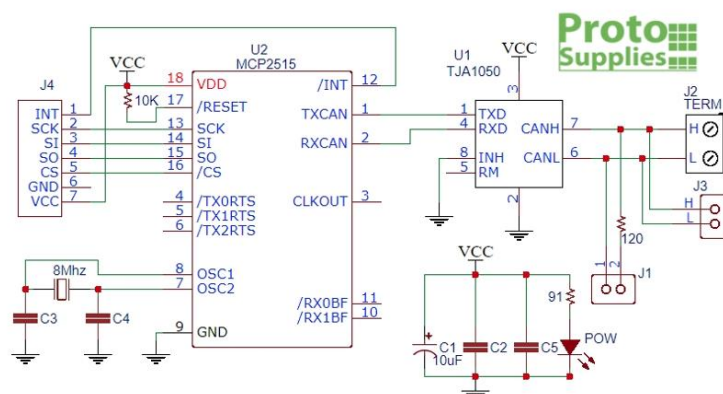
$$V_{diff} = V_{CANH} - V_{CANL}$$

Hình 3.50. CAN bus signal levels

### 3.3.2.2. Thiết kế MODULE thu thập dữ liệu trực tiếp.

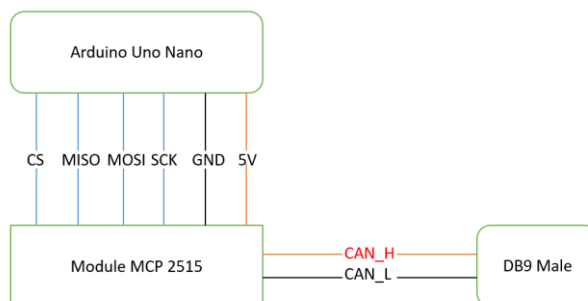
Như chúng tôi đã trình bày ở cơ sở lý thuyết thì một CAN BUS Shield cần có vi điều khiển, CAN controller, CAN transceiver, nếu node đó là node đầu cuối thì cần có một điện trở 120 ôm. Ở đây nhóm đã sử dụng Arduino để làm vi điều khiển và

Module MCP 2515 (hình 3.51). Module này tích hợp bao gồm CAN controller là MCP 2515 và CAN transceiver là TJA1050.



Hình 3.51. Module MCP 2515

Tổng quan về phần thiết kế như hình 3.52 gồm có Module MCP2515 kết nối với Arduino Uno R3 thông qua chuẩn giao tiếp SPI [34]. Đồng thời Arduino cấp nguồn 5V và GND cho Module MCP2515. Để có thể kết nối với mạng CAN BUS trên xe thông qua cổng OBD II female, ta cần có một cáp chuyển đổi OBD II male – DB9 female đồng thời mạch CAN BUS shield cần phải có đầu ra DB9 male để kết nối với đầu DB9 female của cáp chuyển đổi.



Hình 3.52. Sơ đồ kết nối CAN BUS SHIELD

Để có thể đọc được dữ liệu CAN, đầu tiên chúng ta cần lưu ý mạng CAN BUS trên xe điện Renault Twizy hoạt động với tần số **1000KBP**. Sau đó, chúng ta viết chương trình trên phần mềm IDE do vi điều khiển chúng ta sử dụng trong CAN BUS shield là Arduino Uno. Dữ liệu của các hệ thống trên Renault Twizy. Dữ liệu này sẽ được truyền trên CAN BUS khi mạng CAN BUS trên xe hoạt động. Dữ liệu lúc này theo đường sẽ được đi từ CAN BUS → CAN transceiver → CAN controller → vi điều khiển.

Sau khi nhận được dữ liệu: ID, data từ CAN BUS như hình 3.53. Chúng tôi sẽ in dữ liệu nhận được và hiển thị nó trên serial monitor giúp chúng ta có thể quan sát

được dữ liệu hiện có trên arduino thông qua giao tiếp Uart. Dữ liệu mà chúng ta nhận được đó là ID, data hiển thị dưới dạng hexa (hệ cơ số 16). Lúc này chúng ta chỉ biết CAN BUS gồm có những ID và data cụ thể của từng ID đó.

```
Get data from ID: 423
3    0    255  255  0    224  0    0

Get data from ID: 426
0    24    2    0    0    0    0

Get data from ID: 597
0    240  0    162  54    0    1    0

Get data from ID: 5D7
0    0    0    3    208  128  0

Get data from ID: 599
0    0    0    156  255  49    0    0
```

Hình 3.53. Dữ liệu từ CAN BUS

### 3.3.3. Giải mã các ID gói dữ liệu

Việc mã hóa dữ liệu sẽ dựa vào các danh mục bảng ID do CAN BUS gửi lên, độ dài cho mỗi ID gửi lên cho xe Renault Twizy sẽ rơi khoản 6 bytes đến 8 bytes. Để mã hóa các bytes dữ liệu này chúng ta cần sử dụng đến các thuật toán thao tác Bitwise.

Ngoài ra, còn dựa vào phương pháp thực nghiệm sẽ kiểm tra sự thay đổi của hệ thống trên xe sẽ làm data của ID nào thay đổi để xác định ID, data đó của hệ thống nào trên xe. Sau khi xác định được cụ thể ID, data của đối tượng cụ thể. Chúng tôi tiếp tục thực nghiệm để tính toán đưa ra công thức cách tính cũng như là ý nghĩa của data. Cụ thể được liệt kê từ bảng 3.5 đến 3.16.

Bảng 3.5. Bảng giá trị ID 0x155

| Độ dài | Byte | Ý nghĩa             | Cách tính                                                                                                                 |
|--------|------|---------------------|---------------------------------------------------------------------------------------------------------------------------|
| 8      | 1    | Điều khiển sạc      | Giá trị từ 0 - 7<br>giá trị * 300W = công suất sạc, hoặc<br>giá trị * 5A = dòng sạc<br>Giá trị = 0 => bình đầy(SOC 100 %) |
| 8      | 2+3  | Dòng của ắc quy kéo | 12 bit thấp => giá trị thập phân.<br>giá trị thập phân - 2000 = giá trị sai lệch                                          |

|   |     |                          |                                                                                                                                                                   |
|---|-----|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|   |     |                          | Giá trị sai lệch / 4 = dòng của ắc quy kéo<br>Giá trị sai lệch (+) => dòng cung cấp cho sevcon<br>Giá trị sai lệch (-) => dòng được nạp vào từ phanh tái sinh,... |
| 8 | 4   | Trạng thái CAN của Twizy | "94h: in it, off, error<br>54h: hoạt động"                                                                                                                        |
| 8 | 5+6 | SOC                      | Giá trị thập phân / 400 = % SOC                                                                                                                                   |

Bảng 3.6. Bảng giá trị 0x196

| Độ dài | Byte | Ý nghĩa        | Cách tính         |
|--------|------|----------------|-------------------|
| 8      | 6    | Nhiệt độ motor | Temp = value - 40 |

Bảng 3.7. Bảng giá trị ID 0x19F

| Độ dài | Byte | Ý nghĩa     | Cách tính                                                                      |
|--------|------|-------------|--------------------------------------------------------------------------------|
| 8      | 3+4  | speed (+/-) | 8 bit b3 + 4 bit b4<br>$Rpm = (value - 2000) * 10$<br>$km/h = rpm / 7250 * 80$ |

Bảng 3.8. Bảng giá trị ID 0x59B

| Độ dài | Byte | Ý nghĩa                              | Cách tính                                                                      |
|--------|------|--------------------------------------|--------------------------------------------------------------------------------|
| 8      | 1    | gear selection                       | 0x80: D<br>0x20: N<br>0x08: R                                                  |
| 8      | 2    | bàn đạp phanh<br>Đèn phanh và engine | Bit 0: Brake pedal / light ( 0x01 )<br>Bit 2 + 3: motor/controller on ( 0x0C ) |
| 8      | 3    | Công suất / Moment                   | 100 = null<br>> 100 delivery<br>< 100: recup<br>( 88 - 163 ) ?                 |

|   |   |                                           |                                                                                                              |
|---|---|-------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| 8 | 4 | vị trí bàn đạp ga                         | 0x00: stop<br>0xFD: max                                                                                      |
| 8 | 5 | bàn đạp phanh<br>Đèn phanh và Motorstatus | 4 bit cao: 4 = brake inactive / 6 = brake active<br>4 bit thấp: 0 = motor/controller off, 4 = on , 8 = error |
| 8 | 6 | Điện áp tổng cộng                         | Voltage = value / 2                                                                                          |

Bảng 3.9. Bảng giá trị ID 0x424

| Độ dài | Byte | Ý nghĩa                              | Cách tính                                         |
|--------|------|--------------------------------------|---------------------------------------------------|
| 8      | 1    | Trạng thái                           | 9xh: lỗi Pin<br>00: init<br>11: ready<br>12: stop |
| 8      | 3    | Công suất sạc tối đa<br>( max recup) | Giá trị * 500 [0 - 30000]                         |
| 8      | 4    | Công suất xả tối đa<br>( max drive ) | Giá trị * 500 [0 - 30000]                         |
| 8      | 5    | Nhiệt độ Pin min                     | Bằng giá trị thập phân - 40                       |
| 8      | 7    | Tình trạng Pin SOH                   | Giá trị thập phân (%)                             |
| 8      | 8    | Nhiệt độ Pin max                     | Bằng giá trị thập phân - 40                       |

Bảng 3.10. Bảng giá trị ID 0x425

| Độ dài | Byte | Ý Nghĩa    | Cách tính                                                                          |
|--------|------|------------|------------------------------------------------------------------------------------|
| 8      | 1    | Trạng thái | 1D: init<br>24: ready<br>0A: charging<br>2A: driving, trickle<br>2C: start trickle |
| 8      | 5-6  | Điện áp    | value = (Voltage - 13.051)*6.981                                                   |

|   |     |         |                                                    |
|---|-----|---------|----------------------------------------------------|
| 8 | 7-8 | Điện áp | $\text{value} = (\text{Voltage} - 13.051) * 6.981$ |
|---|-----|---------|----------------------------------------------------|

Bảng 3.11. Bảng giá trị ID 0x556

| Độ dài | Byte | Ý Nghĩa            | Cách tính                                         |
|--------|------|--------------------|---------------------------------------------------|
| 8      | 1-8  | Cell Voltage 1 - 5 | 12 bit<br>$\text{Voltage} = \text{giá trị} / 200$ |

Bảng 3.12. Bảng giá trị ID 0x599

| Độ dài | Byte | Ý Nghĩa  | Cách tính                                                |
|--------|------|----------|----------------------------------------------------------|
| 8      | 1-4  | Odometer | $\text{Odo} = \text{value}(\text{decimal}) \text{ (km)}$ |

Bảng 3.13. Bảng giá trị ID 0x554

| Độ dài | Byte | Ý Nghĩa                  | Cách tính                                               |
|--------|------|--------------------------|---------------------------------------------------------|
| 8      | 1-7  | Module temperature 1 - 7 | $\text{Temperature C} = \text{giá trị} - 40 \text{ độ}$ |

Bảng 3.14. Bảng giá trị ID 0x557

| Độ dài | Byte | Ý Nghĩa           | Cách tính                                               |
|--------|------|-------------------|---------------------------------------------------------|
| 8      | 1-8  | Cell Voltage 6-10 | $\text{Temperature C} = \text{giá trị} - 40 \text{ độ}$ |

Bảng 3.15. Bảng giá trị ID 0x55E

| Độ dài | Byte | Ý Nghĩa            | Cách tính                                               |
|--------|------|--------------------|---------------------------------------------------------|
| 8      | 1-8  | Cell Voltage 11-14 | $\text{Temperature C} = \text{giá trị} - 40 \text{ độ}$ |

Bảng 3.16. Bảng giá trị ID 0x55F

| Độ dài | Byte | Ý Nghĩa         | Cách tính                                         |
|--------|------|-----------------|---------------------------------------------------|
| 8      | 6-8  | Voltage overall | 12 bit,<br>$\text{Voltage} = \text{giá trị} / 10$ |

### 3.3.4. Giao tiếp Arduino và Jetson Nano bằng Python [55]

Khi làm dự án Arduino với Jetson Nano, đôi khi chúng ta cần lấy dữ liệu từ Arduino hoặc gửi lệnh để điều khiển MCU. Chúng ta có thể làm điều đó dễ dàng bằng cách giao tiếp qua USB sử dụng ngôn ngữ Python.

Cách gửi dữ liệu từ Arduino đến Jetson gồm có 5 giai đoạn:

- 1) Lập trình Arduino. (Tôi đã sử dụng ví dụ ASCIITable).
- 2) Kết nối Arduino với Jetson bằng đầu nối USB
- 3) Cài đặt pyserial trên Jetson
- 4) Tải code Python (wget <http://bit.ly/jetson-serialpy> -O ser.py)
- 5) Chạy chương trình

Trong hệ thống này, chúng tôi cần gửi một khung truyền có kích thước dữ liệu **15 bytes** gồm các thông tin: 1/ RPM, 2/ Speed, 3/ Odometer, 4/ Motor Temperature, 5/ Gear, 6/ Voltage, 7/ Vehicle Status, 8/ SOC, 9/ Distance, 10/ Battery Temperature, 11/ Motor Status, 12/ TPS, 13/ Required Current, 14/ SOH, 15/ Brake Status

Với kích thước dữ liệu 15 bytes ta tính được thời gian truyền dữ liệu dựa theo công thức:

$$Time = \text{Actual byte duration} * \text{Size}$$

Trong đó:

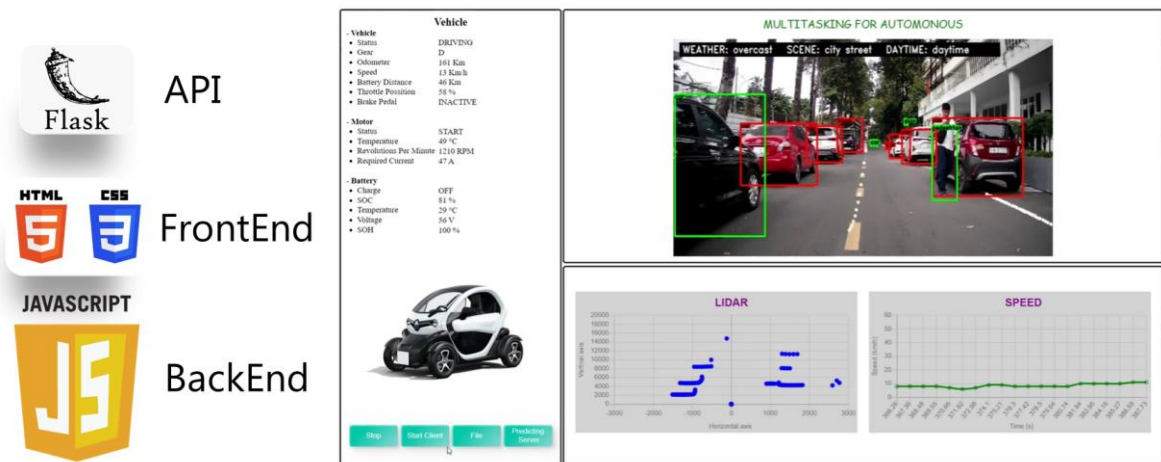
Actual byte duration: Tốc độ truyền thực tế được tra trong bảng

Size: kích thước khung dữ liệu (Bytes)

$$\text{Suy ra: } Time = 86.806 (\mu s) * 15(\text{bytes}) = 1302.09(\mu s) = 0.0013(s)$$

### 3.4. Giao diện người dùng

Trong phần này, chúng tôi sẽ thiết kế giao diện người dùng dựa vào Flask tạo API, ngôn ngữ JavaScript thiết kế BackEnd và hai ngôn ngữ HTML, CSS thiết kế FrontEnd nhằm trực quan hoá các dữ liệu của hệ thống và tạo ra một trải nghiệm dễ sử dụng, thân thiện cho người dùng như hình 3.54. Nó có thể giúp người dùng tìm thấy thông tin cần thiết và hoàn thành các tác vụ một cách dễ dàng hơn.



Hình 3.54. Giao diện người dùng

### 3.4.1. Cấu trúc

Giao diện người dùng của chúng tôi có các layers cho từng tác vụ như sau:

1/ Layer 1: Chứa thông tin trạng thái xe Renault Twizy từ dữ liệu mạng CAN và các nút chức năng thao tác với người dùng.

Các thông tin của xe bao gồm:

- Vehicle: Status, Gear, Odometer, Speed, Battery Distance, Throttle Possition, Brake Pedal
- Motor: Status, Temperature, RPM, Required Current
- Battery: Charge, SOC, Temperature, Voltage, SOH

Ngoài ra, còn có 4 nút thể hiện 4 chức năng chính của UI

- Start/Stop: Hiện/Tắt webcam ở phía server
- Start Clieen/Stop Clientt: Hiện/Tắt webcam ở phía client
- File/Stop File: Chọn/Tắt files ở phía client
- Predict: Dự đoán kết quả dựa trên media đang được show

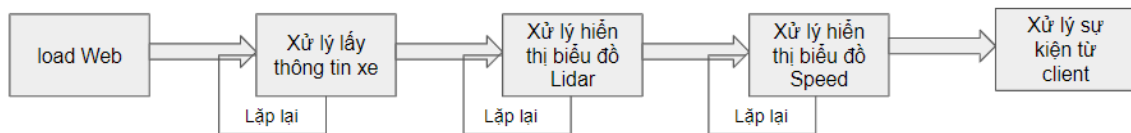
2/ Layer 2: Hiển thị media tùy thuộc vào lựa chọn của client. Bao gồm: webcam của server, webcam của client, video của client đã chọn.

3/ Layer 3: Hiển thị biểu đồ Lidar và Tốc độ xe.

### 3.4.2. Chức năng

Giao diện người dùng của chúng tôi gồm có các chức năng (hình 3.55) như sau:

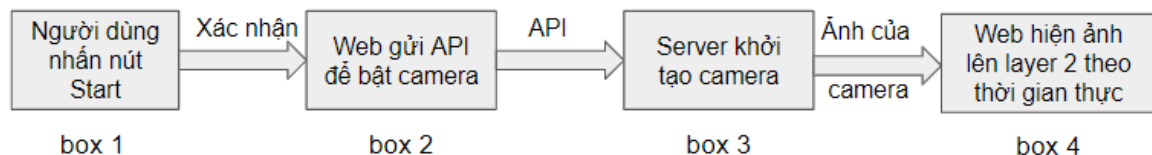
- Bật/Tắt webcam của server: sử dụng camera được kết nối với máy chủ (Jetson Nano)
- Bật/Tắt webcam của client: mục đích cho người dùng thêm lựa chọn có thể sử dụng camera trên thiết bị của mình như điện thoại, laptop,..
- Chọn/Tắt file video đã chọn: mục đích cho người dùng để gỡ lỗi
- Dự đoán kết quả trên thời gian thực: dự đoán kết quả dựa vào media gửi về máy chủ (Jetson Nano)
- Hiển thị biểu đồ Lidar và Speed theo thời gian thực
- Cập nhật thông tin của xe theo thời gian thực



Hình 3.55. Các chức năng của giao diện người dùng

#### 3.4.2.1. Bật/Tắt webcam của server

**Bật webcam server** (hình 3.56): Sự kiện bật webcam server xảy ra khi mà người dùng nhấn nút Start ở Layer 1 (box1). Khi nhấn nút này, web sẽ thực hiện sự kiện bật webcam bằng cách gửi API đến server yêu cầu lấy webcam (box2). Server sẽ nhận được yêu cầu và thực hiện xử lý, server sẽ khởi tạo webcam và gửi từng ảnh của webcam về web (box3). Sau đó, web sẽ nhận được các ảnh từ server và xử lý chúng để hiển thị trên Layer 2 theo thời gian thực (box 4). Lúc này nút Start sẽ được chuyển thành nút Stop.



Hình 3.56. Sơ đồ xử lý bật webcam server

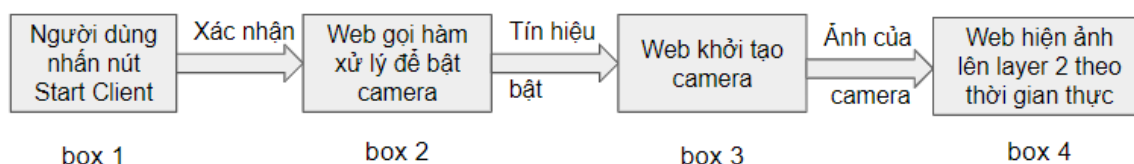
**Tắt webcam server** (hình 3.57): Người dùng nhấn nút Stop ở Layer1 để dừng việc sử dụng webcam. Web thực hiện sự kiện gửi API về server để yêu cầu tắt webcam. Server nhận được yêu cầu và thực hiện xử lý tắt webcam. Đồng thời, hệ thống sẽ xoá biến webcam để tiết kiệm tài nguyên và gửi tín hiệu đã tắt về cho web. Web sẽ nhận được tín hiệu và xử lý để tắt Layer2 và chuyển trạng thái của nút từ Stop sang Start cho lần sử dụng sau.



Hình 3.57. Sơ đồ xử lý tắt webcam server

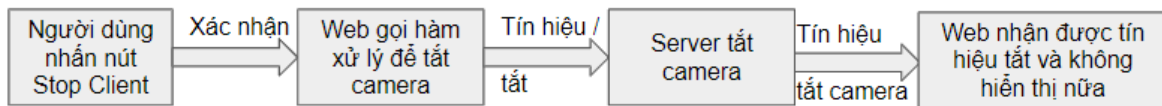
### 3.4.2.2. Bật/Tắt webcam của client

**Bật webcam client** (hình 3.58): Người dùng nhấn nút Start Client ở Layer1, Web thực hiện sự kiện bằng cách kích hoạt camera của thiết bị khách để quay video và hiển thị video đang chạy trên Layer 2. Để ngăn chặn việc mở nhiều camera cùng lúc, hệ thống sẽ tắt các video đang chạy trước đó. Sau khi kích hoạt camera thành công, nút Start Client sẽ chuyển thành Stop Client để người dùng có thể dừng quay video khi cần thiết.



Hình 3.58. Sơ đồ xử lý bật webcam client

**Tắt webcam client** (hình 3.59): Người dùng nhấn nút Stop ở Layer1 để dừng việc sử dụng webcam. Web thực hiện sự kiện gửi API về server để yêu cầu tắt webcam. Server nhận được yêu cầu và thực hiện xử lý tắt webcam. Đồng thời, hệ thống sẽ xoá biến webcam để tiết kiệm tài nguyên và gửi tín hiệu đã tắt về cho web. Web sẽ nhận được tín hiệu và xử lý để tắt Layer2 và chuyển trạng thái của nút từ Stop sang Start cho lần sử dụng sau.



Hình 3.59. Sơ đồ xử lý tắt webcam client

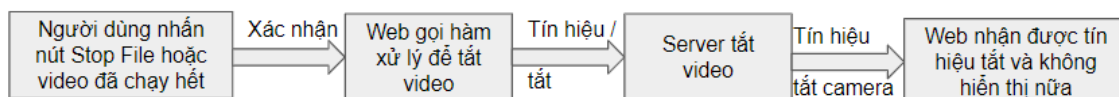
### 3.4.2.3. Chọn/Tắt file video đã chọn

**Bật File video** (hình 3.60): Người dùng nhấn nút File ở Layer1, web sẽ thực hiện sự kiện mở hộp thoại cho phép chọn video từ máy tính của người dùng. Khi người dùng chọn xong video, web sẽ kiểm tra xem có video khác đang chạy trên Layer 2 hay không. Nếu có, web sẽ tắt video đó và hiển thị video đã chọn trên Layer 2. Nút File sẽ chuyển sang trạng thái Stop File.



Hình 3.60. Sơ đồ xử lý mở file

**Tắt File video** (hình 3.61): Người dùng có thể nhấn nút Stop File ở Layer1 hoặc tự động khi video đã chạy hết. Web sẽ thực hiện sự kiện tắt video đang chạy trên Layer 2 và chuyển nút từ Stop File sang File để người dùng có thể chọn video mới.



Hình 3.61. Sơ đồ xử lý tắt file

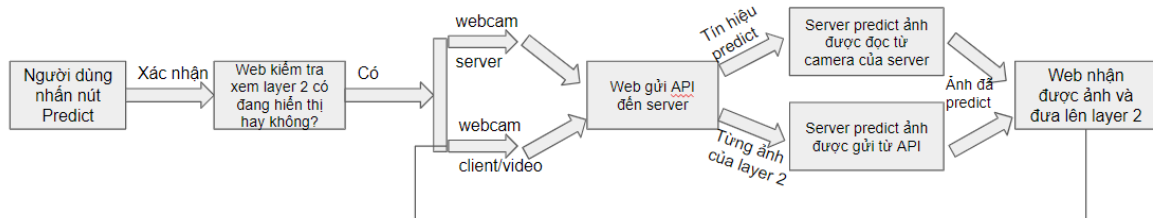
### 3.4.2.4. Dự đoán kết quả trên thời gian thực

**Dự đoán mô hình học sâu (nút Predict)** được mô tả như hình 3.62, cụ thể:

- Khi người dùng nhấn vào nút "Predict" trong Layer 1, hệ thống sẽ kiểm tra xem Layer 2 có đang hiển thị bất kỳ nội dung nào không. Nếu không, hệ thống sẽ không thực hiện thao tác nào cả. Nếu có, hệ thống sẽ tiếp tục kiểm tra xem video đang được hiển thị trên Layer 2 là webcam của server hay là webcam hoặc một video tải lên từ máy người dùng.
- Nếu video đang được phát là webcam của server, hệ thống sẽ gửi một yêu cầu API đến server để tiến hành dự đoán trên video đang được phát trên

Layer 2. Server sẽ tiến hành dự đoán trong quá trình đọc hình ảnh từ camera và sau đó gửi kết quả dự đoán về cho hệ thống web để hiển thị lên Layer 2.

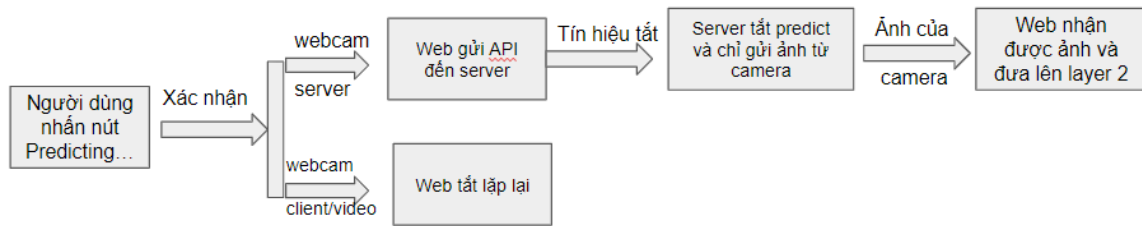
- Nếu video đang được phát là webcam hoặc một video tải lên từ máy người dùng, hệ thống vẫn sẽ gửi một yêu cầu API đến server, nhưng lúc này yêu cầu sẽ chứa các hình ảnh từng khung của video đang được phát trên Layer 2. Server sẽ tiến hành dự đoán trên mỗi hình ảnh và sau đó gửi kết quả dự đoán về cho hệ thống web để hiển thị lên Layer 2.
- Nút “Predict ”cũng sẽ được chuyển thành “Predicting...” để người dùng có thể dừng quá trình predict.



Hình 3.62. Sơ đồ xử lý khi người dùng gửi yêu cầu dự đoán bằng mô hình học sâu

**Tắt dự đoán** được mô tả như hình 3.63, cụ thể:

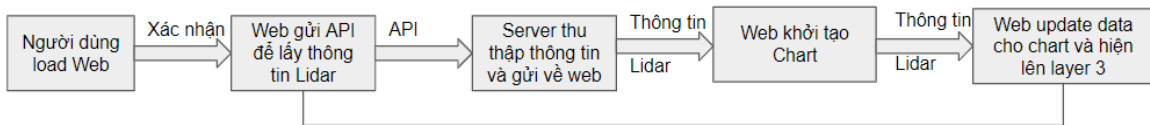
- Khi người dùng muốn dừng quá trình dự đoán, họ chỉ cần nhấn vào nút "Predicting...". Hệ thống sẽ tiếp tục kiểm tra xem quá trình dự đoán đang được thực hiện trên webcam của server hay webcam của client hoặc là một file video.
- Nếu quá trình dự đoán đang được thực hiện trên webcam của server, hệ thống sẽ gửi một yêu cầu API đến server để thông báo dừng dự đoán. Khi server nhận được thông báo này, nó sẽ dừng quá trình dự đoán và chỉ gửi hình ảnh chưa được dự đoán từ webcam của server về cho hệ thống web để tiếp tục hiển thị trên Layer 2.
- Nếu quá trình dự đoán đang được thực hiện trên webcam của client hoặc là một file video, hệ thống chỉ cần dừng việc gửi yêu cầu API liên tục tới server.
- Nút "Predicting..." sẽ được chuyển thành "Predict" để người dùng có thể thực hiện dự đoán lại nếu muốn.



Hình 3.63. Sơ đồ xử lý khi người dùng gửi yêu cầu tắt dự đoán bằng mô hình học sâu

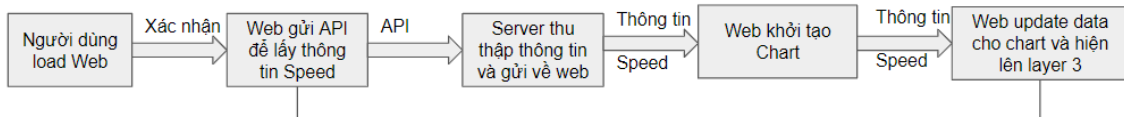
#### 3.4.2.5. Hiện thị biểu đồ Lidar và Speed theo thời gian thực

**Hiện thị biểu đồ LiDAR** (hình 3.64): Khi người dùng truy cập vào trang web, hệ thống sẽ tự động gửi một yêu cầu API đến server để lấy dữ liệu Lidar. Server sẽ tiến hành thu thập dữ liệu và trả lại cho trang web. Nếu nhận được dữ liệu Lidar từ server, hệ thống sẽ hiển thị dữ liệu này thông qua một biểu đồ scatter plot trên Layer 3 để giúp người dùng dễ dàng hiểu và theo dõi dữ liệu. Quá trình này sẽ được diễn ra liên tục mỗi giây một lần để đảm bảo tính thời gian thực của dữ liệu.



Hình 3.64. Sơ đồ xử lý hiện thị biểu đồ LiDAR

**Hiện thị biểu đồ SPEED** (hình 3.65): Khi người dùng truy cập vào trang web, hệ thống sẽ tự động gửi một yêu cầu API đến server để lấy dữ liệu Speed. Server sẽ tiến hành thu thập dữ liệu và trả lại cho trang web. Nếu nhận được dữ liệu Speed từ server, hệ thống sẽ hiển thị dữ liệu này thông qua một biểu đồ line plot trên Layer 3 để giúp người dùng dễ dàng hiểu và theo dõi dữ liệu. Quá trình này sẽ được diễn ra liên tục mỗi giây một lần để đảm bảo tính thời gian thực của dữ liệu.

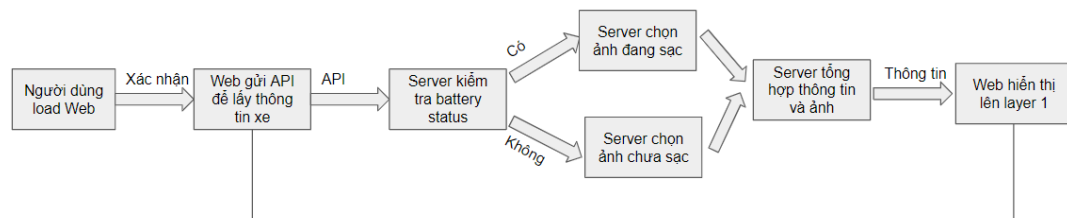


Hình 3.65. Sơ đồ xử lý hiện thị biểu đồ SPEED

#### 3.4.2.6. Cập nhật thông tin của xe theo thời gian thực

Cập nhật thông tin của xe theo thời gian thực được mô tả như hình 3.66, cụ thể khi người dùng truy cập vào trang web, hệ thống sẽ tự động gửi một yêu cầu API

đến server để lấy thông tin về xe. Server sẽ kiểm tra trạng thái pin của xe và chọn một hình ảnh đại diện cho trạng thái đó. Tiếp theo, server sẽ tiến hành thu thập các thông tin khác về xe và gửi tất cả đến trang web. Khi nhận được thông tin về xe, trang web sẽ hiển thị tất cả thông tin lên Layer 1. Quá trình này sẽ được diễn ra liên tục mỗi giây một lần để đảm bảo tính thời gian thực của thông tin.



Hình 3.66. Sơ đồ xử lý cập nhật thông tin của xe

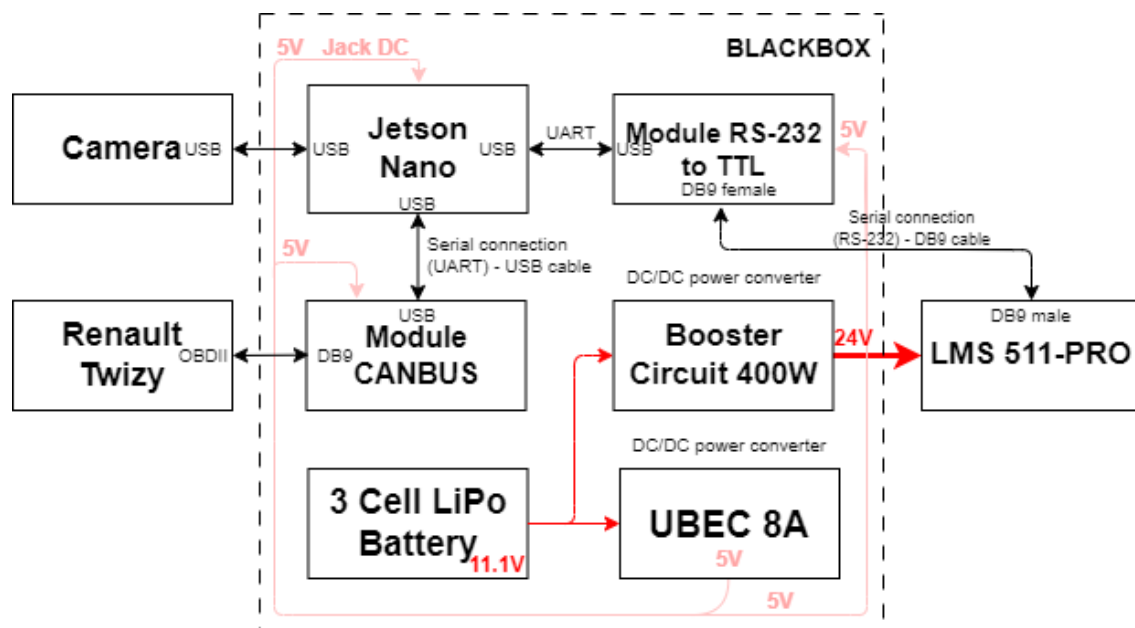
## CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM

### 4.1. Môi trường đánh giá

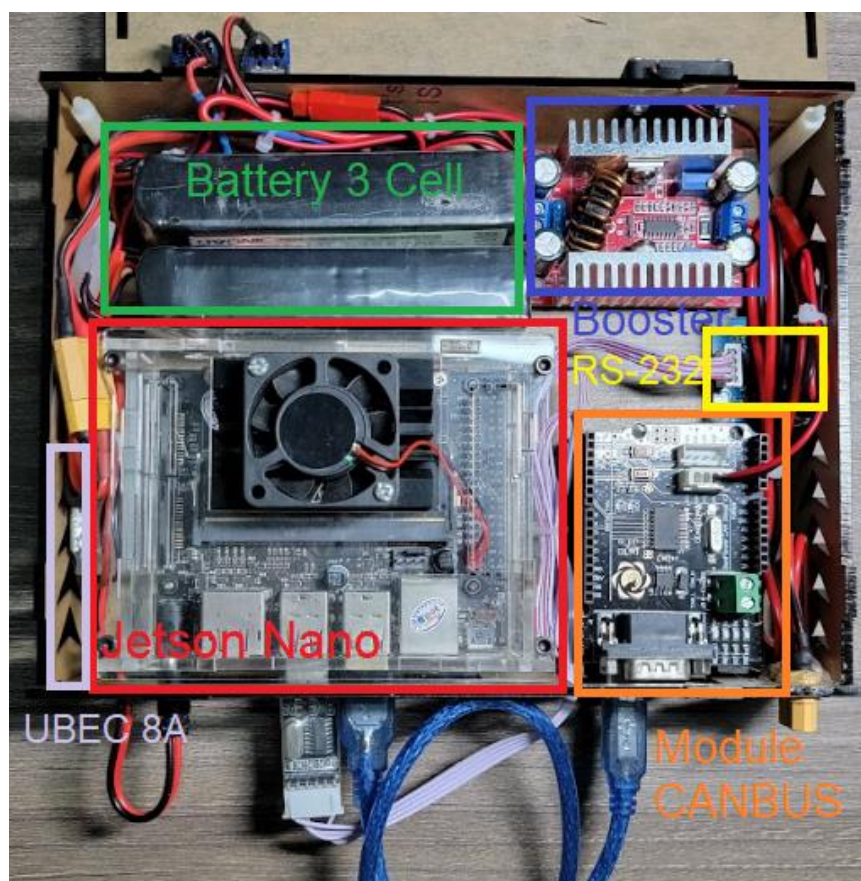
Tất cả thực nghiệm được chạy tại “Khuôn viên trường Đại học Sư phạm Kỹ thuật Thành phố Hồ Chí Minh” trên phần cứng và phần mềm như bảng 4.1 với sơ đồ mạch điện như hình 4.1 và sơ đồ phần cứng như hình 4.2.

Bảng 4.1. Thống kê phần cứng và phần mềm của các tác vụ

| Tác vụ           | Mô hình học sâu                                                    | Phát hiện vật cản (LiDAR)                                            | Thông tin trạng thái (CANBUS)                           |
|------------------|--------------------------------------------------------------------|----------------------------------------------------------------------|---------------------------------------------------------|
| <b>Phần cứng</b> | Jetson Nano<br>Camera                                              | LMS 511-PRO<br>Module RS-232 to TTL (Module MAX3232 + Module CP2102) | Module CANBUS (Arduino Uno R3 + CAN Bus Shield MCP2515) |
| <b>Phần mềm</b>  | SDK Jetpack 4.6.1<br>Python 3.8<br>Pytorch 1.8.1<br>TensorRT 8.2.1 | SOPAS 3.3.3<br>Pyserial 3.5                                          | Arduino IDE 1.8.12                                      |



Hình 4.1. Sơ đồ mạch điện hệ thống



Hình 4.2. Sơ đồ phân cứng hệ thống

## 4.2. Kết quả mô hình học sâu

Để chứng minh được hiệu quả của việc huấn luyện mô hình đa tác vụ so với từng tác vụ. Chúng tôi so sánh mô hình của mình với **BackBone** và **Neck** lần lượt là **ResNet101** và **BiFPN** khi huấn luyện đa tác vụ và từng tác vụ. Bảng 4.2 thể hiện sự so sánh này. Có thể thấy, phương pháp đa tác vụ đạt kết quả vượt trội so với phương pháp một tác vụ, quan trọng hơn mô hình đa tác vụ có thể tiết kiệm rất nhiều thời gian, hơn 40 khung hình trên giây so với việc thực thi từng tác vụ.

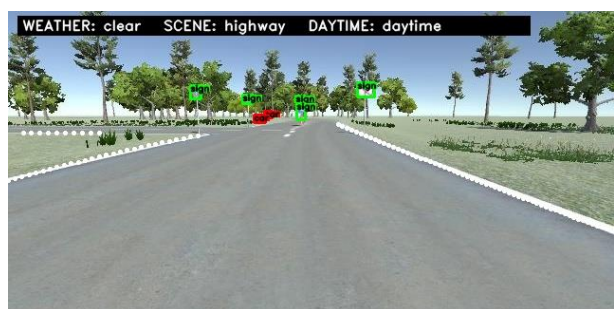
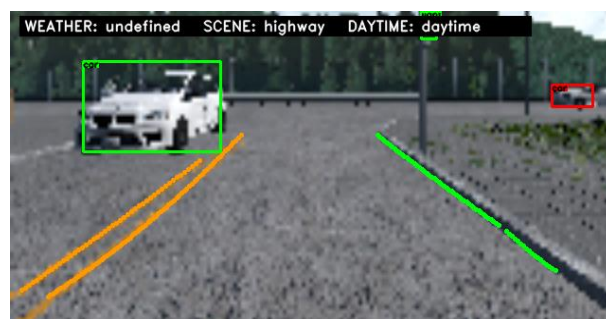
Bảng 4.2. So sánh giữa việc học đa tác vụ và học từng tác vụ

| Mô hình                            | mAP (%)     | IoU (%)     | FPS(RTX 3090) |
|------------------------------------|-------------|-------------|---------------|
| Detection(only)                    | 88.4        | -           | 243.90        |
| Segmentation(only)                 | -           | 94.8        | 49.63         |
| Detection+Segmentation             | 88.4        | 94.8        | 71.82         |
| <b>Multitask (ResNet101+BiFPN)</b> | <b>88.1</b> | <b>95.0</b> | <b>110.56</b> |

Ngoài ra, việc sử dụng mô hình của chúng tôi với **BackBone** và **Neck** lần lượt là **MobilenetV2** và **BiFPN** có thể triển khai được trên Jetson Nano với độ chính xác không giảm nhiều được thể hiện trong bảng 4.3, cụ thể với **mAP** là **83%** và **IoU** là **93.3%**. Hơn thế nữa có thể phân nhánh cho mô hình “*phân loại điều kiện lái xe*” với **F1-Score** của Weather, Scene, Time of Day lần lượt là **0.813**, **0.790**, **0.934** mà Jetson Nano vẫn có thể chạy ổn định ở **14.63 FPS**. Hình 4.3 thể hiện trực quan đa tác vụ về phát hiện đối tượng nhận dạng biển báo giao thông và phân đoạn làn đường.

Bảng 4.3. So sánh các mô hình học đa tác vụ triển khai trên Jetson Nano

| Backbone            | Neck         | Object Detection (mAP) | Lane Segment (IoU) | Scene Classification (F1) |              |              | FPS (Jetson) |
|---------------------|--------------|------------------------|--------------------|---------------------------|--------------|--------------|--------------|
|                     |              |                        |                    | Weather                   | Scene        | Time of Day  |              |
| Resnet101           | BiFPN        | 0.881                  | 0.950              | 0.825                     | 0.794        | 0.934        | -            |
| <b>Mobilenet V2</b> | <b>BiFPN</b> | <b>0.830</b>           | <b>0.933</b>       | <b>0.813</b>              | <b>0.790</b> | <b>0.934</b> | <b>14.63</b> |

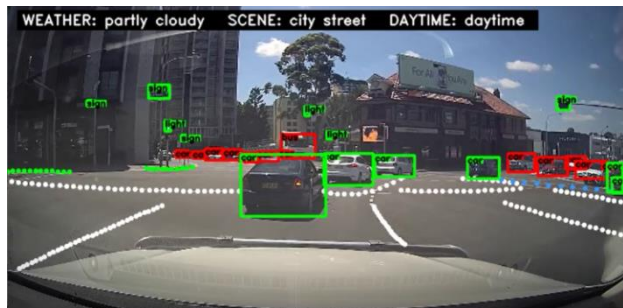
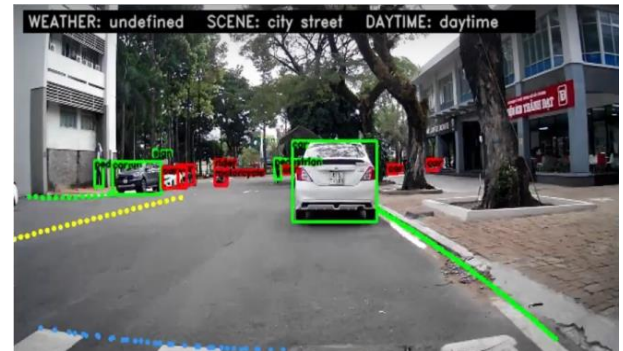
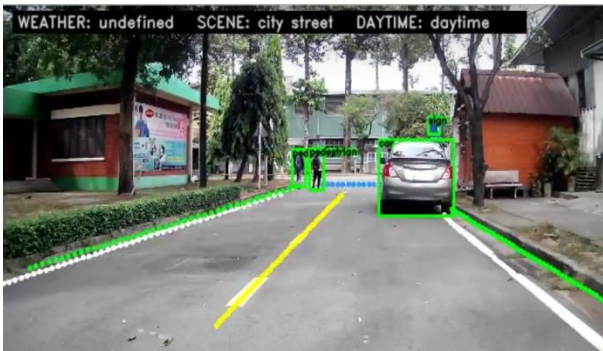
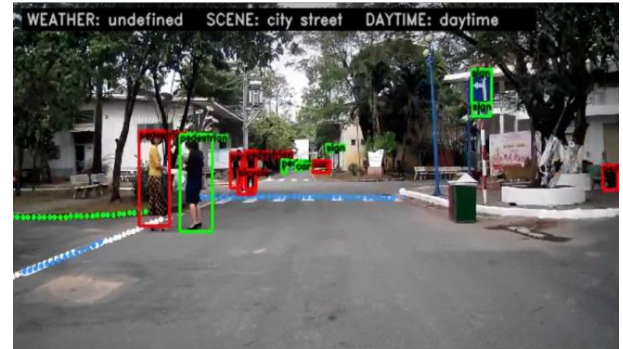


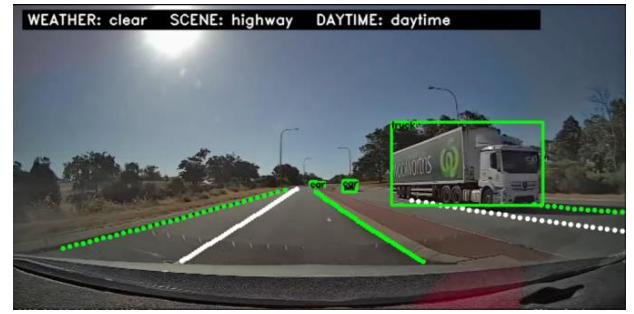
a) Kết quả khi đánh giá mô hình trên phần mềm mô phỏng





b) Kết quả khi đánh giá mô hình trên xe mô hình



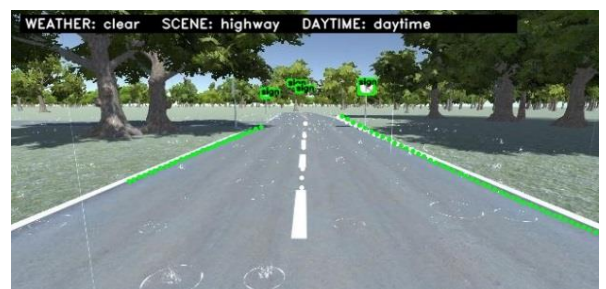
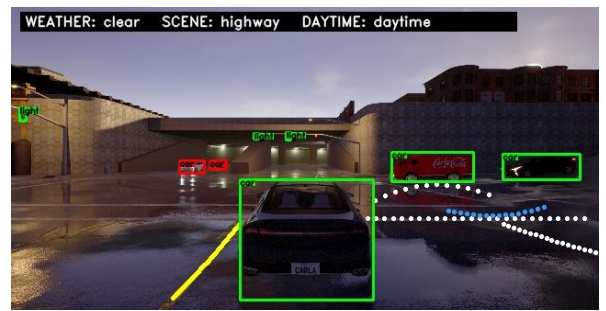
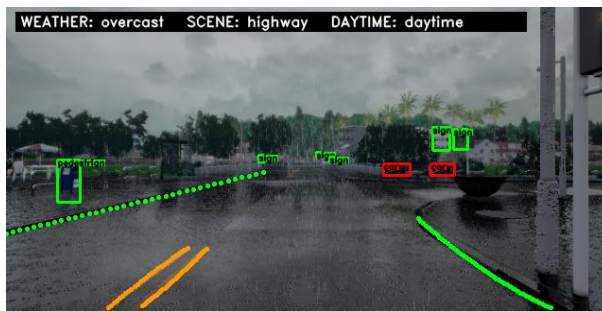


c) Kết quả khi đánh giá mô hình trên thực tế

Hình 4.3. Kết quả của mô hình học đa tác vụ

Thực nghiệm trên các môi trường thách thức

Như trong hình 4.4, chúng tôi cũng thử nghiệm mô hình của mình trong các điều kiện thời tiết và ánh sáng thử thách như ban ngày, ban đêm, ngày mưa, tình hình giao thông đông đúc, ... Kết quả trực quan chứng minh rằng mô hình của chúng tôi hoạt động tốt trong mọi điều kiện kể cả điều kiện khó.



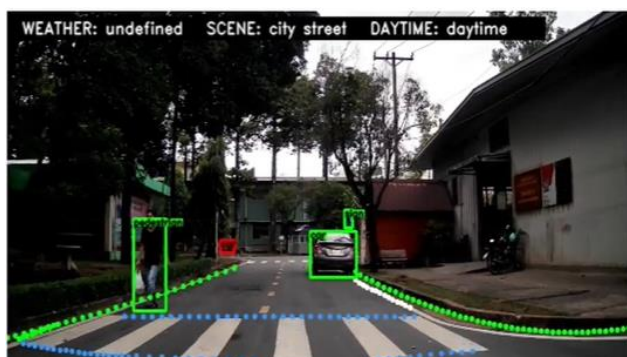


Hình 4.4. Kết quả của mô hình ở điều kiện xấu

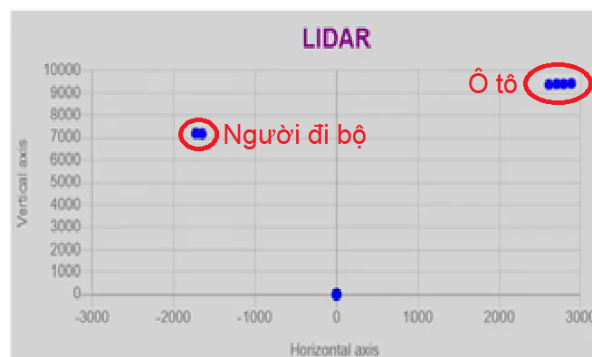
### 4.3. Kết quả LIDAR

Thiết bị LiDAR LMS 511-PRO hoạt động khá tốt nhờ việc sử dụng các thuật toán lọc nhiễu từ nhà sản xuất, thu được dữ liệu xung quanh bằng tia laser và phản hồi tương đối chính xác so với thực tế. Tốc độ lấy dữ liệu cũng tốt với chu kỳ quét 0.02s và thời gian gửi dữ liệu lên hệ thống là 0.035s, tổng thời gian mất là **0.055s** tương đương với 18 FPS, đảm bảo đủ tốt cho các ứng dụng chạy thời gian thực.

Ngoài ra, với mức độ tin cậy cao của LiDAR đã ứng dụng được cho mô hình học sâu tăng độ chính xác (tăng confident, tái tạo lại những đối tượng bị mất) trong các điều kiện thách thức như trời tối, trời mưa,... được thể hiện trong hình 4.5.



a) Kết quả mô hình học sâu



b) Kết quả LiDAR

Hình 4.5. Kết quả nhiệm vụ phát hiện vật cản

#### 4.4. Kết quả thu thập thông số xe

Với hệ thống truyền nhận và mã hóa tín hiệu CAN có tốc độ truyền tải Baudrate nằm ở mức **1M Bytes** trên giây (BPS) thời gian tính toán xác đáng **0.001** mili giây (ms) và tốc độ truyền dữ liệu từ MCU Arduino Uno đến Jetson Nano là **115200** Bytes trên giây với kích thước khung dữ liệu là 15 Bytes tương đương với thời gian thực là **1.3** mili giây (ms). Tín hiệu mạng CAN được truyền đến thiết bị máy chủ nhận từ MCU mất **1.0** mili giây (ms) để hoàn thành. Toàn hệ thống truyền nhận, xử lý tín hiệu và mã hóa CAN đến thiết bị cuối chỉ tiêu tốn **2.3** mili giây (ms) đảm bảo tính chính xác theo thời gian thực trên UI người dùng.

Với hệ thống truyền nhận và mã hóa tín hiệu CAN có tốc độ truyền tải Baudrate nằm ở mức 1M Bytes trên giây (BPS) thời gian tính toán xác đáng 0.001 mili giây (ms) và tốc độ truyền dữ liệu từ MCU Arduino Uno đến Jetson Nano là 115200 Bytes trên giây với kích thước khung dữ liệu là 15 Bytes tương đương với thời gian thực là 1.3 mili giây (ms). Tín hiệu mạng CAN được truyền đến thiết bị máy chủ nhận từ MCU mất 1.0 mili giây (ms) để hoàn thành. Toàn hệ thống truyền nhận, xử lý tín hiệu và mã hóa CAN đến thiết bị cuối chỉ tiêu tốn 2.3 mili giây (ms) đảm bảo tính chính xác theo thời gian thực trên UI người dùng.

Chu trình CAN gửi lên các gói dữ liệu ID theo thứ tự giai đoạn thời gian như trong bảng 4.4. Tôi xin trình bày ở đây một số gói dữ liệu ID đã được giải thể hiện trong hình 4.6.

Bảng 4.4. Chu trình gửi lên các gói ID dữ liệu cho mạng CAN

| ID CAN | PERIOD | ID CAN | PERIOD (ms) |
|--------|--------|--------|-------------|
| 155    | 10     | 556    | 100         |
| 196    | 10     | 557    | 1000        |
| 19F    | 10     | 599    | 100         |
| 59B    | 100    | 55E    | 1000        |
| 424    | 100    | 55F    | 1000        |
| 425    | 100    | -      | -           |

```

Get data from ID: 19F
255  255  125  15   56   255  64   254
-----
Speed of Motor: 0 Rpm
Speed of Vehicle: 0.00 Km/h

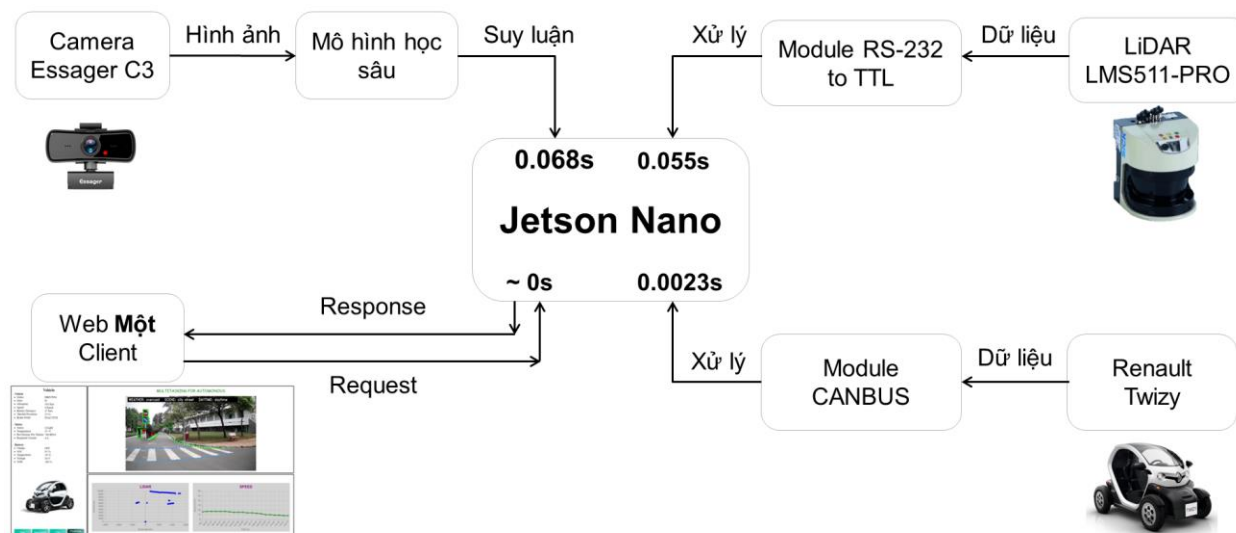
Get data from ID: 196
0     255  231  127  112  89   11   0
-----
Motor Temperature: 49 (C)

```

Hình 4.6. Gói dữ liệu ID 0x19F và ID 0x196 xử lý, mã hóa và xuất ra Monitor

## 4.5. Tổng kết toàn hệ thống

Với việc Jetson Nano cung cấp 472 GFLOPS để chạy các thuật toán AI hiện đại cùng với một CPU quad-core 64-bit ARM , một NVIDIA GPU 128-core được tích hợp trên board mạch. Chúng ta có thể lập trình đa luồng cho phép các tác vụ chạy song song trên cùng một hệ thống, giúp tăng tốc độ xử lý và giảm thời gian chờ, các luồng có thể chia sẻ tài nguyên, dữ liệu và được tổng hợp trên UI. Vì vậy nên thời gian mỗi lần xử lý chỉ tính cho tác vụ tốn nhiều chi phí tính toán nhất cụ thể là tác vụ mô hình học sâu như hình 4.7.



Hình 4.7. Chi phí tính toán của hệ thống

Thông qua việc khảo sát, đánh giá, tối ưu qua từng giai đoạn nghiên cứu và kiểm soát tốt tốc độ xử lý, chúng tôi đã xây dựng được hệ thống (hình 4.8) đáp ứng được những yêu cầu đề ra và cũng đã phát triển một giao diện người dùng dễ sử dụng và dễ hiểu, nhằm giúp người dùng nhanh chóng và dễ dàng kiểm soát thông tin với hộp đen như hình 4.9.



Hình 4.8. Hệ thống đa tác vụ ứng dụng cho xe tự hành



Hình 4.9. Phần cứng hệ thống đa tác vụ ứng dụng cho xe tự hành

## CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

### 5.1. Kết quả

Thông qua các kết quả đạt được ở trên, hệ thống có khả năng xử lý các tác vụ phát hiện đối tượng, phân đoạn làn đường, phân loại điều kiện lái xe, phát hiện vật cản và thông tin trạng thái của xe một cách đồng thời. Giải quyết được vấn đề của các hệ thống xe tự hành trước đây là xử lý riêng lẻ các nhiệm vụ và thiếu thông tin hệ thống. Ngoài ra chúng tôi cũng hoàn thành mã nguồn huấn luyện và chạy thực nghiệm. Hệ thống đạt được độ chính xác khá tốt, đảm bảo tính chính xác khi sử dụng.

Tuy vậy, nghiên cứu này vẫn còn nhiều hạn chế cần giải quyết:

- Thứ nhất chính là mô hình chỉ mới được huấn luyện trên bộ data mô phỏng, mặc dù dữ liệu có nhiều thử thách nhưng vì môi trường thử nghiệm khá đơn giản nên vẫn còn nhiều thách thức chưa biết để giải quyết.
- Mặc dù, chúng tôi đã chứng minh được hiệu quả của việc **“THIẾT KẾ HỆ THỐNG ĐA TÁC VỤ TRIỂN KHAI TRÊN JETSON NANO ỨNG DỤNG CHO XE TỰ HÀNH”**, nhưng nhóm chưa ứng dụng được các thuật toán điều khiển xe để xe có thể lái tự động.

### 5.2. Hướng phát triển

- Phát triển hệ thống lên các phần nhúng mạnh hơn như Jetson Tx2, Jetson Xavier để có thể tăng độ chính xác cho mô hình.
- Tăng cường thêm dữ liệu thực tế để giúp mô hình học được đa dạng hơn.
- Tăng cường khả năng điều khiển tự động và tự học của xe, để giảm sự can thiệp của người lái và tăng tỷ lệ hoạt động tự động của xe.
- Phát triển công nghệ chẩn đoán và phòng ngừa tai nạn, bao gồm cảm biến và hệ thống định vị để giúp xe điều khiển và phòng ngừa va chạm.
- Phát triển hệ thống kết nối và trao đổi dữ liệu giữa xe và các thiết bị điện tử khác để tăng cường tính năng hỗ trợ điều khiển và giảm tổn thất tai nạn.
- Phát triển công nghệ sạc pin và tăng cường khả năng sử dụng năng lượng.

## TÀI LIỆU THAM KHẢO

- [1] L.-Y. J. P.-L. H. & B.-C. W. Yu-Kuen Tsai, Implementation of autonomous vehicles with the hough transform and fuzzy control, SICE Annual Conference, 2007.
- [2] A. K. R. C. S. M. Vijay Badrinarayanan, "SegNet: A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation," arXiv: 1511.100561v3, 2016.
- [3] V. Shrimali, ""learnopencv," [Online], 2019. [Online]. Available: <https://learnopencv.com/pytorch-for-beginners-basics/>.
- [4] N. Thoại, "Machine learning là gì?," 2015. [Online]. Available: <https://longvan.net/machine-learning-la-gi-ung-dung-cua-machine-learning.html>.
- [5] ITNAVI, "Deep learning là gì? Khái quát kiến thức về Deep learning," 2020. [Online]. Available: <https://itnavi.com.vn/blog/deep-learning-la-gi>.
- [6] T. A. M. S. A.-Z. Saad Albawi, "Understanding of a convolution neural network," International Conference on Engineering and Technology (ICET), 2017.
- [7] Saha, "A Comprehensive Guide to Convolutional Neural Networks — the ELI5 way," 2018. [Online]. Available: <https://towardsdatascience.com/a-comprehensive-guide-toconvolutional-neural-networks-the-eli5-way-3bd2b1164a53>.
- [8] ITechSeeker, "Convolution Neural Network," 2022. [Online]. Available: <http://itechseeker.com/tutorials/computer-vision-with-deep-learning/ly-thuyet-chung/convolutional-neural-network/>.
- [9] D. S. Senthamliarasu, "Zero – padding," 2017. [Online]. Available: <https://subscription.packtpub.com/book/webdevelopment/9781838646301/8/ch08lv11sec21/zero-padding>.
- [10] N. Ahmad, "Convolutional Neural Networks," 2021. [Online]. Available: <https://buffml.com/convolutional-neural-networks/>.
- [11] P. Mahajan, "Max Pooling," 2020. [Online]. Available: <https://poojamahajan5131.medium.com/max-pooling-210fc94c4f11>.
- [12] S. A. Afshine Amidi, "Convolutional Neural Networks," [Online]. Available: <https://stanford.edu/~shervine/l/vi/teaching/cs-230/cheatsheet-convolutional-neural-networks>.

- [13] B. Q. Manh, "ShuffleNet – Deep Network dành cho thiết bị Mobile," 2020. [Online]. Available: <https://viblo.asia/p/shufflenet-deep-network-danh-cho-thiet-bi-mobile-ORNZq1P8Z0n>.
- [14] R. Gandhi, "R-CNN, Fast R-CNN, Faster R-CNN, YOLO – Object Detection Algorithms," Towards Data Science, 2018.
- [15] S. D. R. G. a. A. F. J. Redmon, "You only look once: Unified, real-time object detection," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016.
- [16] D. A. D. E. C. S. S. R. C.-Y. F. A. C. B. Wei Liu, "SSD: Single Shot MultiBox Detector," 2016.
- [17] RangiLyu, "Nanodet -Plus [source code]," 2022. [Online]. Available: <https://github.com/RangiLyu/nanodet>.
- [18] J. G. L. W. S. S. a. X. Z. Tailin Liang, "Pruning and Quantization for Deep Neural Network Acceleration: A Survey," arXiv:2101.09671, 2021.
- [19] C.-Y. a. B. A. a. L. H.-Y. M. Wang, "Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors,," arXiv:2207.02696, 2022.
- [20] C.-Y. W. I.-H. Y. a. H.-Y. M. Liao, "You Only Learn One Representation: Unified Network for Multiple Tasks," arXiv: 2105.04206, 2021.
- [21] R. -I. T. a. F. Q. G. Jin, "Auxiliary Detection Head for One-Stage Object Detection," IEEE Access.
- [22] P. D. Khanh, "Các thuật toán Object Detection," 2019. [Online]. Available: <https://phamdinhkhanh.github.io/2019/09/29/OverviewObjectDetection.html>.
- [23] C. S. H. C. T. H. Zhi Tian, "FCOS: Fully Convolutional One-Stage Object Detection," arXiv:1904.01355v5, 2019.
- [24] C. C. Y. Y. Z. L. S. Z. L. Shifeng Zhang, Bridging the Gap Between Anchor-based and Anchor-free Detection via Adaptive Training Sample Selection, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2020.
- [25] S. L. F. W. Z. L. J. S. Zheng Ge, "YOLOX: Exceeding YOLO Series in 2021," arXiv: 2107.08430, 2021.
- [26] P. D. Khanh, "Image Segmentation," 2020. [Online]. Available: <https://phamdinhkhanh.github.io/2020/06/10/ImageSegmemention.htm>.
- [27] P. F. a. T. B. O. Ronneberger, "U-net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention – MICCA*, 2015.

- [28] A. C. S. K. E. C. Adam Paszke, "ENet: A Deep Neural Network Architecture for Real-Time Semantic Segmentation," arXiv:1606.02147, 2016.
- [29] S. L. N. a. E. A. d. S. Rafael Padilla, "A Survey on Performance Metrics for Object-Detection Algorithms," in *Proceedings of the IEEE conference on Systems, Signals and Image Processing*, 2020.
- [30] G. G. P. D. a. R. G. K. He, "Mask R-CNN," *Proceedings of the IEEE International Conference on Computer Vision*, 2017.
- [31] J. W. C. P. C. G. G. Y. N. S. Changqian Yu, "BiSeNet: Bilateral Segmentation Network for Real-time Semantic Segmentation," arXiv:1808.00897, 2018.
- [32] J. S. F. J. A. Cook, "Controller Area Network (CAN)," 2008. [Online]. Available: CAN\_notes.pdf (umich.edu).
- [33] M. G. L. Eric Peña, "UART: A Hardware Communication Protocol Understanding Universal Asynchronous Receiver/Transmitter," [Online]. Available: <https://www.analog.com/en/analog-dialogue/articles/uart-a-hardware-communication-protocol.html>.
- [34] S. Campbell, "BASICS OF THE SPI COMMUNICATION PROTOCOL," [Online]. Available: <https://www.circuitbasics.com/basics-of-the-spi-communication-protocol/>.
- [35] Geeksforgeeks, "Frontend vs Backend," 2023. [Online]. Available: <https://www.geeksforgeeks.org/frontend-vs-backend/>.
- [36] Kumaresan, "Senmatic Segmentation For Self Driving Car," 2018. [Online]. Available: Retrieved from <https://www.kaggle.com/datasets/kumaresanmanickavelu/lyft-udacity-challenge>.
- [37] C. P. T. Dũng, "CEEC - team Unity," 2021. [Online]. Available: <https://www.youtube.com/watch?v=FrFEhCfoPPQ&list=PLJ2iHFzHKEfkmYd-dz0Hi9bGJMXiLUD9L>.
- [38] Ultralytics, "yolov5 [source code]," 2020. [Online]. Available: <https://github.com/ultralytics/yolov5>.
- [39] L. L. H. J. K. W. Y. G. L. L. Z. K. Q. L. M. C. W. N. Y. L. B. Z. Y. L. L. Z. X. X. X. C. X. W. X. W. Chuyi Li, "YOLOv6: A Single-Stage Object Detection Framework for Industrial Applications," arXiv:2209.02976, 2022.
- [40] X. Z. S. R. a. J. S. K. He, "Deep residual learning for imagerecognition," *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016.

- [41] A. H. M. Z. A. Z. a. L.-C. C. M. Sandler, "Mobilenetv2: Inverted residuals and linear bottlenecks," Proceedings of the IEEE conference on computer vision and pattern recognition, 2018.
- [42] M. T. a. Q. Le, Efficientnet: Rethinking model scaling for convolutional neural networks, International conference on machine learning. PMLR, 2019.
- [43] D. W. E. S. a. T. D. F. Yu, "Deep layer aggregation," Proceedings of the IEEE conference on computer vision and pattern recognition, 2018.
- [44] J. D. H. S. J. K. S. S. S. M. Z. H. A. K. A. K. M. B. e. a. O. Russakovsky, "Imagenet large scale visual recognition challenge," International journal of computer vision, 2015.
- [45] R. P. a. Q. V. L. M. Tan, "Efficientdet: Scalable and efficient object detection," Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, 2020.
- [46] P. D. R. G. K. H. B. H. a. S. B. T.-Y. Lin, "Feature pyramid networks for object detection," Proceedings of the IEEE conference on computer vision and pattern recognition, 2017.
- [47] L. Q. a. Q. Z.-B. Ma Yi-de, "Automated image segmentation using improved pcnn model based on cross-entropy," Proceedings of 2004 International Symposium on Intelligent Multimedia, Video and Speech Processing, 2004.
- [48] J. B. Diederik P. Kingma, "Adam: A method for Stochastic optimization," arXiv:1412.6980, 2014.
- [49] M. M. S. B. J. H. P. P. D. R. P. D. a. C. L. Z. T.-Y. Lin, "'Microsoft coco: Common objects in context," European conference on computer vision. Springer, 2014.
- [50] S. AG, "Operating instructions," [Online]. Available: LMS5xx LMS511-10100 PRO, Product .
- [51] S. AG, "Technical information," [Online]. Available: [https://cdn.sick.com/media/docs/7/27/927/technical\\_information\\_telegram\\_listing\\_ranging\\_sensors\\_lms1xx\\_lms5xx\\_tim2xx\\_tim5xx\\_tim7xx\\_lms1000\\_mrs1000\\_mrs6000\\_nav310\\_ld\\_oem15xx\\_ld\\_lrs36xx\\_lms4000\\_lrs4000\\_en\\_im0045927.pdf](https://cdn.sick.com/media/docs/7/27/927/technical_information_telegram_listing_ranging_sensors_lms1xx_lms5xx_tim2xx_tim5xx_tim7xx_lms1000_mrs1000_mrs6000_nav310_ld_oem15xx_ld_lrs36xx_lms4000_lrs4000_en_im0045927.pdf).
- [52] Lulu, "Most common baud rates table," [Online]. Available: <https://lucidar.me/en/serialib/most-used-baud-rates-table/>.
- [53] Arduino, "ARDUINO UNO R3," [Online]. Available: A000066-datasheet.pdf (arduino.cc).
- [54] Seeedstudio, "CAN-BUS Shield V1.2," [Online]. Available: [https://wiki.seeedstudio.com/CAN-BUS\\_Shield\\_V1.2/](https://wiki.seeedstudio.com/CAN-BUS_Shield_V1.2/).
- [55] RAREblog, "Five steps to connect Jetson Nano and Arduino," 2019. [Online]. Available:

<https://blog.rareschool.com/2019/05/five-steps-to-connect-jetson-nano-and.html>.

## PHỤ LỤC

**Code:** <https://github.com/dotrannhattuong/Multitasking-For-Autonomous>

**Videos:** <https://www.youtube.com/playlist?list=PLxat3wSWURMU8Pe8BAXZcmJvOMWQ2ppxr>